



Bachelor-Thesis

Entwicklung einer Observability-Strategie für das Audience Response System "frag.jetzt" basierend auf dem Konzept der Four Golden Signals.

zur Erlangung des akademischen Grades

Bachelor of Science

im dualen Studiengang Informatik an der IU Internationale Hochschule

von

Leon Rausch

Matrikelnummer: 102201482

31. März 2026

Referent: Prof. Dr. Klaus Quibeldey-Cirkel

Korreferent: Prof. Dr. Philipp Diebold

Kurzfassung

Die Open-Source-Plattform frag.jetzt ermöglicht anonyme Echtzeitfragen in der Hochschullehre. Ihre verteilte Architektur aus mehreren Backend-Diensten, einer relationalen Datenbank und einem Message Broker wurde bislang ohne aggregierte Telemetrierfassung betrieben. Störungen wurden erst wahrgenommen, wenn Endnutzende direkt betroffen waren. Die vorliegende Arbeit entwickelt eine Observability-Strategie, die dieses Defizit adressiert. Ausgehend von einer Systemanalyse mit sechs identifizierten Risikopunkten und fünf Observability-Anforderungen wird eine servicebezogene Operationalisierung der Four Golden Signals (Latenz, Traffic, Fehler, Sättigung) vorgenommen. Eine kriteriengestützte Evaluation dreier Architekturkandidaten führt zur Auswahl des LGTM-Stacks (Loki, Grafana, Tempo, Mimir/Prometheus) mit OpenTelemetry als Instrumentierungsschicht. Der wesentliche Beitrag der Arbeit liegt in einer methodisch nachvollziehbaren Überführung allgemeiner Observability-Konzepte in eine konkrete, dienstbezogene Strategie für ein reales verteiltes System. Die konzipierte Strategie umfasst eine serviceübergreifende Signal-Matrix, ein rollenbasiertes Dashboard-Konzept, symptomorientierte Alerting-Regeln mit Multi-Signal-Logik sowie strukturierte Runbooks. Die prototypische Umsetzung auf dem Staging-Server von frag.jetzt validiert die Kernmechanismen: Distributed Tracing über den OTel Java Agent, bidirektionale Log-Trace-Korrelation und ein Golden-Signals-Dashboard. Die Evaluation zeigt, dass vier von fünf Anforderungen weitgehend oder vollständig erfüllt werden, während die Ende-zu-Ende-Nachvollziehbarkeit aufgrund unvollständiger Instrumentierungsabdeckung teilweise erfüllt bleibt. Die Arbeit liefert übertragbare methodische Artefakte für vergleichbare cloud-native Plattformen.

Schlüsselwörter: Observability, Four Golden Signals, OpenTelemetry, Distributed Tracing, Alerting, Grafana, Microservices

Abstract

The open-source platform frag.jetzt enables anonymous real-time questions in higher education. Its distributed architecture, comprising multiple backend services, a relational database, and a message broker, had been operated without aggregated telemetry collection. Failures only became visible when end users were already affected. This thesis develops an observability strategy to address this deficit. Starting from a system analysis that identifies six risk points and five observability requirements, the Four Golden Signals (latency, traffic, errors, saturation) are operationalized on a per-service basis. A criteria-driven evaluation of three candidate architectures leads to the selection of the LGTM stack (Loki, Grafana, Tempo, Mimir/Prometheus) with OpenTelemetry as the instrumentation layer. The thesis' main contribution lies in the methodologically transparent transfer of general observability concepts into a concrete service-specific strategy for a real-world distributed system. The resulting strategy comprises a cross-service signal matrix, a role-based dashboard concept, symptom-oriented alerting rules with multi-signal logic, and structured runbooks. A prototypical implementation on the frag.jetzt staging server validates the core mechanisms: distributed tracing via the OTel Java Agent, bidirectional log-trace correlation, and a Golden Signals dashboard. The evaluation demonstrates that four of five requirements are largely or fully met, while end-to-end traceability remains partially fulfilled due to incomplete instrumentation coverage. The thesis contributes transferable methodological artifacts applicable to comparable cloud-native platforms.

Keywords: Observability, Four Golden Signals, OpenTelemetry, Distributed Tracing, Alerting, Grafana, Microservices

Danksagung

Zuallererst danke ich meinen Betreuern **Prof. Dr. Klaus Quibeldey-Cirkel** und **Prof. Dr. Philipp Diebold**, sowie dem Chief Developer von frag.jetzt **Ruben Bimberg** für die fachliche Unterstützung, die wertvollen Hinweise und die Möglichkeit, diese Bachelorarbeit zu verfassen. Ihre offene und konstruktive Kritik hat maßgeblich zu deren Qualität beigetragen.

Mein besonderer Dank gilt auch meinen Mitstudierenden, die durch fachliche Diskussionen und wertvolles Feedback meine Gedanken geschärft haben. Ihre Perspektiven haben mir geholfen, verschiedene Aspekte der Thematik tiefergehend zu beleuchten.

Darüber hinaus möchte ich meiner Familie und meinen Freunden danken, die mir während des gesamten Schreibprozesses moralischen Rückhalt gegeben haben. Ihre Geduld und Ermutigung waren für den Erfolg dieser Arbeit unverzichtbar.

Abschließend danke ich den Entwicklern und der Open-Source-Community für die Bereitstellung der vielfältigen Werkzeuge und Bibliotheken, ohne die die praktische Umsetzung dieses Projekts nicht möglich gewesen wäre.

Hinweis zur geschlechtergerechten Sprache

In dieser Arbeit wird überwiegend eine geschlechtsneutrale Sprache verwendet. Aus Gründen der besseren Lesbarkeit kommt in Einzelfällen jedoch das generische Maskulinum zum Einsatz. Sämtliche Personenbezeichnungen gelten dabei selbstverständlich gleichermaßen für alle Geschlechter.

Die formale Umsetzung orientiert sich an den Empfehlungen des Leitfadens zur gendersensiblen und inklusiven Sprache der IU Internationale Hochschule.

Inhaltsverzeichnis

Kurzfassung	i
Abstract	ii
Danksagung	iii
Hinweis zur geschlechtergerechten Sprache	iv
Abbildungsverzeichnis	ix
Tabellenverzeichnis	x
Listings	xi
Abkürzungsverzeichnis	xii
1 Einleitung	1
1.1 Ausgangssituation und Problemstellung	1
1.2 Zielsetzung und Abgrenzung der Arbeit	2
1.3 Forschungsfragen	2
1.4 Methodisches Vorgehen	3
1.5 Aufbau der Arbeit	3
2 Theoretische Fundierung	4
2.1 Abgrenzung von Monitoring und Observability in verteilten Systemen	4
2.2 Besonderheiten verteilter und cloud-nativer Anwendungen	4
2.3 Telemetriearten und ihr Zusammenspiel: Logs, Metrics und Traces	6
2.4 OpenTelemetry als Standard für herstellerübergreifende Instrumentierung	7
2.5 Die Four Golden Signals als heuristischer Bezugsrahmen	7
2.6 SLIs, SLOs und Error Budgets	8
2.7 Alert Fatigue, Actionable Alerts und Runbooks	8
3 Systemanalyse und Anforderungsdefinition: „frag.jetzt“	10
3.1 Fachlicher Kontext und Nutzungsszenario	10
3.2 Technische Architektur	10
3.3 Kritische User Journeys	12
3.3.1 Journey 1: Frage stellen.	12
3.3.2 Journey 2: Live-Voting.	12
3.3.3 Journey 3: KI-Zusammenfassung.	12

3.4	Risikopunkte und Zuordnung zu den Golden Signals	12
3.4.1	R1: Fehlende Timeouts und Resilienz.	12
3.4.2	R2: Datenbankengpässe bei Massenabstimmungen.	13
3.4.3	R3: Backend als kritischste Schwachstelle ohne Observability.	13
3.4.4	R4: WebSocket-Verbindungsverlust.	13
3.4.5	R5: Fehlende Ressourcenlimits.	13
3.4.6	R6: Single-Host-Topologie als systemweiter Ausfallpfad.	13
3.5	Observability-Ist-Zustand	14
3.6	Ableitung der Observability-Anforderungen	14
3.6.1	A1: Ende-zu-Ende-Nachvollziehbarkeit über Dienstgrenzen.	14
3.6.2	A2: Mehrstufige Messbarkeit entlang der Golden Signals.	14
3.6.3	A3: Strukturiertes, korrelierbares Logging.	14
3.6.4	A4: Handlungsfähige Alarmierung.	15
3.6.5	A5: Beobachtbarkeit der Infrastrukturschicht.	15
4	Methodisches Vorgehen und Bewertungsrahmen	16
4.1	Arbeitslogik der Arbeit	16
4.2	Vorgehen bei der Ableitung der Golden Signals pro Service	16
4.3	Kriterien für Stack-Auswahl und spätere Evaluation	17
4.3.1	Vergleichskriterien für die Stack-Auswahl	17
4.3.2	Bewertungskriterien für die Gesamtstrategie	18
5	Evaluation und Auswahl des Observability-Stacks	19
5.1	OpenTelemetry als verbindliche Instrumentierungsschicht	19
5.2	Vergleich der Zielarchitekturen	19
5.2.1	Elastic Observability	19
5.2.2	Grafana/LGTM-Stack: Prometheus + Loki + Tempo + Grafana	20
5.2.3	Prometheus + Jaeger + Loki + Grafana	20
5.3	Vergleich und Synthese	21
5.4	Begründete Zielentscheidung für frag.jetzt	22
5.4.1	Entscheidungsbegründung im Kontext von frag.jetzt	22
5.4.2	Kompromisse und Einschränkungen	22
6	Konzeption der Observability-Strategie für frag.jetzt	23
6.1	Leitprinzipien der Strategie	23
6.2	Logische Zielarchitektur der Observability	23
6.2.1	Instrumentierungsschicht	24
6.2.2	Sammel- und Verarbeitungsschicht	25
6.2.3	Speicherschicht	25
6.2.4	Analyse- und Visualisierungsschicht	25

6.2.5	Einordnung im Kontext von frag.jetzt	26
6.3	Servicebezogene Operationalisierung der Four Golden Signals	26
6.3.1	PWA-Frontend	26
6.3.2	Spring-Boot-Backend	28
6.3.3	PostgreSQL	28
6.3.4	Externe KI-Dienste	28
6.4	Rollenbasiertes Informations- und Dashboard-Konzept	29
6.5	Alerting-Konzept und Runbook-Logik	29
6.5.1	Alarmkategorien	29
6.5.2	Runbook-Verknüpfung	30
7	Prototypische Implementierung	31
7.1	Umfang und Abgrenzung der prototypischen Umsetzung	31
7.2	Instrumentierung ausgewählter Kernkomponenten	31
7.2.1	Observability-Stack und Telemetrie-Pipeline	31
7.2.2	Applikationsinstrumentierung über den OTel Java Agent	32
7.2.3	Infrastrukturexporter und Scrape-Konfiguration	33
7.2.4	Logging-Anbindung	33
7.2.5	Exemplarische Validierungsszenarien	34
7.3	Umsetzung des Golden-Signals-Dashboards	35
7.4	Alerting-Regeln und Beispiel-Runbooks	36
7.5	Herausforderungen und Lösungen bei der Umsetzung	36
7.5.1	Ressourcendimensionierung	36
7.5.2	Netzwerktopologie und Dienstkommunikation	36
7.5.3	Signalqualität und Rauschunterdrückung	37
8	Evaluation der entwickelten Strategie	38
8.1	Evaluationsmethodik	38
8.2	Anforderungsbezogene Bewertung	38
8.2.1	A1: Ende-zu-Ende-Nachvollziehbarkeit	39
8.2.2	A2: Mehrstufige Messbarkeit entlang der Golden Signals	39
8.2.3	A3: Strukturiertes, korrelierbares Logging	40
8.2.4	A4: Handlungsfähige Alarmierung	40
8.2.5	A5: Beobachtbarkeit der Infrastrukturschicht	42
8.3	Übergreifende Bewertungsdimensionen	42
8.4	Grenzen und methodische Einschränkungen	43
9	Diskussion	44
9.1	Einordnung der Ergebnisse im Verhältnis zur Literatur	44
9.2	Übertragbarkeit auf ähnliche cloud-native Plattformen	45

9.3 Wissenschaftlicher, technischer und operativer Beitrag	45
10 Fazit und Ausblick	47
10.1 Beantwortung der Forschungsfragen	47
10.2 Zentrale Erkenntnisse und übertragbare Artefakte	48
10.3 Ausblick	49
Literaturverzeichnis	51
Index	54
A C4-Container-Diagramm in Vollansicht	54
B OpenTelemetry Collector Konfiguration	55
C Docker Compose Override für die Instrumentierung	57
D Alerting-Regeln	59
D.1 Alert 1: User-facing Latency Degradation	59
D.2 Alert 2: Backend Error Spike	60
D.3 Alert 3: Infrastructure Saturation with Service Impact	61
E Runbooks	63
E.1 Runbook: Erhöhte Backend-Latenz	63
E.2 Runbook: Erhöhte Fehlerquote / HTTP 5xx	65
E.3 Runbook: Infrastruktur-Sättigung (DB / Queue / Host)	67
F Golden-Signals-Dashboard	70
G Verzeichnis der KI-Nutzung	72
G.1 Erklärung zur Nutzung von KI-basierten Werkzeugen	72
G.2 Declaration on the Use of AI-Based Tools	72
G.3 Copilot Prompts für die unterstützende Systemanalyse von frag.jetzt	75

Abbildungsverzeichnis

2.1	Gegenüberstellung von lokalem Komponentenmonitoring und durchgängiger Anfrageverfolgung über mehrere Dienste (Distributed Tracing).	5
3.1	C4-Container-Diagramm der frag.jetzt-Architektur mit Applikationsdiensten, Infrastrukturkomponenten und Kommunikationswegen (Vollseitenansicht des Diagramms in Anhang A).	11
6.1	Logische Zielarchitektur der Observability-Pipeline fuer frag.jetzt	24
7.1	Span-Waterfall eines Backend-Traces (<code>POST /livepoll/find</code>) mit 13 Spans und 15,4ms Gesamtdauer.	33
7.2	Service Graph aus Tempo mit den Kommunikationspfaden <code>user → fragjetzt-backend → fragjetzt-postgres</code> sowie <code>user → fragjetzt-ws-gateway</code>	34
7.3	Ausschnitt des Golden-Signals-Dashboards mit den Sektionen <i>Latency</i> und <i>Traffic</i>	35
7.4	Ausschnitt der <i>Errors</i> -Sektion mit Dummy-Fehlerrate zur Veranschaulichung der Fehlerindikatoren.	35
8.1	Trace-Waterfall in Tempo für den Endpunkt <code>GET /room/{id}/moderator</code> mit 11 Spans und zugeordneten Datenbankoperationen.	39
8.2	Log-to-Trace-Korrelation in Loki: Ein Logeintrag mit klickbarem TraceID-Link (links) navigiert zum zugehörigen Trace-Waterfall in Tempo (rechts).	40
8.3	Latenz-Alert im Zustand <i>Firing</i> nach 6 Minuten. Die Annotations enthalten Symptombeschreibung, Prüfpfad und Runbook-Link.	41
8.4	Infrastruktur-Sektion des Golden-Signals-Dashboards mit PostgreSQL-Verbindungen, RabbitMQ-Queue-Tiefe, Host-Ressourcen und Service Graph.	42
A.1	C4-Container-Diagramm der frag.jetzt-Architektur in Vollseitenansicht	54

Tabellenverzeichnis

3.1	Applikationstragende Dienste und ihre Observability-Relevanz	11
3.2	Zuordnung der Risikopunkte zu Golden Signals und Nutzerwirkung.	13
5.1	Vergleichende Bewertung der drei Observability-Zielarchitekturen im Kontext von frag.jetzt	21
6.1	Serviceübergreifende Signal-Matrix mit Zuordnung von Dienst, Golden Signal, Messgröße, Erhebungszweck und primärer Stakeholder-Rolle.	27
7.1	Übersicht der instrumentierten Komponenten und beobachteten Telemetriesignale.	32
7.2	Zuordnung der prototypischen Umsetzung zu den Observability-Anforderungen.	37
8.1	Ergebnissynthese der anforderungsbezogenen Evaluation.	38
F.1	Panelübersicht des Golden-Signals-Dashboards.	70
G.1	Verzeichnis der in dieser Arbeit genutzten KI-Werkzeuge.	73

Listings

B.1	OpenTelemetry Collector Konfiguration (<code>config.yaml</code>)	55
C.1	Docker Compose Override (<code>docker-compose.override.yml</code>).	57
F.1	Auszug aus der Dashboard-JSON: Panel „Backend HTTP Latency“ (gekürzt). .	71
G.1	Copilot-Prompts zur Architekturanalyse	75

Abkürzungsverzeichnis

Abkürzung	Bedeutung
API	Application Programming Interface
AGPL	Affero General Public License
AI	Artificial Intelligence
APM	Application Performance Monitoring
ARS	Audience Response System
C4	Context, Containers, Components, Code
CI/CD	Continuous Integration / Continuous Deployment
CNCF	Cloud Native Computing Foundation
CPU	Central Processing Unit
DB	Database
DFG	Deutsche Forschungsgemeinschaft
ELK	Elasticsearch, Logstash, Kibana
FF	Forschungsfrage
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
ID	Identifier
I/O	Input / Output
JSON	JavaScript Object Notation
JVM	Java Virtual Machine
KI	Künstliche Intelligenz
LGTM	Loki, Grafana, Tempo, Mimir
LLM	Large Language Model

Fortsetzung auf nächster Seite

Abkürzung	Bedeutung
NLP	Natural Language Processing
OOM	Out of Memory
OTel	OpenTelemetry
OTLP	OpenTelemetry Protocol
PWA	Progressive Web App
Q&A	Question and Answer
R2DBC	Reactive Relational Database Connectivity
REST	Representational State Transfer
SDK	Software Development Kit
SLI	Service Level Indicator
SLO	Service Level Objective
SPA	Single-Page Application
SRE	Site Reliability Engineering
SSPL	Server Side Public License
STOMP	Simple Text Oriented Messaging Protocol
TSDB	Time Series Database
UI	User Interface
WAL	Write-Ahead Log
WS	WebSocket

1 Einleitung

1.1 Ausgangssituation und Problemstellung

Die Open-Source-Plattform *frag.jetzt* wird in der Hochschullehre eingesetzt, um anonyme Rückfragen in Echtzeit zu ermöglichen.¹ Die Architektur umfasst ein webbasiertes Frontend, mehrere Backend- und Kommunikationsdienste, eine relationale Datenbank sowie einen Message Broker, die gemeinsam über Docker Compose auf einem einzelnen Host betrieben werden. Die Plattform verfügt über eine funktionsfähige CI/CD-Pipeline und eine etablierte Deployment-Infrastruktur. Was dem Betrieb jedoch fehlt, ist eine durchgängige Erfassung und Auswertung von Betriebsdaten. Metriken, Logs und Traces werden nicht systematisch erhoben und nicht entlang des Anfragepfads korreliert. Störungen werden erst wahrgenommen, wenn Endnutzende direkt betroffen sind.

Dieses Defizit ist kein Einzelfall. Verteilte Anwendungen erzeugen durch die Kommunikation ihrer Dienste über Netzwerke komplexe Abhängigkeiten, in denen Fehlerursachen häufig nicht mehr einzelnen Komponenten zugeordnet werden können (Mahida, 2023, S. 1). Niedermaier et al. (2019, S. 9) identifizieren knappe Ressourcen, fehlende Erfahrung und eine reaktive Implementierung von Monitoring als drei der häufigsten Probleme beim Betrieb solcher Systeme. Klassisches Monitoring, verstanden als Überwachung vordefinierter Metriken und statischer Schwellwerte, stößt unter diesen Bedingungen an Grenzen, weil es auf vorab bekannte Fehlerbilder beschränkt bleibt und diagnostische Tiefe für eine Ursachenanalyse vermissen lässt (Hallur, 2024, S. 602). Observability erweitert diese Perspektive, indem sie aus extern erfassbaren Ausgaben eines Systems auf dessen internen Zustand schließen lässt und dabei auch neuartige, vorab nicht antizipierte Störungen nachvollziehbar macht (Usman et al., 2022, S. 86907). Die diagnostische Kraft entsteht allerdings erst durch die gezielte Verknüpfung verschiedener Telemetriearten, die in der Praxis häufig isoliert erhoben werden (Tzanettis et al., 2022, S. 1–4).

Als Ordnungsrahmen für die servicebezogene Strukturierung von Beobachtungsdaten dienen die im Kontext des Site Reliability Engineering (SRE) beschriebenen *Four Golden Signals*: Latenz, Traffic, Fehler und Sättigung (Ewaschuk, 2016, S. 60). Diese vier Größen erfassen grundlegende Symptome eines Dienstes und eignen sich als Ausgangspunkt, wenn nur ein begrenzter Satz von Messgrößen erhoben werden kann. Ihre Wirksamkeit hängt allerdings davon ab, wie sie in Alarmierung, Kontextanreicherung und betriebliche Abläufe eingebettet werden (Ewaschuk, 2016, S. 63–64).

Bestehende wissenschaftliche Arbeiten behandeln die genannten Konzepte überwiegend isoliert voneinander. So werden Observability-Frameworks meist abstrakt analysiert, die Four Golden

¹ Das Projekt ist öffentlich zugänglich unter <https://gitlab.arsnova.eu/arsnova/frag.jetzt/>. Alle architekturbezogenen Aussagen in dieser Arbeit stützen sich auf die Analyse des öffentlich verfügbaren Quellcodes.

Signals vorrangig als theoretisches Modell behandelt und Alerting-Strategien unabhängig von konkreten Architekturen diskutiert. Eine methodisch transparente Übertragung dieser Bausteine auf ein spezifisches verteiltes System, bei der Systemanalyse, Signalableitung, Stack-Auswahl, Alerting und Evaluation ineinandergreifen, wird in den herangezogenen Quellen bisher nicht umfassend abgebildet. Die vorliegende Arbeit adressiert diese Lücke.

1.2 Zielsetzung und Abgrenzung der Arbeit

Ziel dieser Arbeit ist die Entwicklung einer Observability-Strategie für *frag.jetzt*, die technische Messgrößen systematisch mit betrieblichen Fragestellungen verknüpft. Das wissenschaftliche Ziel besteht in einer methodisch nachvollziehbaren Ableitungslogik von der Systemanalyse über die servicebezogene Operationalisierung der Four Golden Signals bis zur kriteriengestützten Evaluation. Das technische Ziel umfasst die prototypische Umsetzung auf dem Staging-Server mit Instrumentierung ausgewählter Kernkomponenten, einem konsolidierten Dashboard und exemplarischen Alerting-Regeln. Das praktische Ziel liegt in der Bereitstellung wiederverwendbarer Artefakte, insbesondere einer Signal-Matrix, strukturierter Runbooks und einer dokumentierten Alerting-Konfiguration.

Die Arbeit grenzt sich bewusst von Aspekten ab, die den Rahmen einer Bachelorarbeit überschreiten. Die Instrumentierung beschränkt sich auf die JVM-basierten Dienste und die Infrastrukturschicht. Frontend-Telemetrie und die Instrumentierung des AI Providers werden nicht umgesetzt. Formale SLOs werden nicht definiert, ML-basierte Anomalieerkennung ist nicht Gegenstand der Arbeit, und die rollenspezifischen Dashboard-Sichten werden konzeptionell beschrieben, im Prototyp jedoch nicht vollständig realisiert.

1.3 Forschungsfragen

Aus der Problemstellung und der Zielsetzung ergeben sich drei aufeinander aufbauende Forschungsfragen.

- FF1:** Wie lassen sich die Four Golden Signals für die spezifischen Services (PWA-Frontend, Spring-Boot-Backend, PostgreSQL, KI-Services) von *frag.jetzt* konkret definieren und instrumentieren?
- FF2:** Welche Observability-Stack-Kombination erfüllt die Anforderungen von *frag.jetzt* unter Berücksichtigung der Single-Host-Topologie und der begrenzten Teamressourcen am besten?
- FF3:** Wie kann ein intelligentes Alerting-System konzipiert werden, das False Positives reduziert und actionable Insights für verschiedene Stakeholder-Rollen (DevOps, Entwickler, Product Owner) bereitstellt?

FF1 fragt nach der servicebezogenen Übersetzung eines allgemeinen Rahmenwerks auf eine konkrete Architektur. FF2 adressiert die kriteriengestützte Werkzeugauswahl unter den spezifi-

schen Rahmenbedingungen des Projekts. FF3 zielt auf ein Alerting-Konzept, das Alarme mit operativem Kontext und strukturierten Handlungsanweisungen verbindet.

1.4 Methodisches Vorgehen

Die Arbeit folgt einer konstruktiv-artefaktororientierten Arbeitslogik. Aus der theoretischen Fundierung und einer Systemanalyse von frag.jetzt werden Observability-Anforderungen abgeleitet. Darauf aufbauend erfolgen eine kriteriengestützte Stack-Evaluation, die Konzeption einer Observability-Strategie mit Alerting und Runbooks sowie deren prototypische Umsetzung auf dem Staging-Server. Abschließend prüft eine Evaluation die Strategie gegen die zuvor definierten Anforderungen. Die methodische Logik und die Bewertungskriterien werden in Kapitel 4 ausführlich dargelegt.

1.5 Aufbau der Arbeit

Die Arbeit gliedert sich in zehn Kapitel. Kapitel 2 bis 4 schaffen die Grundlagen: die theoretische Fundierung mit Monitoring, Observability, Telemetriearten und Golden Signals (Kapitel 2), die Systemanalyse von frag.jetzt mit Risikopunkten und fünf Observability-Anforderungen (Kapitel 3) sowie der methodische Bewertungsrahmen (Kapitel 4). Kapitel 5 bis 7 bilden den Kern der Eigenleistung: die kriteriengestützte Stack-Auswahl (Kapitel 5), die Konzeption der Observability-Strategie mit Signal-Matrix, Dashboard-Konzept und Alerting-Framework (Kapitel 6) sowie die prototypische Implementierung auf dem Staging-Server (Kapitel 7). Kapitel 8 bis 10 schließen die Arbeit mit der Evaluation gegen die Anforderungen (Kapitel 8), der Einordnung in den Literaturkontext (Kapitel 9) sowie Fazit und Ausblick (Kapitel 10) ab.

2 Theoretische Fundierung

2.1 Abgrenzung von Monitoring und Observability in verteilten Systemen

Für die Entwicklung einer Observability-Strategie ist zunächst eine begriffliche Trennung von Monitoring und Observability erforderlich. Monitoring bezeichnet die kontinuierliche Erfassung und Analyse von Betriebsdaten wie Logs und Metriken, um den aktuellen Zustand eines Systems sichtbar zu machen (Usman et al., 2022, S. 86907). Es setzt voraus, dass bekannte Zustandsindikatoren und erwartbare Abweichungen vorab definiert wurden, und prüft, ob diese Grenzwerte eingehalten werden.

Observability geht über diese hypothesengeleitete Überwachung hinaus. Im Kern beschreibt der Begriff die Fähigkeit, aus den externen Ausgaben eines Systems auf dessen interne Zustände zu schließen (Usman et al., 2022, S. 86907). Entscheidend ist dabei die Möglichkeit, auch neuartige und vorab nicht antizipierte Störungen nachvollziehen zu können (Abieba et al., 2025, S. 272). Solche unvorhergesehenen Fehlerbilder, die als *unknown unknowns* bezeichnet werden, entstehen gerade in verteilten Architekturen aus dem Zusammenspiel vieler parallel agierender Komponenten (Banerjee, 2025, S. 916). Die Interviewstudie von Niedermaier et al. (2019, S. 11) bestätigt diese Perspektive aus der Industriepraxis: Unternehmen wechseln zunehmend von einer komponentenisolierten Zustandsprüfung hin zu einer nutzungs- und geschäftsprozessorientierten Gesamtsicht.

Monitoring bildet damit die notwendige Voraussetzung für Observability, wird jedoch nicht davon ersetzt (Usman et al., 2022, S. 86907). Für die vorliegende Arbeit bedeutet das, eine Strategie, die sich auf das reine Erfassen bekannter Grenzwerte beschränkt, reicht für ein verteiltes System wie frag.jetzt nicht aus. Benötigt wird vielmehr ein Ansatz, der technische Messdaten mit diagnostischer Tiefe und Nutzerperspektive verbindet (Hallur, 2024, S. 602).

2.2 Besonderheiten verteilter und cloud-nativer Anwendungen

Verteilte und cloud-native Anwendungen werden häufig als Microservices-Architekturen umgesetzt, die aus zahlreichen, unabhängig deploybaren Diensten bestehen und in Containern betrieben werden (Sakinala, 2025, S. 102). Die resultierende Komplexität lässt sich auf vier Eigenschaften verdichten: Modularität, Verteilung, Dynamik und Flüchtigkeit (Usman et al., 2022, S. 86908). Frühere Überwachungsansätze, die auf einzelne Schichten zugeschnitten waren, stoßen unter diesen Bedingungen an Grenzen, weil sie Interaktionen zwischen Diensten zur Laufzeit nur unzureichend abbilden (Madupati, 2023, S. 24).

Der Grund liegt in der starken Laufzeitkopplung trotz struktureller Entkopplung. Ein Leistungsproblem in einem Dienst kann durch Zustandsänderungen in einer ganz anderen Komponente verursacht werden, etwa durch verzögerte Datenbankzugriffe oder überlastete Downstream-Pfade

(Garimilla, 2024, S. 155). Isolierte Überwachungskonzepte ohne übergreifenden Kontext führen zu diagnostischen Lücken und erschweren die Fehlerbehebung erheblich (Niedermaier et al., 2019, S. 8).

Daraus ergibt sich die Notwendigkeit einer Ende-zu-Ende-Sicht. Distributed Tracing gilt als das Verfahren, mit dem Serviceaufrufketten pro Anfrage nachvollzogen werden können (Li et al., 2022, S. 4). Ein Trace erfasst den kausal zusammenhängenden Ablauf einer Anfrage durch mehrere Stationen eines verteilten Systems und macht sichtbar, wo Verzögerungen oder Fehler auftreten. Die dafür erforderliche Weitergabe von Identifikatoren entlang des Anfragepfads ist eine entscheidende Voraussetzung, um Fehlerursachen überhaupt eingrenzen zu können (Niedermaier et al., 2019, S. 11). Abbildung 2.1 verdeutlicht den Unterschied zwischen lokaler Komponentenüberwachung und einer durchgängigen Anfrageverfolgung.

Gegenüberstellung: Lokales Komponentenmonitoring vs. Durchgängige Anfrageverfolgung

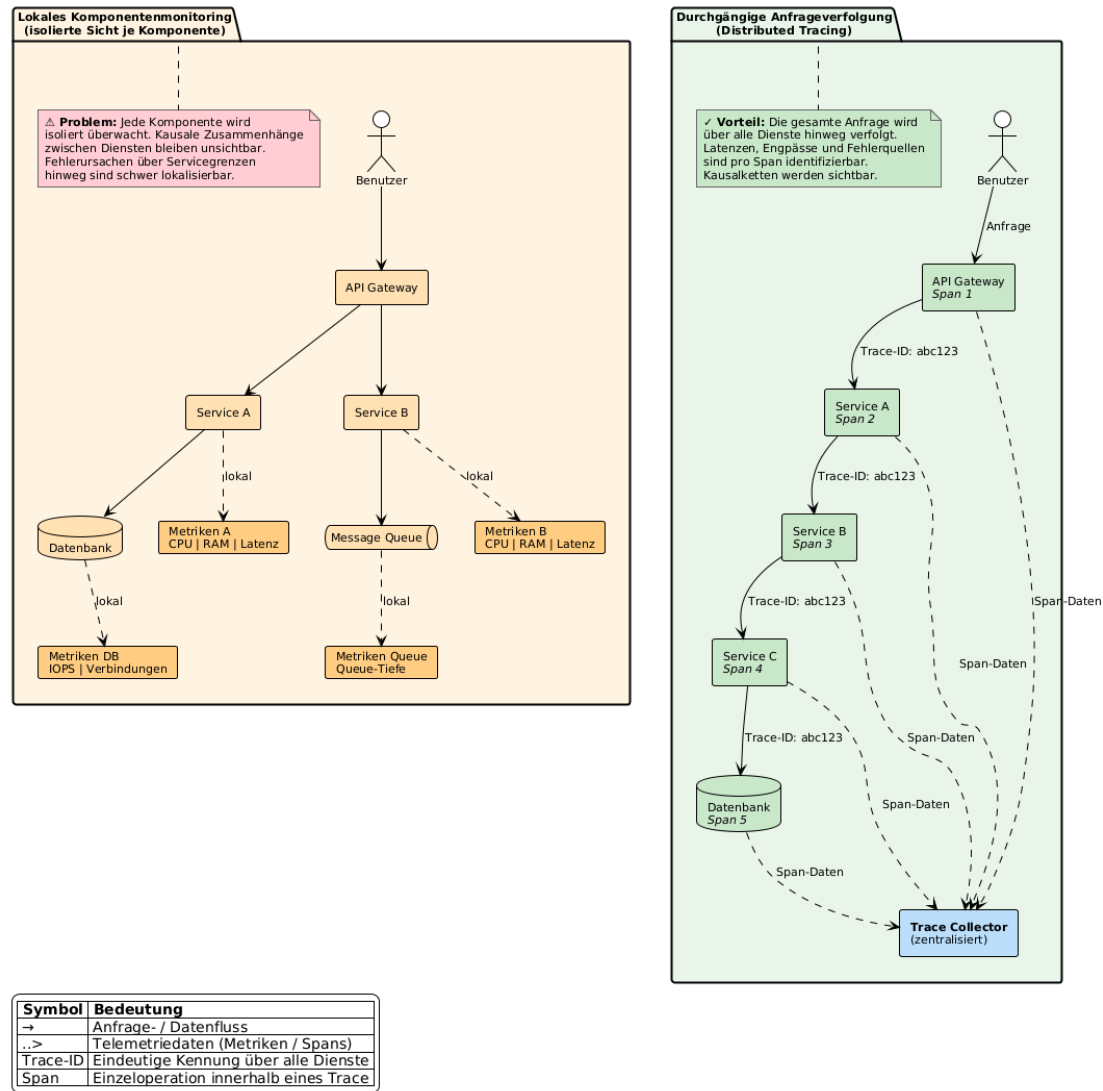


Abbildung 2.1: Gegenüberstellung von lokalem Komponentenmonitoring und durchgängiger Anfrageverfolgung über mehrere Dienste (Distributed Tracing). Quelle: Eigene Darstellung.

2.3 Telemetriearten und ihr Zusammenspiel: Logs, Metrics und Traces

Für die Beobachtbarkeit verteilter Anwendungen sind Logs, Metrics und Traces die drei am weitesten verbreiteten Telemetrieformen (Li et al., 2022, S. 6; Usman et al., 2022, S. 86912–86913). Keine dieser Formen liefert für sich genommen eine vollständige Betriebssicht (Pappula et al., 2021, S. 48). Ihr Nutzen entsteht erst aus den jeweils unterschiedlichen Perspektiven, die sie auf dasselbe Laufzeitverhalten eröffnen (Garimilla, 2024, S. 157).

Logs dokumentieren diskrete Ereignisse in zeitlicher Abfolge und eignen sich insbesondere für die Analyse unerwarteter Fehlerzustände (Albuquerque und Correia, 2025, S. 2). Allerdings erzeugen Microservices-Architekturen erhebliche Logmengen (Madupati, 2023, S. 25). Die zeitliche Zuordnung von Logereignissen über mehrere Dienste hinweg erfordert strukturierte Formate und Korrelationskennungen (Sakinala, 2025, S. 102). Zudem ist die Ableitung von Trends aus Logs vergleichsweise aufwendig, da die Verarbeitung großer Textmengen rechenintensiv ist (Albuquerque und Correia, 2025, S. 11).

Metrics verdichten Laufzeitverhalten zu numerischen Messwerten, die über die Zeit aggregiert werden können und sich vergleichsweise einfach speichern, abfragen und korrelieren lassen (Usman et al., 2022, S. 86913). Sie eignen sich besonders für Zustandsentwicklungen, Schwellenüberwachung und Alarmauslösung. Gleichzeitig abstrahieren Metrics stärker als Logs: Sie zeigen, dass eine Latenz steigt, erklären jedoch nicht automatisch den Grund. Reine Infrastrukturmetriken können für anwendungsbezogene Diagnosen zu grobkörnig ausfallen (Albuquerque und Correia, 2025, S. 11).

Traces erlauben die Rekonstruktion konkreter Anfragepfade über mehrere Komponenten hinweg und ergänzen Metriken um eine prozessbezogene Perspektive (Albuquerque und Correia, 2025, S. 2). Allerdings ist Tracing aufwendiger als die Erhebung einzelner Metriken, da die Instrumentierung ressourcenintensiver sein kann (Giamattei et al., 2024, S. 15) und Sampling-Strategien erforderlich werden, um den Overhead in ein vertretbares Verhältnis zum diagnostischen Nutzen zu bringen (Albuquerque und Correia, 2025, S. 10).

Der komplementäre Charakter dieser Datenarten ist in der Literatur breit anerkannt (Li et al., 2022, S. 6; Abieba et al., 2025, S. 272). Über Korrelationskennungen können Logs mit Traces verknüpft werden, sodass zeitliche Abläufe und Fehlermeldungen gemeinsam auswertbar werden (Albuquerque und Correia, 2025, S. 10). Eine leistungsfähige Observability-Plattform muss Anomalien daher nicht isoliert, sondern im Zusammenspiel aller drei Datenarten sichtbar machen (Tzanettis et al., 2022, S. 1–4).

2.4 OpenTelemetry als Standard für herstellerübergreifende Instrumentierung

Je stärker Observability auf mehrere Datenarten, Sprachen und Werkzeuge angewiesen ist, desto dringender wird eine gemeinsame Instrumentierungsgrundlage. In heterogenen Monitoring-Architekturen führen geschlossene Datenformate häufig zu Interoperabilitätsproblemen und strukturellen Integrationsengpässen (Sakinala, 2025, S. 105; Giamattei et al., 2024, S. 17). Das CNCF-Projekt OpenTelemetry adressiert dieses Problem als herstellerneutrale Instrumentierungsschicht, die aus der Zusammenführung von OpenCensus und OpenTracing hervorgegangen ist und die Erzeugung sowie Erfassung von Telemetriedaten vereinheitlicht (Usman et al., 2022, S. 86911). Offene Spezifikationen erlauben es Entwicklungsteams, ihren Code zu instrumentieren, ohne sich an ein bestimmtes Analyse-Backend zu binden (Li et al., 2022, S. 22).

Zugleich darf OpenTelemetry nicht mit einem vollständigen Observability-Stack gleichgesetzt werden. Vielmehr stellt es standardisierte SDKs für die Erfassung von Logs, Metriken und Traces bereit, während Speicherung, Analyse und Visualisierung dieser Telemetriedaten in nachgelagerten Backends und Auswertungswerkzeugen wie Prometheus, Loki, Jaeger oder Grafana erfolgen (Sakinala, 2025, S. 106–107; Gudimetla, 2025, S. 501–502). Instrumentierung erzeugt stets zusätzlichen Entwicklungs- und Pflegeaufwand und muss deshalb zur Entwicklungsgeschwindigkeit des jeweiligen Projekts passen (Giamattei et al., 2024, S. 15). Für die spätere Stack-Evaluation ist diese Differenzierung entscheidend. Die Wahl der Instrumentierungsschicht und die Wahl der Analyse-Backends sind zwei getrennte Architekturentscheidungen.

2.5 Die Four Golden Signals als heuristischer Bezugsrahmen

Die Four Golden Signals (*Latency*, *Traffic*, *Errors* und *Saturation*) bilden in der SRE-nahen Literatur einen bewusst reduzierten, aber praxistauglichen Rahmen zur Beobachtung nutzerorientierter Dienste. Ewaschuk (2016, S. 60) beschreibt sie als die vier Größen, auf die sich die Überwachung eines clientseitig wahrnehmbaren Dienstes mindestens konzentrieren sollte, wenn nur ein begrenzter Satz von Metriken erhoben werden kann.

Latency erfasst die Zeit, die ein Dienst zur Bearbeitung einer Anfrage benötigt. Dabei müssen erfolgreiche und fehlgeschlagene Anfragen getrennt betrachtet werden, weil schnell ausgelieferte Fehlantworten die Gesamtstatistik verzerren können (Ewaschuk, 2016, S. 60).

Traffic bezeichnet die auf einen Dienst wirkende Nachfrage und muss als dienstspezifische Lastgröße verstanden werden, etwa als HTTP-Anfragen pro Sekunde oder als Anzahl aktiver Sitzungen (Ewaschuk, 2016, S. 60).

Errors umfassen die Rate fehlgeschlagener Anfragen. Dabei geht es nicht nur um explizite Fehler wie HTTP-500-Antworten, sondern auch indirekte Fehlzustände, etwa sachlich falsche Ergebnisse oder Antwortzeiten jenseits zugesicherter Schwellen (Ewaschuk, 2016, S. 60).

Saturation beschreibt, wie stark ein Dienst seine limitierenden Ressourcen ausschöpft. Da

Dienste häufig bereits vor Erreichen ihrer nominellen Volllast Leistungsabfälle zeigen, werden steigende Latenzen in der Praxis als Frühindikator für bevorstehende Kapazitätsengpässe interpretiert (Ewaschuk, 2016, S. 60–61).

Diese vier Größen werden auch von Usman et al. (2022, S. 86913–86914) als wesentliche Messperspektiven eingeordnet, die als Grundlage für Alerting, Fehlersuche und Kapazitätsplanung dienen. Solche Metriken sind jedoch nur dann analytisch wertvoll, wenn sie an die konkrete Rolle des jeweiligen Dienstes angepasst werden (Albuquerque und Correia, 2025, S. 12). Die Stärke des Rahmenwerks liegt in seiner Ordnungsfunktion: Es zwingt dazu, Beobachtungsgrößen entlang weniger, aus Nutzersicht aussagekräftiger Dimensionen zu strukturieren.

2.6 SLIs, SLOs und Error Budgets

Während die Four Golden Signals einen Orientierungsrahmen für relevante Beobachtungsgrößen liefern, beantworten sie noch nicht, welches Niveau an Zuverlässigkeit als ausreichend gelten soll. Service Level Indicators (SLIs) binden konkrete Messgrößen an servicebezogene Zielgrößen (SLOs) zurück, die geschäftsrelevante Anforderungen aus der Nutzerperspektive fassbar machen (Niedermaier et al., 2019, S. 11). Damit entsteht eine Brücke zwischen technischer Messung und fachlicher Erwartung, die als gemeinsamer Referenzrahmen für die teamübergreifende Zusammenarbeit dient (Niedermaier et al., 2019, S. 12).

Gerade in verteilten Anwendungen ist diese Brücke wichtig, weil isolierte Infrastrukturmetriken wenig darüber aussagen, ob ein Dienst seine Aufgabe aus Nutzersicht erfüllt. Alarme sollten nicht auf beliebige Grenzwertverletzungen reagieren, sondern an Schwellen orientiert sein, deren Überschreitung die Einhaltung definierter SLOs gefährdet (Vinnakota und Kolla, 2025, S. 176).

Das Error Budget bezeichnet den rechnerisch aus einem SLO abgeleiteten Spielraum für tolerierte Unzuverlässigkeit innerhalb eines festgelegten Zeitfensters (Dasari, 2025, S. 993). Die praktische Bedeutung liegt in der Steuerungsfunktion: Solange Budget vorhanden ist, können Änderungen mit kontrolliertem Risiko ausgerollt werden. Ist es aufgebraucht, erhält Stabilisierung Vorrang gegenüber Weiterentwicklung (Dasari, 2025, S. 993–994). Damit verändert sich auch die Funktion von Alerting. Ein Alarm, der lediglich auf lokale Ressourcenauslastung oder starre Einzelgrenzwerte reagiert, kann technisch korrekt und dennoch betrieblich wenig hilfreich sein.

2.7 Alert Fatigue, Actionable Alerts und Runbooks

Die Literatur zu Monitoring und Alerting zeigt übereinstimmend, dass nicht die Menge an Alarmen, sondern deren geringe Handlungsqualität ein zentrales Betriebsproblem darstellt. Bei zu hoher Frequenz werden Warnungen nur noch überflogen oder ignoriert, sodass echte Störungen im Rauschen untergehen können (Ewaschuk, 2016, S. 57). Dieser Effekt wird als Alert Fatigue bezeichnet (Vinnakota und Kolla, 2025, S. 173).

In einer strukturell vergleichbaren Problemlage zeigen Jalalvand et al. (2024, S. 2–3) für Security Operations Centres, dass hohe Alarmvolumina, zahlreiche False Positives und mangelnde Priorisierung Analyseteams überlasten. Obwohl der fachliche Kontext ein anderer ist, verdeutlicht der Befund, dass Alarmmüdigkeit ein allgemeines Entwurfsproblem alarmintensiver Betriebsumgebungen ist und nicht nur ein Randproblem einzelner Werkzeuge.

Vor diesem Hintergrund gewinnt der Begriff des *actionable alert* an Bedeutung. Alerts sollten nur dann ausgelöst werden, wenn eine tatsächliche oder unmittelbar drohende Beeinträchtigung für den Nutzer vorliegt (Ewaschuk, 2016, S. 63–64). Beim Eintreffen eines Alarms sollte der operative Kontext so aufbereitet sein, dass unmittelbar erkennbar wird, welches Problem vorliegt, welche Ursachen wahrscheinlich sind und welche Maßnahmen eingeleitet werden müssen (Vinnakota und Kolla, 2025, S. 174). Wirksame Alerting-Strategien müssen zudem zwischen handlungsrelevanten Hinweisen und bloßem Hintergrundrauschen differenzieren. Als problematisch gelten starre Schwellwerte in dynamischen Umgebungen, weil sie saisonale Schwankungen oder Abhängigkeiten zwischen Diensten unzureichend berücksichtigen (Vinnakota und Kolla, 2025, S. 173, 176).

Eng mit handlungsfähiger Alarmierung ist die Rolle von Runbooks verbunden. Jede kritische Meldung sollte mit einem Runbook verknüpft sein, das Verweise auf relevante Observability-Werkzeuge, vordefinierte Behebungsschritte, klare Eskalationspfade und Angaben zur Zuständigkeit enthält (Vinnakota und Kolla, 2025, S. 174). Für wiederkehrende, schematische Reaktionsmuster empfiehlt die SRE-Literatur eine konsequente Automatisierung, damit algorithmische Alarmreaktionen nicht dauerhaft menschliche Aufmerksamkeit binden (Ewaschuk, 2016, S. 66).

3 Systemanalyse und Anforderungsdefinition: „frag.jetzt“

Die folgende Systemanalyse umfasst eine detaillierte Untersuchung des Quellcodes, der Deployment-Konfigurationen sowie der Abhängigkeitsstruktur der fünf Repositories des frag.jetzt-Ökosystems.¹ Zur effizienten Navigation und Exploration der Codebasis wurden dabei unterstützend das KI-Werkzeug *GitHub Copilot* herangezogen.² Ziel der Untersuchung ist die Identifikation observability-relevanter Architekturmerkmale, Risikopunkte und bestehender Instrumentierungslücken als Grundlage für die Anforderungsdefinition.

3.1 Fachlicher Kontext und Nutzungsszenario

frag.jetzt ist ein webbasiertes Audience-Response-System für Lehrveranstaltungen. Teilnehmende können anonym Fragen stellen, auf Beiträge abstimmen und an Live-Umfragen teilnehmen. Ergänzend bietet das System KI-gestützte Funktionen wie Keyword-Extraktion und thematische Zusammenfassungen. Das Nutzungsmuster konzentriert sich auf enge Zeitfenster. Während einer Veranstaltung können mehrere hundert Teilnehmende gleichzeitig aktiv sein, während die Last außerhalb nahe null liegt. Für die Observability-Strategie folgt daraus, dass Engpässe vor allem unter Spitzenlast erkannt werden müssen, nicht im Tagesdurchschnitt. Niedermaier et al. (2019, S. 8) identifizieren solche Dynamik als zentrale Herausforderung beim Monitoring verteilter Systeme, da konventionelle Schwellenwerte vorübergehende Lastspitzen nicht zuverlässig abbilden.

3.2 Technische Architektur

Das Ökosystem umfasst fünf Repositories, die über Docker Compose auf einem einzelnen Host zu zehn Containern orchestriert werden. Kubernetes kommt nicht zum Einsatz. Die Anwendung gliedert sich in vier applikationstragende Dienste (Tabelle 3.1) und mehrere Infrastrukturkomponenten.

Die Infrastruktur umfasst die PostgreSQL-Hauptdatenbank, eine dedizierte Keycloak-Datenbank, eine separate AI-Datenbank mit logischer Replikation aus der Hauptdatenbank, RabbitMQ als Message Broker, Keycloak für Authentifizierung (OIDC/SSO) und einen E-Mail-Dienst. Die komponentenübergreifenden Abhängigkeiten werden in Abb. 3.1 zusammengefasst. Für die Observability-Strategie ist die Deployment-Topologie maßgeblich: Alle Container laufen auf einem einzelnen Host ohne konfigurierte Ressourcenlimits (CPU, Speicher) und ohne Health-Check-Direktiven auf Container-Ebene.

¹ Quellcode abrufbar unter <https://gitlab.arsnova.eu/arsnova/>.

² Die hierfür verwendeten Prompts sind in Anhang G.3 dokumentiert.

Tabelle 3.1: Applikationstragende Dienste und ihre Observability-Relevanz. Quelle: Eigene Darstellung.

Dienst	Technologie	Observability-Relevanz
Frontend	Angular, Nginx	Einstiegspunkt; Nginx routet an Backend (/api), WS-Gateway (/gateway), Keycloak (/auth), AI Provider (/ai)
Backend	Spring Boot, WebFlux, R2DBC	Zentraler kritischer Dienst: 95 Endpunkte, 19 Controller
WS-Gateway	Kotlin, Netty, STOMP	Echtzeit-Updates via RabbitMQ; Session-Tracking in-memory
AI Provider	Python, FastAPI, LangChain	19 LLM-Provider; externe Abhängigkeiten ohne Timeout

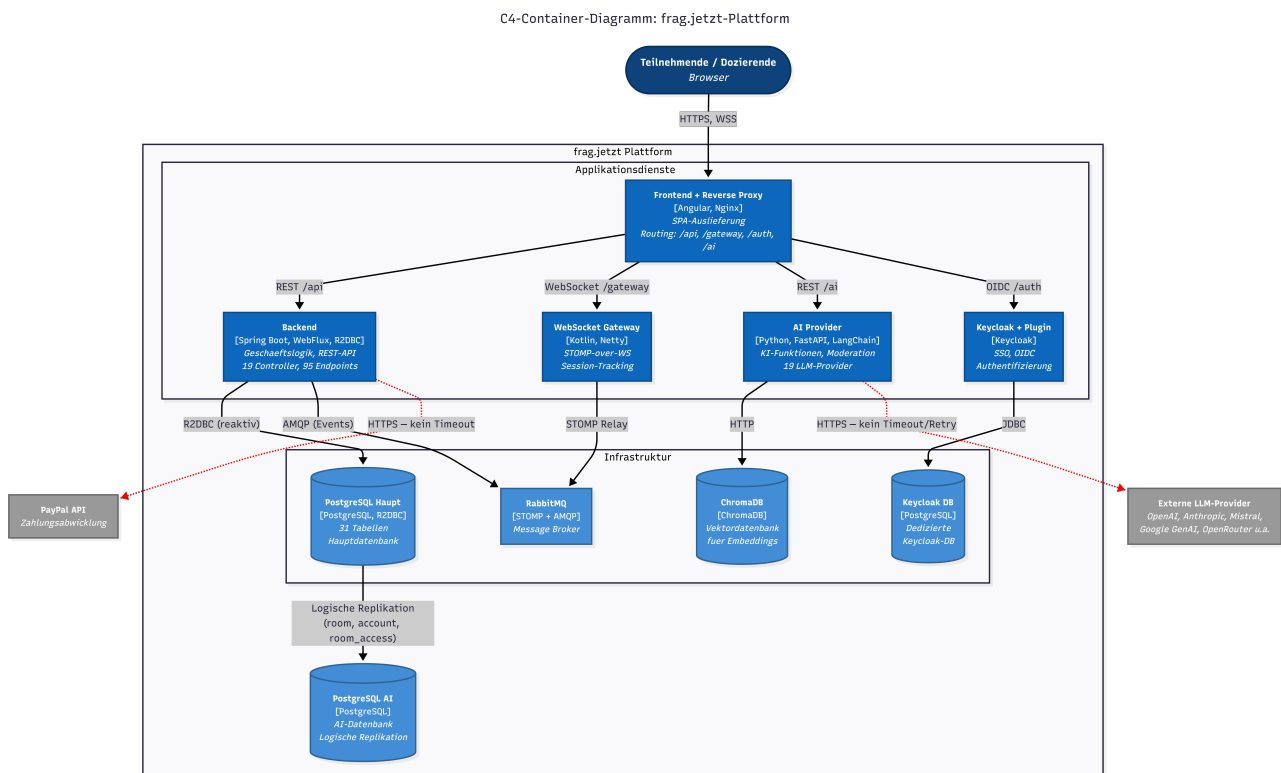


Abbildung 3.1: C4-Container-Diagramm der frag.jetzt-Architektur mit Applikationsdiensten, Infrastrukturkomponenten und Kommunikationswegen (Vollseitenansicht des Diagramms in Anhang A). Quelle: Eigene Darstellung.

3.3 Kritische User Journeys

Eine wirksame Observability-Strategie orientiert sich an den tatsächlichen Nutzungspfaden, nicht an abstrakten Systemkomponenten (Niedermaier et al., 2019, S. 10–11). Für frag.jetzt lassen sich drei geschäftskritische Abläufe identifizieren.

3.3.1 Journey 1: Frage stellen.

Die Frage wird vom Frontend per REST an das Backend übermittelt, dort validiert und in der Datenbank persistiert, wobei ein Trigger eine Kommentarnummer generiert. Anschließend wird sie über RabbitMQ als Aktualisierung in Echtzeit an alle Clients im Raum weitergeleitet. Der Ablauf durchläuft vier Komponenten. Messgrößen sind Backend-Antwortzeit, Dauer des DB-Inserts und WebSocket-Zustelllatenz.

3.3.2 Journey 2: Live-Voting.

Votes werden per REST persistiert. Ein Trigger aktualisiert synchron die Score-Spalte der zugehörigen Frage. Bei parallelen Abstimmungen kann es zu konkurrierenden Schreibzugriffen auf dieselbe Datenbankzeile kommen, wodurch Transaktionen aufgrund von Sperrmechanismen aufeinander warten müssen (Row-Level Contention), was unter Spitzenlast zu steigenden Antwortzeiten führen kann (vgl. Abschnitt 3.4, R2).

3.3.3 Journey 3: KI-Zusammenfassung.

Die Anfrage gelangt vom Frontend über das Backend an den AI Provider, der einen externen LLM-Dienst aufruft. Kein externer Aufruf im System hat einen konfigurierten Timeout. Ein nicht reagierender Upstream-Dienst kann daher Request-Laufzeiten unbegrenzt verlängern und die rechtzeitige Antwortlieferung an das Frontend gefährden.

3.4 Risikopunkte und Zuordnung zu den Golden Signals

Aus der Architekturanalyse lassen sich sechs Risikopunkte ableiten, die in Tabelle 3.2 den betroffenen Golden Signals und ihrer Nutzerwirkung zugeordnet werden.

3.4.1 R1: Fehlende Timeouts und Resilienz.

Weder Backend noch AI Provider konfigurieren Timeouts, Retries oder Circuit Breaker auf ausgehenden HTTP-Aufrufen. Ein nicht reagierender externer Dienst kann Request-Laufzeiten unbegrenzt verlängern und die Ressourcenauslastung erhöhen.

3.4.2 R2: Datenbankengpässe bei Massenabstimmungen.

Die Vote-Trigger führen bei jedem Schreibvorgang einen zusätzlichen `SELECT` und `UPDATE` auf der Comment-Tabelle aus. Bei gleichzeitigen Abstimmungen entsteht Row-Level Contention.

3.4.3 R3: Backend als kritischste Schwachstelle ohne Observability.

Das Backend verarbeitet den Großteil der Nutzerinteraktionen, besitzt jedoch keine Infrastruktur zur Observability. Da weder Endpunkte wie Actuator noch Metriken, Tracing oder strukturierte Logs implementiert sind, können Fehlerzustände systemseitig nicht proaktiv erfasst werden. Sie fallen meist erst durch konkrete Fehlfunktionen bei den Endanwendern auf.

3.4.4 R4: WebSocket-Verbindungsverlust.

Für RabbitMQ existieren keine Health-Checks. Ein Broker-Ausfall unterbricht alle Echtzeit-Updates, bleibt aber ohne Überwachung der Queue-Tiefe und Subscription-Anzahl zunächst unbemerkt.

3.4.5 R5: Fehlende Ressourcenlimits.

Kein Container definiert CPU- oder Speicherlimits. Ein einzelner Dienst mit unkontrolliertem Ressourcenverbrauch kann durch Ressourcenverdrängung alle übrigen Dienste beeinträchtigen.

3.4.6 R6: Single-Host-Topologie als systemweiter Ausfallpfad.

Alle zehn Container laufen auf einem gemeinsamen Host. Ein Hardware- oder Betriebssystemausfall betrifft das Gesamtsystem. In Kombination mit R5 entsteht ein einzelner Sättigungspfad, über den ein Dienst alle übrigen destabilisieren kann.

Tabelle 3.2: Zuordnung der Risikopunkte zu Golden Signals und Nutzerwirkung. Quelle: Eigene Darstellung.

Risiko	Dienst(e)	Signale	Nutzerwirkung
R1	Backend, AI Provider	Latency, Errors	Hängende Anfragen, Timeouts
R2	Backend, PostgreSQL	Latency, Saturation	Verzögerte Votes in Spitzenlast
R3	Backend	Alle	Fehler erst durch Nutzerbeschwerden sichtbar
R4	WS-Gateway, RabbitMQ	Errors, Traffic	Ausbleibende Echtzeit-Updates
R5	Alle Container	Saturation	Systemweite Instabilität
R6	Gesamtsystem	Saturation, Errors	Totalausfall bei Host-Ausfall

3.5 Observability-Ist-Zustand

Die Quellcode-Analyse ergibt, dass die bestehende Observability-Infrastruktur minimal ist. Das Backend besitzt weder Actuator noch Metriken, kein Tracing und keine strukturierten Logs. Das WS-Gateway konfiguriert einen Health- und Prometheus-Endpoint, jedoch ist die benötigte Micrometer-Registry nicht als Abhängigkeit deklariert. RabbitMQ hat das Prometheus-Plugin aktiviert, der Port wird aber nicht exponiert. Systemweit fehlen Distributed Tracing, Korrelations-IDs, zentrale Log-Aggregation und ein Monitoring-Stack vollständig. Störungen werden gegenwärtig erst wahrgenommen, wenn Endnutzende direkt betroffen sind. Ein Zustand, den Niedermaier et al. (2019, S. 9–10) als Folge einer rein reaktiven Monitoring-Implementierung einordnen.

3.6 Ableitung der Observability-Anforderungen

Aus Architektur, Kernabläufen, Risikopunkten und Ist-Zustand lassen sich fünf Anforderungen ableiten.

3.6.1 A1: Ende-zu-Ende-Nachvollziehbarkeit über Dienstgrenzen.

Die User Journeys durchlaufen jeweils mehrere Dienste. Kausale Zusammenhänge zwischen Störungen sind ohne dienstübergreifende Sichtbarkeit nicht erkennbar. Albuquerque und Correia (2025, S. 6) beschreiben *Distributed Tracing* als grundlegendes Entwurfsmuster für diese Anforderung: Jeder eingehenden Anfrage wird eine eindeutige ID zugewiesen, die über alle Dienste propagiert und in Logs sowie Spans mitgeführt wird, sodass der gesamte Ausführungspfad rekonstruierbar ist.

3.6.2 A2: Mehrstufige Messbarkeit entlang der Golden Signals.

Die Instrumentierung muss auf drei Ebenen erfolgen: (1) Browser-Telemetrie im Frontend für Nutzererfahrungsmetriken, (2) Service-Metriken für Latenz, Durchsatz, Fehlerrate und Sättigung der vier Applikationsdienste und der Datenbank, sowie (3) Infrastruktur-Metriken für Container-Ressourcen, Verbindungspools und Queue-Tiefen. Albuquerque und Correia (2025, S. 10, 16) beschreiben *Application Metrics* und *Infrastructure Metrics* als komplementäre Entwurfsmuster, deren Zusammenspiel erst eine ganzheitliche Systemsicht ermöglicht (Albuquerque & Correia, 2025, S. 22).

3.6.3 A3: Strukturiertes, korrelierbares Logging.

Das Logging muss auf strukturierte Formate (JSON) mit Korrelations-IDs umgestellt und zentral aggregiert werden, damit Logeinträge verschiedener Dienste einer gemeinsamen Anfrage zugeordnet werden können (Albuquerque & Correia, 2025, S. 9–10).

3.6.4 A4: Handlungsfähige Alarmierung.

Alarme sollen auf nutzerwirksame Zustände reagieren (steigende Antwortzeiten, wachsende Fehlerraten oder nicht erreichbare Schnittstellen) und ausreichend Kontext für eine gezielte Erstreaktion enthalten. Formale SLOs existieren derzeit nicht und können als zukünftige Weiterentwicklung betrachtet werden.

3.6.5 A5: Beobachtbarkeit der Infrastrukturschicht.

Die Single-Host-Topologie (R6) und die fehlenden Ressourcenlimits (R5) erzeugen blinde Flecken, die nur durch Infrastruktur-Metriken sichtbar werden. Die Strategie muss Container-Ressourcenverbrauch, Datenbankverbindungspool-Auslastung, RabbitMQ-Queue-Tiefen und Dienstverfügbarkeit über Health-Endpunkte einbeziehen. Niedermaier et al. (2019, S. 10) formulieren eine solche ganzheitliche Sicht über System- und Infrastrukturebenen hinweg als Kernanforderung an Monitoring-Konzepte verteilter Systeme.

Diese Anforderungen bilden die Grundlage der folgenden Kapitel: A1–A3 adressieren primär FF 1, A2 und A5 liefern Evaluationskriterien für FF 2, und A4 bildet die Grundlage für FF 3.

4 Methodisches Vorgehen und Bewertungsrahmen

Dieses Kapitel beschreibt die methodische Logik, mit der die Arbeit allgemeine Observability-Konzepte in einen anwendungsbezogenen Strategieentwurf für frag.jetzt überführt.

4.1 Arbeitslogik der Arbeit

Die Arbeitslogik umfasst fünf aufeinander aufbauende Schritte. Im ersten, bereits abgeschlossenen Schritt wurden aus Architektur, Nutzungspfaden, Risikopunkten und dem Istzustand der Observability konkrete Anforderungen an die spätere Lösung abgeleitet. Im zweiten Schritt werden diese Anforderungen in zwei Richtungen operationalisiert. Einerseits als Vergleichsmaßstab für die Auswahl eines geeigneten Technologie-Stacks, andererseits als Ableitungsrahmen für servicebezogene Golden-Signals-Messgrößen, Visualisierungen und Alarmierungsregeln. Im dritten Schritt erfolgt die Auswahl einer Zielkombination aus Instrumentierungs-, Speicher-, Visualisierungs- und Alerting-Komponenten. Im vierten Schritt wird darauf aufbauend eine konkrete Observability-Strategie mit Signal-Matrix, Dashboard-Konzept, symptomorientierter Alert-Logik und Runbook-Struktur entworfen und in ausgewählten Kernkomponenten prototypisch umgesetzt. Im fünften Schritt wird geprüft, inwieweit die entwickelte Strategie die zuvor abgeleiteten Anforderungen erfüllt.

Methodisch entsteht damit eine durchgängige Kette von der Problemanalyse über die Anforderungs- und Strategieableitung bis zur prototypischen Validierung. Der Schwerpunkt der Eigenleistung liegt in der Übersetzung allgemeiner Observability-Prinzipien in dienstspezifische Messgrößen, Entscheidungsregeln und betriebliche Artefakte für frag.jetzt. Die folgende Konzeption stützt sich auf die in Kapitel 3 erhobenen Architekturbefunde.

4.2 Vorgehen bei der Ableitung der Golden Signals pro Service

Die Herausforderung liegt weniger in der Kenntnis der vier Golden Signals als in ihrer konkreten Übersetzung auf heterogene Dienste mit unterschiedlichen Rollen und Kommunikationsmustern. Damit diese Übersetzung nachvollziehbar erfolgt, folgt die Arbeit einer fünfstufigen Ableitungslogik.

Im ersten Schritt wird die Rolle des Dienstes im Gesamtsystem bestimmt, weil die funktionale Einordnung maßgeblich beeinflusst, welche Beobachtungsgrößen aussagekräftig sind. Der zweite Schritt identifiziert die kritischen Interaktionen anhand der User Journeys und Risikopunkte aus Kapitel 3, da Metriken an realen Nutzungspfaden ausgerichtet sein sollten (Niedermaier et al., 2019, S. 10–11). Im dritten Schritt werden geeignete Messgrößen abgeleitet, wobei pro Golden Signal mindestens eine primäre Messgröße bestimmt wird. Nicht jedes Signal lässt sich bei jedem Dienst als White-Box-Metrik erfassen. Bei externen KI-Services etwa ist Saturation

nur indirekt beobachtbar.

Der vierte Schritt begründet initiale Schwellenwerte. Da für frag.jetzt keine historischen Produktivdaten vorliegen, werden operative Startschwellen auf Basis von Lasttests und Erfahrungswerten festgelegt, die in einer Staging-Umgebung überprüft und im Betrieb angepasst werden können. Formale SLOs existieren derzeit nicht und werden als zukünftige Weiterentwicklung betrachtet. Niedermaier et al. (2019, S. 9) bestätigen, dass unklare nichtfunktionale Anforderungen und reaktive Monitoring-Entwicklung zu den häufigsten Herausforderungen verteilter Systeme gehören. Im fünften Schritt wird bestimmt, welche Visualisierung und Alarmierung aus den Messgrößen folgen. Nicht jede Metrik rechtfertigt einen Alarm, und die Zielgruppe bestimmt die Aufbereitung (Vinnakota & Kolla, 2025, S. 175).

4.3 Kriterien für Stack-Auswahl und spätere Evaluation

Die Arbeit verwendet zwei bewusst getrennte Kriterienebenen. Die Stack-Auswahl (Kapitel 5) beantwortet die Werkzeugfrage, welche Technologiekombination die Anforderungen am besten adressiert. Die Evaluation (Kapitel 8) beantwortet die Ergebnisfrage, ob die entwickelte Strategie die identifizierten Probleme tatsächlich löst.

4.3.1 Vergleichskriterien für die Stack-Auswahl

Aufbauend auf den Anforderungen A1–A5 und der Literatur (Faseeha et al., 2025, S. 72014–72015); (Giamattei et al., 2024, S. 3–4) werden folgende acht Vergleichskriterien verwendet:

- **Abdeckung der Telemetriepfeiler:** Unterstützung von Metriken, Logs und Traces in integrierter Form.
- **Korrelationsfähigkeit:** Verknüpfung verschiedener Telemetriearten über gemeinsame Bezeichner (Trace-IDs, Korrelations-IDs).
- **OpenTelemetry-Kompatibilität:** Unterstützung des herstellerneutralen CNCF-Standards für Instrumentierung und Datenexport.
- **Dashboarding und Alerting:** Eignung für rollenspezifische Dashboards und differenzierte Alarmregeln.
- **Integrationsaufwand:** Technischer Aufwand für die Einbindung in die bestehende Docker-Compose-Topologie.
- **Ressourcenbedarf:** Zusätzlicher Speicher-, CPU- und Festplattenbedarf auf einem einzelnen Host (vgl. R6).
- **Erweiterbarkeit:** Anpassungsfähigkeit bei wachsendem Telemetrieveolumen oder zusätzlichen Diensten.
- **Community und Ökosystem:** Breite der Nutzerbasis, Dokumentationsreife und Framework-Integrationen.

4.3.2 Bewertungskriterien für die Gesamtstrategie

Die Gesamtstrategie wird direkt an den Anforderungen A1–A5 gemessen. Für jede Anforderung wird eine konkrete Prüffrage formuliert: Lässt sich ein Request Ende-zu-Ende rekonstruieren (A1)? Deckt die Instrumentierung alle drei Messebenen ab (A2)? Können Logeinträge über Korrelations-IDs verknüpft werden (A3)? Reagieren Alarme auf nutzerwirksame Zustände und enthalten sie ausreichend Kontext für eine Erstreaktion (A4)? Werden Container-Ressourcen, Datenbankpool-Auslastung und Queue-Tiefen erfasst (A5)?

Die Beantwortung stützt sich auf drei Evidenzquellen: die Artefaktanalyse von Instrumentierungskonfigurationen, Dashboard-Definitionen und Alarmregeln, die Konfigurations- und Instrumentierungsprüfung sowie ausgewählte Last- und Störungsszenarien in einer produktionsnahen Testumgebung. Ergänzend fließen drei übergreifende Bewertungsdimensionen ein: Stakeholder-Tauglichkeit, Abdeckung der Risikopunkte R1–R6 und Umsetzungsrealismus.

5 Evaluation und Auswahl des Observability-Stacks

Das vorliegende Kapitel wendet die in Abschnitt 4.3 hergeleiteten acht Vergleichskriterien auf drei vollständige Observability-Architekturen an, um FF 2 zu beantworten: Welche Observability-Stack-Kombination eignet sich am besten für die Überwachungsanforderungen von frag.jetzt?

5.1 OpenTelemetry als verbindliche Instrumentierungsschicht

Bevor die Analyse-Backends verglichen werden, wird OpenTelemetry als verbindliche Instrumentierungsgrundlage festgelegt. Die Begründung wurde in Kapitel 2 vorbereitet: OpenTelemetry vereinheitlicht die Erzeugung und Erfassung von Logs, Metriken und Traces als herstellerneutraler CNCF-Standard, ohne selbst Speicherung oder Darstellung zu übernehmen. Eine standardisierte Instrumentierung entkoppelt den Applikationscode von der Backend-Wahl, sodass die Analyse-Ebene austauschbar bleibt (Sakinala, 2025, S. 107). Für frag.jetzt als polyglotte Anwendung mit vier Laufzeitumgebungen (Java, Kotlin, Python, TypeScript) ist ein sprachübergreifender Instrumentierungsansatz besonders wertvoll, weil ohne einen gemeinsamen Rahmen für jede Sprache separate Bibliotheken gepflegt werden müssten (Sakinala, 2025, S. 106). Die Trägerschaft durch die CNCF sichert außerdem langfristige Weiterentwicklung (Sakinala, 2025, S. 107). Der folgende Vergleich setzt OpenTelemetry daher als gegebene Instrumentierungsschicht voraus und bewertet die Kandidaten-Stacks ausschließlich als Empfangs-, Speicher-, Visualisierungs- und Alarmierungsebene.

5.2 Vergleich der Zielarchitekturen

Für den Vergleich werden ausschließlich Architekturen herangezogen, die Logs, Metriken und Traces vollständig adressieren und über eine gemeinsame Auswertungsebene verfügen. Alle drei Kandidaten basieren auf quelloffenen Kernkomponenten und werden in der Fachliteratur und Praxis zur Microservices-Beobachtbarkeit regelmäßig herangezogen (Faseeha et al., 2025, S. 72019–72021); (Tzanettis et al., 2022, S. 8); (Giamattei et al., 2024, S. 1, 5).

5.2.1 Elastic Observability

Elastic Observability erweitert den klassischen ELK-Stack um APM und OTel-Integration. Der APM-Server empfängt Traces, Metriken und Logs über OTLP nativ (Elastic, n. d.). Die Stärke liegt in der tiefgreifenden Loganalyse durch Volltextindizierung (Banerjee, 2025, S. 920). Für frag.jetzt ergeben sich allerdings gewichtige Nachteile: Die vollständige Indizierung verursacht erheblichen Speicher- und Rechenaufwand (Banerjee, 2025, S. 918), und auf der Single-Host-Topologie konkurriert ein Elasticsearch-Knoten mit seinem JVM-Heap unmittelbar mit den Anwendungscontainern. Hinzu kommt die mehrfache Lizenzänderung (Apache 2.0 → ELv2/SSPL

→ teils AGPL seit 2024), die die langfristige Planungssicherheit im Vergleich zu CNCF-getragenen Projekten einschränkt (Elastic, 2024).

5.2.2 Grafana/LGTM-Stack: Prometheus + Loki + Tempo + Grafana

Die zweite Zielarchitektur (auch als LGTM-Stack bekannt) kombiniert Prometheus für Metriken, Loki für Logs und Tempo für Traces mit Grafana als gemeinsamer Visualisierungs- und Alarmierungsplattform (Gudimetla, 2025, S. 501). Loki verfolgt einen ressourcenschonenden Ansatz, indem es ausschließlich Labels indiziert statt den vollständigen Loginhalt (Faseeha et al., 2025, S. 72019–72020), was den Speicherbedarf um 50–80 % gegenüber Volltextindizierung reduziert, während die Abfrageflexibilität jedoch eingeschränkter ist (Gudimetla, 2025, S. 502). Tempo empfängt Traces über standardisierte Protokolle, darunter OTLP sowie Jaeger- und Zipkin-Formate und ermöglicht deren Verknüpfung mit Logs und Metriken innerhalb von Grafana (Grafana Labs, n. d. a); (Tzanettis et al., 2022, S. 8). Als quelloffenes Monitoringsystem für numerische Zeitreihendaten übernimmt Prometheus die Metrikerfassung sowie die Kurzzeitspeicherung. Dabei ruft das System die Metriken über HTTP-Endpunkte in einem Pullverfahren ab und legt sie in einer Time-Series Database (TSDB) ab (Faseeha et al., 2025, S. 72019). Mit PromQL stellt es eine spezialisierte Abfragesprache für Aggregation, Alerting und Dashboarding bereit, wodurch es sich besonders für die operative Auswertung von Latenz-, Fehler-, Traffic- und Sättigungsmetriken eignet. Grafana unterstützt alle drei Datenquellen nativ und vereint sie in einer navigierbaren Oberfläche (Sundström, 2025, S. 16). Mimir bleibt als skalierbares Langzeit-Backend eine evolutive Ausbauoption (Gudimetla, 2025, S. 502).

5.2.3 Prometheus + Jaeger + Loki + Grafana

Die dritte Architektur verfolgt einen Bausteinansatz aus eigenständigen Open-Source-Werkzeugen. Prometheus und Jaeger sind graduated CNCF-Projekte mit aktiver Community (Giamattei et al., 2024, S. 15). Jaeger übernimmt in dieser Architektur die verteilte Anfrageverfolgung und ermöglicht damit die Analyse von Latenzen, Serviceabhängigkeiten und Fehlerpfaden über mehrere Dienste hinweg. Anders als Tempo ist Jaeger hier jedoch als eigenständige Tracing-Komponente mit eigener Weboberfläche und separaten Storage-Backends eingebunden (Faseeha et al., 2025, S. 72020). Dadurch fällt die Architektur weniger homogen aus als der Grafana-Stack. Dem Vorteil der Komponentenreife steht eine erhöhte Integrationskomplexität gegenüber: Die drei Werkzeuge folgen unterschiedlichen Konfigurationsformaten und Storage-Modellen (Gudimetla, 2025, S. 501). Die Korrelation zwischen den Telemetriearten ist über Grafana grundsätzlich möglich, bleibt aber weniger eng integriert als in einer Architektur, deren Komponenten von vornherein aufeinander abgestimmt sind (Sundström, 2025, S. 17).

5.3 Vergleich und Synthese

Tabelle 5.1 fasst die Bewertung der drei Zielarchitekturen entlang der acht Vergleichskriterien zusammen. Die Einstufungen beruhen auf der qualitativen Analyse in den vorherigen Abschnitten und stellen eine argumentativ begründete Einschätzung dar, keine empirisch erhobenen Messwerte.

Tabelle 5.1: Vergleichende Bewertung der drei Observability-Zielarchitekturen im Kontext von frag.jetzt (3 = sehr gut, 2 = befriedigend, 1 = eingeschränkt). Alle drei Kandidaten adressieren Logs, Metriken und Traces. OpenTelemetry wird als gemeinsame Instrumentierungsschicht vorausgesetzt. Quelle: Eigene Darstellung.

Kriterium		Elastic Observability (Elastic APM, ES, Kibana)		Grafana-Stack (Prom., Loki, Tempo, Grafana)		Prom./Jaeger/Loki (Prom., Jaeger, Loki, Grafana)
Telemetrie- pfeiler	3	Logs, Metriken und Traces über APM/OTel abgedeckt	3	Alle drei Pfeiler durch Prometheus, Loki und Tempo	3	Alle drei Pfeiler durch Einzelkomponenten
Korrelation	2	Trace-Log-Korrelation in Kibana; Metrik-Anbindung über Lens	3	Trace-Log-Metrik-Navigation nativ in Grafana	2	Via Grafana-Plugins, weniger eng integriert
OTel-Kompa- tibilität	3	OTLP-Empfang nativ, EDOT-Distributionen verfügbar	3	Alle Komponenten OTel-nativ	3	Alle Komponenten OTel-nativ
Dashboard. & Alerting	2	Kibana + Watcher/Rules; weniger flexibel als Grafana	3	Grafana Unified Alerting + Alertmanager	3	Grafana + Alertmanager
Integrations- aufwand	1	Logstash/Beats-Pipeline, APM-Server, JVM-Tuning	2	Mehr Container, aber homogene Konfiguration via OTel-Collector	1	Drei Konfig.-Formate und Storage-Backends
Ressourcen- bedarf	1	Elasticsearch speicherintensiv, JVM-Heap konkurriert mit Anwendung	2	Höher als Prom. allein, deutlich geringer als Elastic	2	Jaeger benötigt separates Speicher-Backend
Erweiter- barkeit	2	Nativ skalierbar, aber Cluster-Management komplex	3	Prometheus via Mimir erweiterbar; Loki/Tempo modular	3	Jede Komponente einzeln skalierbar
Community & Ökosys- tem	2	Große Community, aber Lizenzänderungen (ELv2/SSPL, seit 2024 teils AGPL)	2	Grafana Labs getragen, gewachsend, Tempo jünger als Jaeger	3	Prom. u. Jaeger graduated CNCF, Loki etabliert
Summe		16		21		20

Elastic Observability erzielt die niedrigste Gesamtbewertung, weil der hohe Ressourcenbedarf und der komplexe Integrationspfad auf einer Single-Host-Topologie besonders ins Gewicht fallen. Die beiden Grafana-basierten Architekturen liegen eng beieinander. Der Unterschied manifestiert sich in den Kriterien Korrelation und Integrationsaufwand.

5.4 Begründete Zielentscheidung für frag.jetzt

5.4.1 Entscheidungsbegründung im Kontext von frag.jetzt

Drei Argumente sprechen für den Grafana-Stack gegenüber der Alternative aus Prometheus, Jaeger und Loki.

Das erste Argument betrifft die *Integrationskomplexität*. frag.jetzt wird von einem sehr kleinen Entwicklungsteam betrieben, das Observability erstmals einführt. Niedermaier et al. (2019, S. 9) identifizieren knappe Ressourcen und fehlende Erfahrung als eigenständige Herausforderung (C7). Die Jaeger-Variante erfordert die Pflege dreier unterschiedlicher Konfigurationsformate und Storage-Backends (Gudimetla, 2025, S. 501), während Loki und Tempo aus demselben Ökosystem stammen und vergleichbare Konfigurationsmuster verwenden.

Das zweite Argument adressiert die *Korrelierbarkeit*. Sundström (2025, S. 16) beschreibt Grafana als die Oberfläche, die Metriken, Traces und Logs in einer navigierbaren Gesamtsicht vereint. In der alternativen Kombination wäre Jaeger als separates Trace-Backend einbindbar, die Verknüpfung bliebe jedoch weniger eng integriert, und die Jaeger-eigene Oberfläche würde als paralleles Werkzeug bestehen bleiben.

Das dritte Argument betrifft die *Austauschbarkeit bei stabilem Instrumentierungskern*. Sollte sich Tempo im Betrieb als unzureichend erweisen, ließe sich der Trace-Export im OTel Collector auf Jaeger umleiten, ohne Applikationscode anzupassen. Ebenso kann die Metrikspeicherung bei wachsendem Datenvolumen auf Mimir umgestellt werden (Gudimetla, 2025, S. 502). Diese Flexibilität ist besonders relevant, weil sich nichtfunktionale Anforderungen häufig erst im laufenden Betrieb konkretisieren (Niedermaier et al., 2019, S. 9–10).

5.4.2 Kompromisse und Einschränkungen

Die Entscheidung geht mit bewussten Kompromissen einher. Tempo ist ein jüngerer Projekt als Jaeger, das als graduated CNCF-Projekt über einen breiteren Reifegrad verfügt (Giamattei et al., 2024, S. 15). Die gewählte Architektur verzichtet auf Mimir als Metrik-Backend, weil die lokale Prometheus-TSDB für das moderate Telemetrie-volumen von frag.jetzt ausreichend ist. Lokis labelbasierte Indexierung erfordert eine sorgfältige Label-Auswahl (Grafana Labs, n. d. b), was bei der überschaubaren Zahl von Diensten beherrschbar bleibt.

Diese Nachteile werden dadurch relativiert, dass die offizielle Grafana-Labs-Dokumentation für Standardkonfigurationen ausreichend ist, die Vorteile einer homogenen Komponentenfamilie für ein kleines Team überwiegen und OpenTelemetry als Instrumentierungsschicht einen späteren Wechsel einzelner Backend-Komponenten mit vertretbarem Aufwand ermöglicht. Damit beantwortet die gewählte Kombination aus Prometheus, Loki, Tempo und Grafana, instrumentiert über OpenTelemetry, FF 2.

6 Konzeption der Observability-Strategie für frag.jetzt

Die vorangegangenen Kapitel haben den theoretischen Bezugsrahmen gelegt, den Ist-Zustand von frag.jetzt analysiert und den Observability-Stack begründet ausgewählt. Das vorliegende Kapitel überführt diese Vorarbeiten in eine zusammenhängende Strategie und beantwortet, *was* beobachtet werden soll, *warum* gerade diese Größen betrieblich bedeutsam sind und *nach welchen Prinzipien* die Lösung aufgebaut wird.

6.1 Leitprinzipien der Strategie

Vier Prinzipien greifen die Anforderungen A1–A5 aus Kapitel 3 auf und übersetzen sie in Gestaltungsleitlinien für die nachfolgenden Entwurfsentscheidungen.

P1: Servicebezogene Messbarkeit. Jeder Dienst von frag.jetzt wird als eigenständige Beobachtungseinheit behandelt, für die Messgrößen entlang der vier Golden Signals abgeleitet werden. Das Vorgehen folgt der Empfehlung, Beobachtungsgrößen an den Schnittstellen der Dienste zu erheben, weil sich dort Symptome manifestieren, bevor tieferliegende Ursachen sichtbar werden (Ewaschuk, 2016, S. 59). Anforderung A2 wird damit unmittelbar adressiert.

P2: Ende-zu-Ende-Nachvollziehbarkeit. An jedem der in Abschnitt 3.2 identifizierten Dienstübergänge muss Trace-Kontext propagiert werden, damit kausal zusammenhängende Aktivitäten über Dienstgrenzen hinweg korrelierbar bleiben (Bissig, 2024, S. 32). Ohne diese Verknüpfung bleiben Logs und Metriken isolierte Datenpunkte (Sundström, 2025, S. 14–15). Das Prinzip greift Anforderung A1 und den Korrelationsaspekt von A3 auf.

P3: Rollenorientierte Aufbereitung. Unterschiedliche Stakeholder stellen unterschiedliche Fragen an dasselbe Laufzeitverhalten (Bissig, 2024, S. 54). Abschnitt 6.4 überführt dieses Prinzip in drei konkrete Sichtebenen.

P4: Schrittweise Erweiterbarkeit. Da frag.jetzt Observability erstmals einführt und knappe Ressourcen eine eigenständige Herausforderung darstellen (Niedermaier et al., 2019, S. 9–10), wird die Strategie als erweiterbarer Ausgangspunkt angelegt. Die Instrumentierung über OpenTelemetry stellt sicher, dass Backend-Komponenten austauschbar bleiben (Gudimetla, 2025, S. 502).

6.2 Logische Zielarchitektur der Observability

Die Observability-Architektur folgt einem vierstufigen Schichtenmodell, wie es Kosińska et al. (2023, S. 73037–73038) als grundlegendes Muster für cloud-native Anwendungen beschreiben. Abbildung 6.1 zeigt den Datenfluss von der Instrumentierung über Sammlung und Speicherung bis zur Auswertung. Die werkzeugspezifische Konfiguration wird in Kapitel 7 dokumentiert.

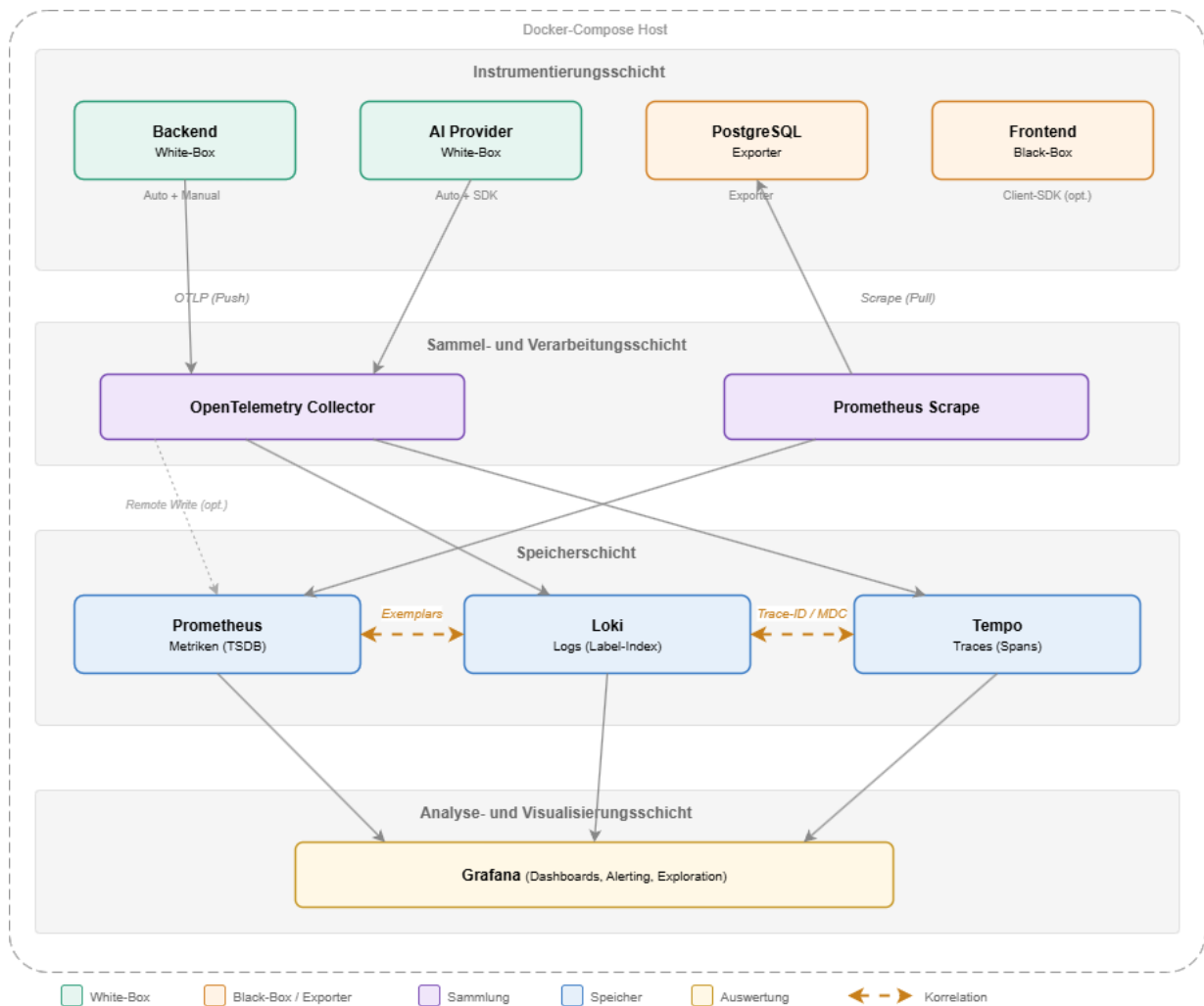


Abbildung 6.1: Logische Zielarchitektur der Observability-Pipeline fuer frag.jetzt. Gestrichelte Querverbindungen kennzeichnen Korrelationsmechanismen zwischen den Speicher-Backends.
Quelle: Eigene Darstellung.

6.2.1 Instrumentierungsschicht

Aus der Architektur von frag.jetzt ergibt sich eine differenzierte Instrumentierungsstrategie. Konzeptionell sind Backend, WebSocket Gateway und AI Provider als Eigenentwicklungen grundsätzlich über Auto-Instrumentierung mit selektiver manueller Ergänzung beobachtbar. Die Quellcodeanalyse in Kapitel 3 hat gezeigt, dass keine dieser Komponenten bislang Telemetrie emittiert (R3). Im prototypischen Umfang dieser Arbeit werden jedoch nur die JVM-basierten Dienste Backend und WS-Gateway instrumentiert; die Instrumentierung des AI Providers bleibt einem späteren Ausbauschritt vorbehalten. Demgegenüber agieren die externen LLM-Provider als Black-Box-Abhängigkeiten, bei denen mindestens die Ein- und Ausgangsseite jedes Aufrufs instrumentiert werden müsste (Bissig, 2024, S. 31).

OpenTelemetry übernimmt die Rolle der standardisierten Instrumentierungsschicht (vgl. Abschnitt 5.1). Die entscheidenden Übergabepunkte für die Kontextpropagation sind die Verbindungen zwischen Frontend und Reverse-Proxy, zwischen Reverse-Proxy und Backend, zwischen

Backend und Datenbank sowie zwischen Backend und AI Provider. Eine besondere Herausforderung stellt die Echtzeitkommunikation über den Message Broker dar, da dort eine manuelle Injektion des Trace-Kontexts in das Nachrichtenformat erforderlich wird (Sundström, 2025, S. 25). Ergänzend werden Container-Metriken und datenbankspezifische Indikatoren über spezialisierte Exporter erhoben, die Prometheus-kompatible Endpunkte bereitstellen.

6.2.2 Sammel- und Verarbeitungsschicht

Der OpenTelemetry Collector fungiert als zentrale Routing-Komponente, die Instrumentierung und Backend entkoppelt (Sakinala, 2025, S. 107). Für frag.jetzt wird der Collector als push-basierte Empfangskomponente für anwendungsseitige Traces und Metriken konfiguriert, weil ein Push-Modell bei einer zukünftigen Erweiterung keine Umstellung erfordert (Bissig, 2024, S. 39). Infrastrukturmetriken folgen dagegen dem etablierten Pull-Modell über Prometheus-Scraping. Bei dem für frag.jetzt erwarteten geringen Datenvolumen kann zunächst auf Sampling verzichtet werden. Eine Anpassung ist erst dann vorgesehen, wenn das Telemetrieveolumen die Speicherkapazität des Hosts belastet.

6.2.3 Speicherschicht

Die aufbereiteten Signale werden in drei spezialisierten Backends abgelegt: Metriken in der lokalen Prometheus-TSDB, Logs in Loki mit labelbasierter Indexierung und Traces in Tempo. Entscheidend ist die Korrelierbarkeit über die drei Speicher hinweg, die bei getrennter Speicherung nicht automatisch entsteht (Bissig, 2024, S. 44). Drei Mechanismen bilden die diagnostischen Brücken: die Trace-ID als gemeinsamer Identifikator über alle drei Pfeiler, die Injektion der Trace-ID in den Mapped Diagnostic Context (MDC) des Logging-Frameworks (Sundström, 2025, S. 20) und Metriken-Exemplars, die den direkten Übergang von einem aggregierten Metrikwert zum zugehörigen Trace ermöglichen (Bissig, 2024, S. 25); (Albuquerque & Correia, 2025, S. 12). Die Speichertrennung ist nur dann diagnostisch tragfähig, wenn diese Verknüpfungspfade von Beginn an in der Instrumentierung verankert werden.

6.2.4 Analyse- und Visualisierungsschicht

Grafana bildet die einheitliche Oberfläche für Abfrage, Visualisierung und Alarmierung aller gespeicherten Telemetriedaten (Sundström, 2025, S. 16). Die Korrelation wird über Datenquellen-übergreifende Navigation hergestellt: Von einem auffälligen Metrikpunkt lässt sich über ein Exemplar direkt zum zugehörigen Trace springen, und aus einem Trace heraus führen verknüpfte Logabfragen zum relevanten Kontext.

6.2.5 Einordnung im Kontext von frag.jetzt

Die beschriebene Vier-Schichten-Architektur orientiert sich an Referenzmodellen für Kubernetes-basierte Umgebungen (Gudimetla, 2025, S. 501). frag.jetzt weicht davon in drei Punkten ab. Erstens läuft das Ökosystem auf einem einzelnen Host, sodass der Collector nicht als Sidecar, sondern als eigenständiger Container betrieben wird. Zweitens fehlt ein Service Mesh, weshalb die Telemetrieerzeugung vollständig auf Anwendungsebene erfolgen muss. Drittens erlaubt das geringe Telemetriedatenvolumen einen Verzicht auf Sampling und vorgelagerte Aggregation, erfordert aber eine bewusste Kapazitätsbeobachtung bei wachsendem Einsatz. Die Architektur hält den Einstieg bewusst schlank und lässt Erweiterungspfade wie Mimir für Langzeitmetriken oder Tail-Based Sampling offen, ohne die Grundstruktur zu verändern (Borges et al., 2024, S. 1–2).

6.3 Servicebezogene Operationalisierung der Four Golden Signals

Die Ableitung der Beobachtungsgrößen folgt dem in Abschnitt 4.2 beschriebenen Vorgehen und konzentriert sich auf die vier für die Forschungsfragen zentralen Beobachtungseinheiten: Frontend, Backend, Datenbank und externe KI-Dienste. Nginx, RabbitMQ und das WebSocket Gateway werden als Infrastruktur- bzw. Integrationskomponenten über die Infrastrukturmetriken mitbeobachtet, erhalten aber keine eigenständige Signal-Matrix. Ewaschuk (2016, S. 60) bezeichnet die Wahl des Traffic-Indikators ausdrücklich als *high-level system-specific metric*, was bedeutet, dass die Übertragung der vier Dimensionen auf jeden Dienst eine eigenständige Entwurfsleistung erfordert. Tabelle 6.1 fasst die Ergebnisse dieser Ableitung in einer Signal-Matrix zusammen. Die folgenden Unterabschnitte erläutern nur die Besonderheiten, die sich nicht unmittelbar aus der Matrix erschließen.

6.3.1 PWA-Frontend

Das Angular-Frontend unterscheidet sich grundlegend von den serverseitigen Komponenten, weil die Ausführung im Browser stattfindet und die Infrastruktur nicht direkt kontrollierbar ist. Aus Nutzersicht manifestiert sich Latenz als wahrgenommene Ladezeit, zu der neben der Server-Antwortzeit auch clientseitige Faktoren wie Netzwerkqualität und Rendering beitragen. Basisindikatoren sind die Dauer von API-Aufrufen und die Ende-zu-Ende-Ladezeit beim Betreten eines Raums.

Die Nachfrageintensität lässt sich über aktive WebSocket-Verbindungen und HTTP-Anfragen pro Zeiteinheit messen. Fehlerindikatoren umfassen fehlgeschlagene API-Aufrufe (4xx/5xx), unbehandelte JavaScript-Exceptions und WebSocket-Verbindungsabbrüche. Klassische Sättigungsmetriken wie CPU und Speicher beziehen sich auf das Endgerät und liegen außerhalb des Einflussbereichs des Betriebsteams. Da die Grenzen der Frontend-Beobachtbarkeit eng gesteckt sind, liegt der instrumentierungstechnische Schwerpunkt auf den serverseitigen Komponenten.

Tabelle 6.1: Serviceübergreifende Signal-Matrix mit Zuordnung von Dienst, Golden Signal, Messgröße, Erhebungszweck und primärer Stakeholder-Rolle. Quelle: Eigene Darstellung.

Dienst	Signal	Messgröße	Zweck	Rolle
Backend	Latency	Antwortdauer je Endpunkt (p50, p95, p99)	Nutzererfahrung, Engpasserkennung	Entwicklung
Backend	Traffic	Anfragen/s je Pfad, Broker-Nachrichten/s	Lastbild, Kapazitätsplanung	DevOps
Backend	Errors	HTTP-5xx-Rate, Exception-Rate, Downstream-Fehler	Fehlererkennung, Trendanalyse	Entwicklung
Backend	Saturation	CPU, Speicher, R2DBC-Pool-Nutzung	Ressourcenwarnung	DevOps
PostgreSQL	Latency	Abfrageausführungszeit (Durchschnitt, Maximum)	Indexbedarf, Contention-Erkennung	Entwicklung
PostgreSQL	Traffic	Transaktionen/s, aktive Verbindungen	Nutzungsintensität	DevOps
PostgreSQL	Errors	Deadlocks, fehlgeschl. Transaktionen, Lock-Waits	Stabilitätsüberwachung	Entwicklung
PostgreSQL	Saturation	Verbindungspool-Quote, Cache-Hit-Rate, Disk-I/O	Kapazitätswarnung	DevOps
AI Prov.	Latency	LLM-Antwortzeit je Provider/-Modell (Histogramm)	SLA-Proxy, Kaskadenkennung	Entwicklung
AI Prov.	Traffic	LLM-Aufrufe/Zeiteinheit je Funktionstyp	Nutzungsmuster, Kontingentplanung	Product Owner
AI Prov.	Errors	Provider-Fehlerrate, 429er-Rate, Parsing-Fehler	Verfügbarkeit ext. Abhängigkeiten	Entwicklung
AI Prov.	Saturation	Antwortzeit-Trend, Rate-Limit-Häufigkeit, lokale CPU	Kapazitätswarnung	DevOps
Frontend	Latency	API-Aufruf-Dauer, Raum-Ladezeit	Nutzererfahrung	Product Owner
Frontend	Traffic	Aktive WS-Verbindungen, HTTP-Anfragen/s	Nutzungsgrad	Product Owner
Frontend	Errors	HTTP-Fehler, JS-Exceptions, WS-Abbrüche	Fehlererkennung clientseitig	Entwicklung
Frontend	Saturation	Antwortzeit-Trend, Retry-Häufigkeit	Überlastindikation	DevOps

6.3.2 Spring-Boot-Backend

Das Backend ist die zentrale Verarbeitungskomponente und der in Kapitel 3 identifizierte kritische Punkt. Der primäre Latenzindikator ist die Antwortdauer je REST-Endpunkt als Histogramm (p50, p95, p99), weil Durchschnittswerte die Erfahrung langsam bedienter Nutzer verbergen können (Ewaschuk, 2016, S. 60). Ebenso muss die Latenz erfolgreicher und fehlgeschlagener Anfragen getrennt erfasst werden, da schnell zurückgegebene Fehler den Gesamtdurchschnitt verzerren. Ergänzend werden die Latenzen ausgehender Aufrufe an PostgreSQL, den Message Broker und den AI Provider separat erhoben. Das reaktive Programmiermodell auf Basis von WebFlux erfordert dabei besondere Aufmerksamkeit, da die Messung auf Span-Ebene innerhalb der reaktiven Kette erfolgen muss (Sundström, 2025, S. 25).

Die Anfragerate pro Endpunkt bildet den Traffic ab. Für die geschäftskritischen Abläufe aus Abschnitt 3.3 ist eine gesonderte Erfassung sinnvoll. Fehler werden auf mehreren Ebenen erfasst: HTTP-5xx-Statuscodes, unbehandelte Exceptions und fehlgeschlagene Downstream-Aufrufe. Da kein externer Aufruf einen Timeout besitzt (R1), äußern sich Timeout-bedingte Fehler als hängende Anfragen, was die Latenzüberwachung umso dringlicher macht. Die Sättigung wird über Container-Metriken und den R2DBC-Connection-Pool gemessen. Ewaschuk (2016, S. 60–61) empfiehlt, Sättigung indirekt über steigende Latenz am 99. Perzentil zu erkennen, weil Latenzzunahmen häufig ein Frühindikator für Ressourcenerschöpfung sind.

6.3.3 PostgreSQL

Die Datenbank ist an allen drei geschäftskritischen Nutzungspfaden beteiligt. Die Vote-Trigger und die Comment-Nummern-Generierung (Abschnitt 3.3) erzeugen potenzielle Engpässe. Der zentrale Latenzindikator ist die Abfrageausführungszeit, wobei lang laufende Abfragen auf fehlende Indizes oder Row-Level Contention hindeuten. Die Transaktionsrate und die aktive Verbindungszahl bilden den Traffic ab. Relevante Fehlersignale umfassen Deadlocks, fehlgeschlagene Transaktionen und Verbindungsablehnungen. Die Sättigung wird über das Verhältnis genutzter zu maximal verfügbarer Verbindungen, die Buffer-Cache-Hit-Rate und die Disk-I/O-Auslastung gemessen. Ein sinkender Cache-Hit-Anteil deutet darauf hin, dass die Arbeitsspeicherkonfiguration nicht mehr zum Datenvolumen passt.

6.3.4 Externe KI-Dienste

Die Beobachtung der externen KI-Dienste beschränkt sich auf die Schnittstelle zwischen AI Provider und den angebundenen Modellen, da der Quellcode der LLM-Provider nicht verfügbar ist (Bissig, 2024, S. 31). Die LLM-Antwortzeit wird als Histogramm pro Provider und Modell erfasst. Da kein Timeout konfiguriert ist (R1), kann ein einzelner hängender Aufruf den AI Provider blockieren und über die Aufrufkette das Backend beeinflussen. Der Traffic wird als Anzahl der LLM-Aufrufe pro Zeiteinheit je Funktionstyp gemessen. Fehlersignale sind HTTP-

Fehlerantworten der Provider-APIs (insbesondere 429 Rate Limit Exceeded), Timeouts und Parsing-Fehler. Neun von 19 LLM-Providern sind explizit mit `max_retries=0` konfiguriert, sodass ein fehlgeschlagener Aufruf unmittelbar durchschlägt. Da die LLM-Provider keinen Einblick in ihre Ressourcenauslastung gewähren, wird Sättigung über Proxy-Indikatoren wie steigende Antwortzeiten und zunehmende Rate-Limit-Fehler abgebildet.

Tabelle 6.1 bildet die Grundlage für das rollenbasierte Dashboard-Design (Kap 6.4) und definiert den Rahmen für das Alerting-Konzept (Kap 6.5), weil nicht jede Messgröße Alarmrelevant ist.

6.4 Rollenbasiertes Informations- und Dashboard-Konzept

Für `frag.jetzt` wird eine hierarchische Informationsarchitektur in drei Sichtebenen umgesetzt, deren Zuschnitt sich aus den Stakeholder-Gruppen und der Signal-Matrix ableitet.

Übersichtssicht (Product Owner / Management). Diese Ebene beantwortet die Frage, ob `frag.jetzt` aus Nutzersicht funktioniert. Sie zeigt verdichtete Indikatoren wie aktive Raumsitzungen, globale Fehlerrate und Gesamtlatenz der Kernabläufe. Detailinformationen sind nur über Verlinkung erreichbar.

Dienst- und Infrastruktursicht (DevOps / Betrieb). Diese Ebene beantwortet die Frage, wo das Problem liegt. Sie stellt Ressourcenmetriken je Container, Datenbankauslastung, Message-Broker-Kennzahlen und eine aus Trace-Daten gewonnene Service-Map dar.

Diagnose- und Entwicklungssicht (Entwicklung). Diese Ebene beantwortet die Frage, was genau im Code passiert. Sie bietet endpunktbezogene Latenzhistogramme, Fehleraufschlüsselungen, Trace-Ansichten und die Möglichkeit, aus einem Trace in die zugehörigen Logeinträge zu springen. Die technische Umsetzung wird in Kapitel 7 dokumentiert.

6.5 Alerting-Konzept und Runbook-Logik

Aufbauend auf den in Abschnitt 2.7 erarbeiteten Grundlagen folgt das Alerting-Konzept drei Grundsätzen. Alarme reagieren symptomorientiert auf nutzerwirksame Zustände statt auf interne Ursachenindikatoren (Ewaschuk, 2016, S. 63–64). Jeder Alarm muss handlungsfähig sein, also Problem, betroffene Komponente und empfohlene Erstmaßnahme erkennbar machen (Vinnakota & Kolla, 2025, S. 174). Schweregrade orientieren sich an der Nutzerwirkung: *Critical* für akute Ausfälle geschäftskritischer Funktionen, *Warning* für sich abzeichnende Verschlechterungen.

6.5.1 Alarmkategorien

Aus der Signal-Matrix lassen sich vier Kategorien ableiten. Die konkreten Schwellenwerte und Zeitfenster werden in Kapitel 7 festgelegt.

Verfügbarkeitsalarme (Critical). Ein Alarm wird ausgelöst, wenn eine geschäftskritische Funktion nicht mehr erreichbar ist, etwa wenn das Backend über einen definierten Zeitraum keine erfolgreichen Antworten liefert oder die Datenbankverbindung dauerhaft fehlschlägt.

Latenzalarme (Warning / Critical). Steigt die Antwortzeit eines Kernendpunkts über einen definierten Grenzwert, etwa das 95. Perzentil über ein gleitendes Fenster, wird zunächst eine Warnung und bei weiterer Verschlechterung ein kritischer Alarm erzeugt. Für die KI-Dienste gelten angepasste Grenzwerte.

Fehleralarme (Warning / Critical). Eine Fehlerrate oberhalb eines definierten Anteils löst eine Warnung aus. Die explizite Trennung von client- (4xx) und serverseitigen (5xx) Fehlern verhindert, dass ungültige Eingaben denselben Alarm wie interne Serverfehler erzeugen.

Sättigungsalarme (Warning). Ressourcenmetriken wie CPU-Auslastung und Auslastung des Connection-Pools werden als vorausschauende Warnung konfiguriert. Ewaschuk (2016, S. 60–61) empfiehlt, Sättigung indirekt über steigende Latenz am 99. Perzentil zu erkennen.

6.5.2 Runbook-Verknüpfung

Jeder Alarm wird mit einem Runbook verknüpft, das Problembeschreibung, betroffene Komponente, Diagnoseschritte, Erstmaßnahmen und Eskalationspfad enthält. Für den Prototyp werden exemplarische Runbooks für erhöhte Backend-Latenz, erhöhte Fehlerquote und Infrastruktur-Sättigung erstellt.

Regelbasierte Alarme stoßen an Grenzen, sobald sich Lastmuster im Tages- oder Semesterverlauf verändern und starre Schwellenwerte entweder zu viele False Positives oder zu späte Warnungen produzieren (Vinnakota & Kolla, 2025, S. 173). Anomaliebasierte Verfahren setzen jedoch ein trainiertes Modell des Normalverhaltens voraus, das für frag.jetzt mangels historischer Produktivdaten derzeit nicht existiert (Soldani & Brogi, 2022, S. 5–6). Die standardisierte Metrikerhebung über Prometheus und die offene Abfrageschnittstelle über PromQL erlauben es, solche Verfahren als zukünftigen Ausbauschritt ohne Änderung der Instrumentierung anzubinden.

7 Prototypische Implementierung

Dieses Kapitel dokumentiert die praktische Umsetzung der in Kapitel 6 konzipierten Observability-Strategie und beschreibt die Instrumentierung der Kernkomponenten, exemplarische Validierungsszenarien, das Golden-Signals-Dashboard, die Alerting-Regeln sowie aufgetretene Herausforderungen.

7.1 Umfang und Abgrenzung der prototypischen Umsetzung

Die Umsetzung erfolgte auf dem Staging-Server von frag.jetzt, auf dem der vollständige Anwendungsstack in einer produktionsnahen Docker-Compose-Konfiguration betrieben wird. Der Observability-Stack wurde als eigenständiges Compose-Projekt deployt und über ein gemeinsames Docker-Netzwerk an die Applikationscontainer angebunden.

Die Implementierung umfasst drei Schwerpunkte: das Deployment des LGTM-Stacks mit allen Speicher-Backends und Infrastruktur-Exportern, die Instrumentierung von Backend und WS-Gateway über den OpenTelemetry Java Agent und die Erstellung eines Golden-Signals-Dashboards mit zugehörigen Alerting-Regeln. Bewusst ausgeklammert wurden die Instrumentierung des AI Providers und des Frontends, die clientseitige Browser-Telemetrie, die Produktivhärtung sowie die rollenspezifischen Dashboard-Sichten aus Kapitel 6.4. Eine durchgängige Ende-zu-Ende-Beobachtbarkeit aller Systemgrenzen kann daher nicht nachgewiesen werden. Die Teilumsetzung erlaubt jedoch die Prüfung der zentralen Mechanismen.

7.2 Instrumentierung ausgewählter Kernkomponenten

Die Instrumentierung folgt dem in Kapitel 6.2 beschriebenen Schichtenmodell. Tabelle 7.1 gibt eine Übersicht über die instrumentierten Komponenten, die erzeugten Signale und den im Prototyp beobachteten Nachweis.

7.2.1 Observability-Stack und Telemetrie-Pipeline

Das Deployment umfasst acht Container. Der OpenTelemetry Collector empfängt sämtliche Applikationstelemetrie über OTLP und verteilt sie an die drei Speicher-Backends: Metriken an Prometheus, Logs an Loki und Traces an Tempo. Für die Infrastrukturschicht erfassen cAdvisor, der Node Exporter und der PostgreSQL Exporter die jeweiligen Metriken. Alle Observability-Container sind mit expliziten Speicherlimits konfiguriert, um das Risiko einer unkontrollierten Ressourcenverdrängung (R5) nicht im Observability-Stack selbst zu reproduzieren. Grafana wird mit provisionierten Datenquellen gestartet, die insbesondere die Verknüpfungen zwischen den Backends herstellen: Tempo-Traces können auf korrespondierende Loki-Logs verweisen und umgekehrt.

Tabelle 7.1: Übersicht der instrumentierten Komponenten und beobachteten Telemetriesignale.
Quelle: Eigene Darstellung.

Komponente	Instrumentierung	Erzeugte Signale	Nachweis im Prototyp
Backend (Spring Boot)	OTel Java Agent (Auto-Instr.)	HTTP-Spans, DB-Spans, Logback-Logs mit Trace-ID, JVM-Metriken	Span-Waterfall mit DB- Operationen in Grafana sichtbar
WS-Gateway (Netty)	OTel Java Agent (Auto-Instr.)	WebSocket-Spans, RabbitMQ-Spans, JVM- Metriken	Service Graph zeigt Knoten mit Anfragerate
PostgreSQL	PostgreSQL Ex- porter	Verbindungszähler, Transak- tionsraten, Lock-Statistiken	<code>pg_stat_activity_count</code> in Prometheus abfragbar
RabbitMQ	Prometheus- Plugin	Queue-Tiefen, Nachrichten- raten	<code>rabbitmq_queue_messages</code> in Dashboard-Panel sicht- bar
Host	Node Exporter, cAdvisor	CPU, Speicher, Netzwerk, Container-Metriken	Kapazitätsanzeige im Dash- board mit Schwellenwertfär- bung

7.2.2 Applikationsinstrumentierung über den OTel Java Agent

Für die JVM-basierten Dienste wurde der OpenTelemetry Java Agent in Version 2.26.0 eingesetzt. Der Agent wird als Read-Only-Volume in den Container gemountet und über Umgebungsvariablen in der Docker-Compose-Override-File konfiguriert. Dieser Ansatz vermeidet Änderungen am Applikationscode und an den bestehenden Docker-Images.

Der Agent instrumentiert automatisch HTTP-Anfragen (Spring WebFlux, Netty), R2DBC-Datenbankzugriffe, RabbitMQ-Interaktionen und Logback-Logausgaben. Im Ergebnis erzeugt jeder eingehende HTTP-Request einen Root-Span, der untergeordnete Spans für Datenbankoperationen enthält. Für die instrumentierten Dienste ist damit der Verarbeitungspfad vom HTTP-Eingang bis zur Datenpersistierung in einem zusammenhängenden Trace rekonstruierbar, was Anforderung A1 in Teilen adressiert. Abbildung 7.1 visualisiert diese Hierarchie exemplarisch anhand eines Backend-Traces, bei dem sich die Gesamtlatenz in ihre Einzelschritte aufschlüsseln lässt.

Darüber hinaus visualisiert der Service Graph in Grafana (siehe Abbildung 7.2) die Kommunikationsbeziehungen zwischen den instrumentierten Diensten. Im Prototyp zeigt dieser aktuell drei Knoten (`fragjetzt-backend`, `fragjetzt-ws-gateway` und `fragjetzt-postgres`) mit ihren jeweiligen Anfrageraten und Antwortzeiten.

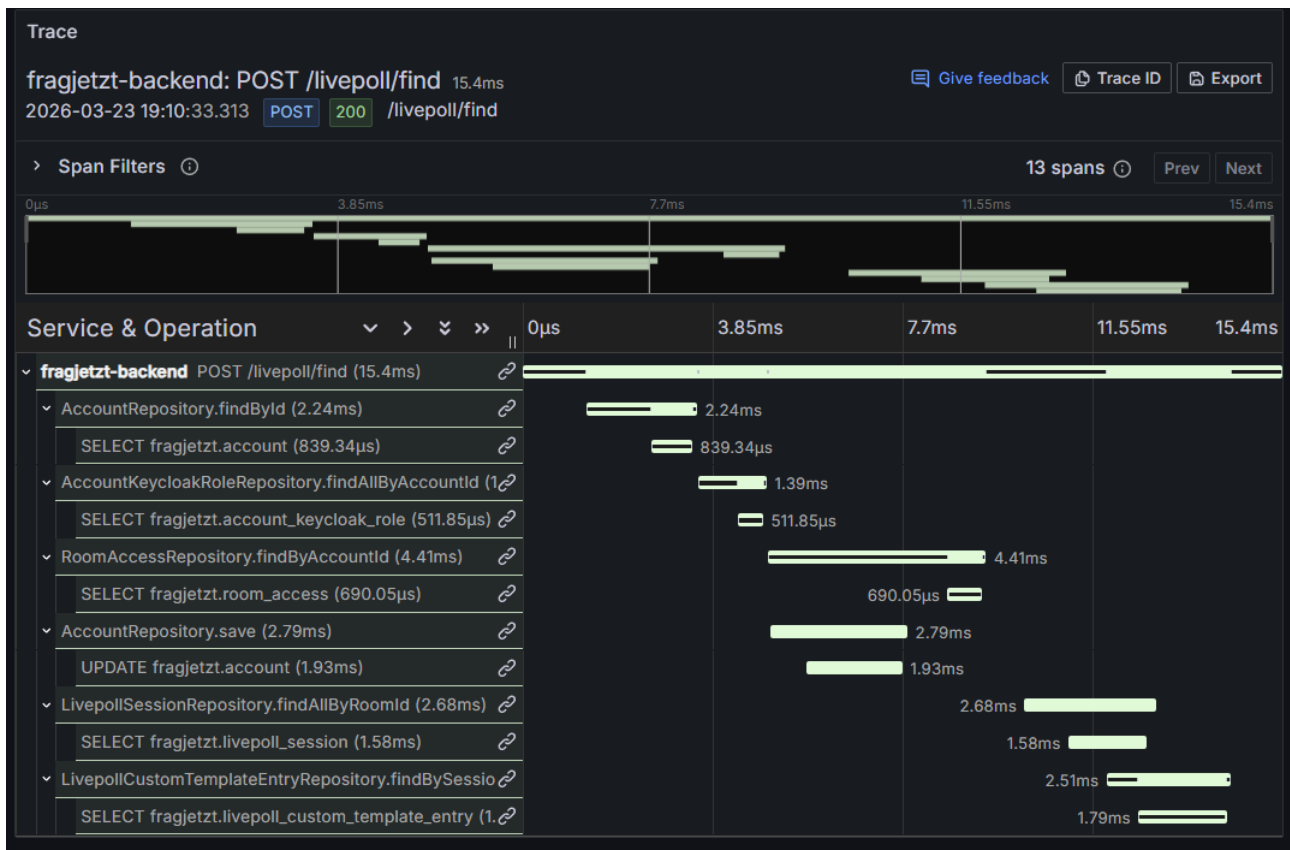


Abbildung 7.1: Span-Waterfall eines Backend-Traces (POST /livepoll/find) mit 13 Spans und einer Gesamtdauer von 15,4 ms. Quelle: Eigene Darstellung.

7.2.3 Infrastrukturausporter und Scrape-Konfiguration

Prometheus scrapet sechs Targets im 15-Sekunden-Intervall. Die Infrastruktur-Exporter liefern Metriken, die direkt auf die Sättigungsindikatoren der Signal-Matrix aus Kapitel 6.3 abbilden und damit Anforderung A5 auf Metrikebene im Prototyp adressieren. Ergänzend empfängt Prometheus über den OTLP-Receiver Applikationsmetriken vom OTEL Collector. Tempo generiert aus den eingehenden Traces automatisch Span-Metriken und schreibt diese per Remote Write an Prometheus zurück. Aus diesen Span-Metriken werden der Service Graph und die Traffic- sowie Fehlerratenpanels im Dashboard gespeist.

7.2.4 Logging-Anbindung

Das Backend erzeugte im Standardbetrieb nahezu keine Logausgaben, weil Spring Boot WebFlux eingehende HTTP-Requests nicht automatisch protokolliert. Für den Prototyp wurde Debug-Level-Logging für die HTTP-Verarbeitungsschicht selektiv aktiviert, sodass Request-bezogene Einträge in Loki verfügbar sind und über die vom OTEL Agent automatisch injizierten Trace-IDs mit den zugehörigen Spans korreliert werden können. Diese Korrelierbarkeit adressiert Anforderung A3 in Grundzügen. Für einen Produktiveinsatz wäre strukturierte JSON-Protokollierung mit selektiver Schweregradfilterung vorzuziehen.

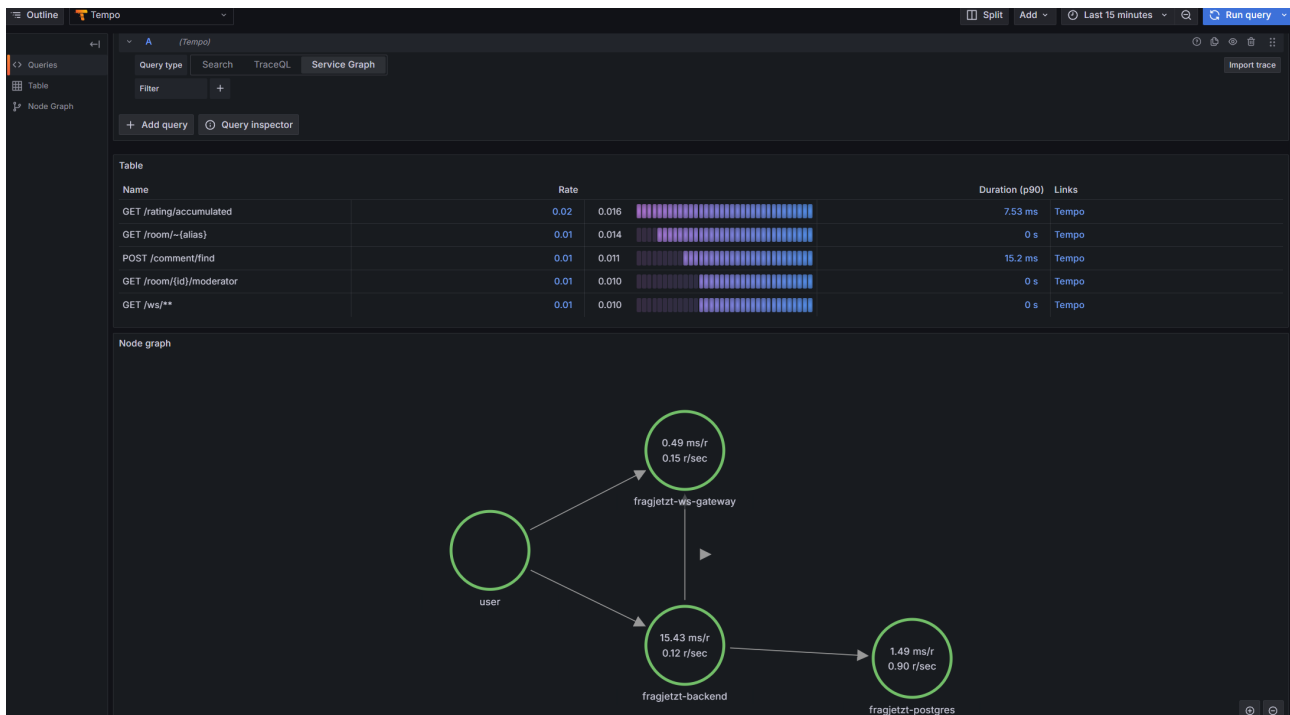


Abbildung 7.2: Service Graph aus Tempo mit den Kommunikationspfaden user → fragjetzt-backend → fragjetzt-postgres sowie user → fragjetzt-ws-gateway. Quelle: Eigene Darstellung.

7.2.5 Exemplarische Validierungsszenarien

Um die Funktionsfähigkeit der Telemetrie-Pipeline zu belegen, wurden drei Szenarien auf dem Staging-Server durchgeführt.

Szenario 1: Normaler Nutzerrequest (Frage stellen). Ein Nutzer erstellte über die Weboberfläche eine neue Frage. In Grafana war der entsprechende Trace mit 8 Spans sichtbar, darunter der INSERT-Span auf der PostgreSQL-Datenbank. Das Latenz-Panel zeigte den Request im p50-Bereich bei circa 12 ms.

Szenario 2: Erhöhte Last auf Voting-Endpunkt. Durch wiederholte manuelle Abstimmungen wurde eine erhöhte Anfragerate erzeugt. Im Dashboard stieg die Anfragerate erkennbar an. Der PostgreSQL-Verbindungszähler blieb im unkritischen Bereich, was bei manueller Last erwartbar ist und ein automatisiertes Lastwerkzeug erfordern würde.

Szenario 3: Provozierter Fehlerfall. Ein Request an einen nicht existierenden Endpunkt erzeugte einen 404-Statuscode. In Tempo war der zugehörige Trace mit Fehler-Tag sichtbar, im Errors-Panel erschien der Endpunkt in der Tabelle der fehlerhäufigsten Span-Operationen. Der 5xx-Alert wurde erwartungsgemäß nicht ausgelöst.

7.3 Umsetzung des Golden-Signals-Dashboards

Das Dashboard enthält 17 Panels in fünf Sektionen, die in fünf Sektionen unterteilt sind und die Signal-Matrix aus Kapitel 6.3 visuell umsetzen. Die Sektionen *Latency* und *Traffic* sind beispielhaft in Abbildung 7.3 dargestellt. Im Bereich *Latency* werden die Backend-Antwortzeiten als Perzentil-Zeitreihen (p50, p95, p99) sowie eine aufgeschlüsselte Latenzansicht pro Endpunkt, basierend auf der Metrik `http_server_request_duration_seconds_bucket` visualisiert. Die Sektion *Traffic* stellt die Anfragerate pro Dienst aus Span-Metriken sowie die meistfrequentierten Endpunkte als gestapeltes Balkendiagramm dar. Die Sektion *Errors* kombiniert eine Span-basierte Fehlerrate pro Service mit einem numerischen Indikator für die HTTP-5xx-Quote. Wie in Abbildung 7.4 ersichtlich, werden zusätzlich die fehlerhäufigsten Span-Operationen tabellarisch aufgeführt. Die Sektion *Saturation* umfasst JVM-Heap-Auslastung, Thread-Zähler und CPU-Nutzung als Kapazitätsanzeige mit Schwellenwertfärbung. Die *Infrastruktur*-Sektion zeigt PostgreSQL-Verbindungen, RabbitMQ-Queue-Tiefen, Host-Ressourcen, Garbage-Collection-Pausen und den Service Graph aus Tempo.

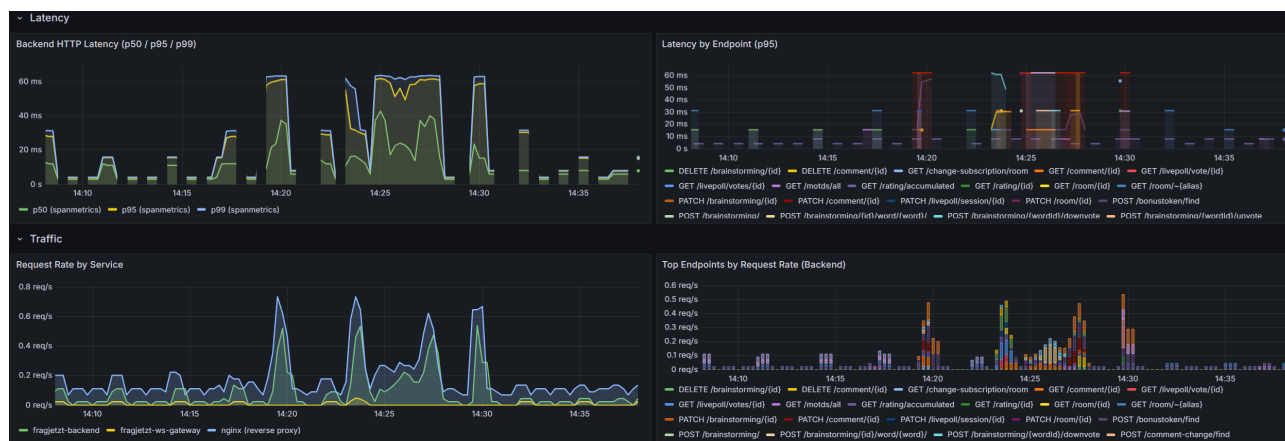


Abbildung 7.3: Ausschnitt des Golden-Signals-Dashboards mit den Sektionen *Latency* und *Traffic*. Quelle: Eigene Darstellung.

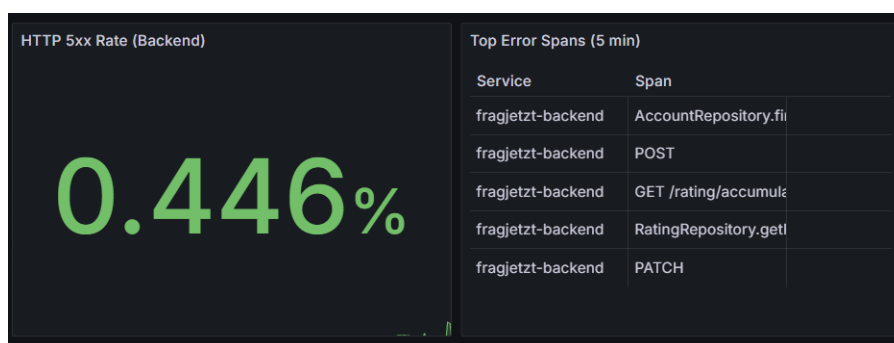


Abbildung 7.4: Ausschnitt der *Errors*-Sektion mit Dummy-Fehlerrate zur Veranschaulichung der Fehlerindikatoren. Quelle: Eigene Darstellung.

Sämtliche Panels verwenden `$_rate_interval` als dynamisches Zeitfenster für Rate-Berechnungen. Die vollständige JSON-Definition befindet sich in Anhang F.

7.4 Alerting-Regeln und Beispiel-Runbooks

Die Alerting-Logik aus Kapitel 6.5 wird exemplarisch in drei Grafana-Alerting-Regeln überführt, die Anforderung A4 adressieren.

Die erste Regel (*Critical*) löst aus, wenn die HTTP-5xx-Fehlerrate des Backends über fünf Minuten den Schwellenwert von 5 % überschreitet. Die zweite Regel (*Warning*) reagiert auf eine PostgreSQL-Verbindungsauslastung über 70 % des konfigurierten Limits von 100 Verbindungen. Die dritte Regel (*Warning*) überwacht die Backend-Latenz im p95-Bereich und alarmiert bei einer anhaltenden Überschreitung von 500 ms über drei Minuten.

Die Evaluationsfenster von drei bis fünf Minuten fungieren als Debounce-Mechanismus, der transiente Spitzen toleriert (Vinnakota & Kolla, 2025, S. 176). Jede Regel enthält in ihrer Annotation-Konfiguration einen Link zum zugehörigen Runbook. Die drei Regeln wurden im Prototyp angelegt und konfiguriert. Ihre Verifikation erfolgte durch gezielte Belastungsszenarien auf dem Staging-Server und wird in Kapitel 8 dokumentiert.

Ergänzend wurden drei Runbooks als strukturierte Dokumente erstellt, die der Fünf-Felder-Struktur (Symptom, Kontext, Diagnose, Behebung, Eskalation) aus Kapitel 6.5 folgen. Das erste behandelt den Alarm “Erhöhte Backend-Latenz”, das zweite den Alarm “Backend-Fehlerrate über Schwellenwert” und das dritte den Alarm “Infrastruktur-Sättigung”. Die Runbooks befinden sich im Anhang E.

7.5 Herausforderungen und Lösungen bei der Umsetzung

Drei Kategorien von Herausforderungen traten auf, die in vergleichbaren Deployments ebenfalls zu erwarten wären.

7.5.1 Ressourcendimensionierung

Tempo wurde initial mit einem Speicherlimit von 256 MB konfiguriert. Unter der Trace-Last erschöpfte der Container seinen verfügbaren Speicher und wurde wiederholt durch den Kernel beendet (OOM-Kill). Jeder Neustart hinterließ unvollständige WAL-Dateien, die beim nächsten Start erneut Fehler verursachten. Die Lösung bestand in der Erhöhung des Speicherlimits auf 512 MB und der Bereinigung des WAL-Volumes. Das Problem veranschaulicht, dass die Ressourcendimensionierung von Observability-Komponenten auf einem Single-Host-System sorgfältige Abstimmung erfordert (Bissig, 2024, S. 52).

7.5.2 Netzwerktopologie und Dienstkommunikation

Tempo generiert aus eingehenden Traces Span-Metriken, die über Remote Write an Prometheus übermittelt werden und den Service Graph in Grafana speisen. Im initialen Deployment scheiterte

dieser Export an der Netzwerktrennung zwischen den Compose-Projekten. Die Lösung bestand darin, Prometheus in beiden Docker-Netzwerken verfügbar zu machen.

7.5.3 Signalqualität und Rauschunterdrückung

Die Auto-Instrumentierung erfasst alle erkennbaren Operationen, ohne zwischen geschäftsrelevanten und internen Abläufen zu unterscheiden. Periodische Hintergrundprozesse wie der `ScheduledMessageSender` dominierten die Trace-Liste und machten diagnostisch relevante Traces kaum auffindbar. Statt Sampling, das auch relevante Daten verlieren kann, wurden gezielt Span-Namen ohne diagnostischen Wert auf Collector-Ebene über einen `filter/traces-Processor` verworfen.

Tabelle 7.2 ordnet die umgesetzten Komponenten den Anforderungen aus Kapitel 3 zu und benennt die jeweiligen Einschränkungen.

Tabelle 7.2: Zuordnung der prototypischen Umsetzung zu den Observability-Anforderungen.
Quelle: Eigene Darstellung.

Anf.	Umsetzung im Prototyp	Einschränkung
A1	Distributed Tracing über OTel Agent; Span-Waterfall und Service Graph in Grafana; Trace-basierte Validierung in Szenario 1	Nur Backend und WS-Gateway instrumentiert; AI Provider und Frontend nicht abgedeckt
A2	Applikations- und Infrastrukturmetriken in Prometheus; Dashboard-Sektionen entlang der vier Golden Signals	Keine Browser-Telemetrie; Frontend-Ebene fehlt
A3	Log-Weiterleitung an Loki über OTel Agent mit automatischer Trace-ID-Injektion	Debug-Level-Logging als Demonstrationskonfiguration; kein strukturiertes JSON-Format
A4	Drei symptomorientierte Alerting-Regeln mit Debounce-Logik; drei Runbooks mit Fünf-Felder-Struktur; alle drei Alerts in Belastungsszenarien ausgelöst	Kurzfristige Fehler-Metriken lieferten keine konsistent aktuellen Werte; Sättigungs-Alert mit angepasstem Schwellenwert validiert; keine dynamischen Schwellenwerte
A5	cAdvisor, Node Exporter, PostgreSQL Exporter, RabbitMQ-Plugin; Infrastrukturektion im Dashboard	Im Prototyp für die vorhandene Infrastruktur abgebildet

8 Evaluation der entwickelten Strategie

Das vorliegende Kapitel prüft, inwieweit die in Kapitel 6 konzipierte und in Kapitel 7 prototypisch umgesetzte Observability-Strategie die Anforderungen aus Kapitel 3 erfüllt.

8.1 Evaluationsmethodik

Die Evaluation folgt dem in Abschnitt 4.3 definierten Bewertungsrahmen mit drei Evidenzquellen: Artefaktanalyse, Konfigurations- und Instrumentierungsprüfung sowie Last- und Störungsszenarien. Im Prototyp (Kapitel 7) wurden die ersten beiden Quellen ausgeschöpft. Die dritte blieb auf manuelle Plausibilitätstests beschränkt. Um diese Lücke zu verkleinern, wurden drei gezielte Belastungsszenarien auf dem Staging-Server durchgeführt: CPU-Last auf dem Backend-Container für den Latenz-Alert, temporäre Unterbrechung der Datenbankverbindung für den Fehler-Alert und Erschöpfung des PostgreSQL-Verbindungspools für den Sättigungs-Alert. Ergänzend wurde die bidirektionale Log-Trace-Korrelation auf dem Staging-Server verifiziert.

8.2 Anforderungsbezogene Bewertung

Tabelle 8.1 gibt vorab einen kompakten Überblick über die Erfüllungsgrade. Die folgenden Unterabschnitte begründen diese Einstufungen.

Tabelle 8.1: Ergebnissynthese der anforderungsbezogenen Evaluation. Quelle: Eigene Darstellung.

Anf.	Kurzbefund	Wesentliche Einschränkung	Erfüllungsgrad
A1	Backend-Pfad rekonstruierbar, Gesamtsystem unvollständig	AI Provider und Frontend nicht instrumentiert	teilweise erfüllt
A2	Drei Messebenen für instrumentierte Dienste vorhanden	Keine Browser-Telemetrie	weitgehend erfüllt
A3	Bidirektionale Log-Trace-Korrelation validiert	Debug-Level-Logging, noch nicht produktionsorientiert strukturiert	erfüllt
A4	Alle drei Alerts in Belastungsszenarien ausgelöst; Multi-Signal-Logik validiert	Kurzfristige Fehler-Metriken lieferten keine konsistent aktuellen Werte; Sättigungs-Alert mit angepasstem Schwellenwert getestet	weitgehend erfüllt
A5	Infrastrukturmetriken verfügbar und plausibel	Nicht alle Indikatoren unter Last geprüft	weitgehend erfüllt

8.2.1 A1: Ende-zu-Ende-Nachvollziehbarkeit

Prüffrage: Lässt sich ein Request Ende-zu-Ende rekonstruieren?

Die Instrumentierung über den OTEL Java Agent erzeugt für jeden eingehenden HTTP-Request einen Root-Span mit untergeordneten Spans für Datenbankoperationen und interne Verarbeitungsschritte. Abbildung 8.1 zeigt exemplarisch einen Trace für den Endpunkt GET /room/{id}/moderator mit 11 Spans, darunter mehrere SELECT-Operationen auf der PostgreSQL-Datenbank. Der Verarbeitungspfad vom HTTP-Eingang bis zur Datenpersistierung ist vollständig rekonstruierbar. Der Service Graph in Grafana bildet die Kommunikationsbeziehungen zwischen den drei instrumentierten Diensten ab.

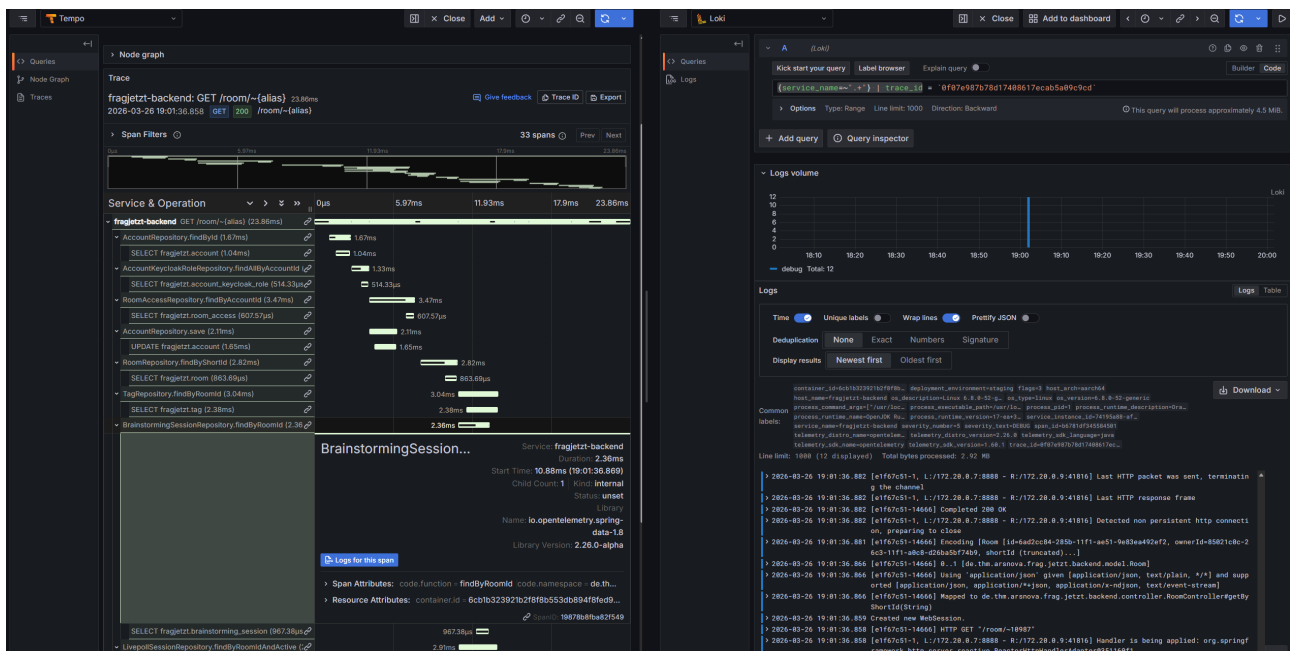


Abbildung 8.1: Trace-Waterfall in Tempo für den Endpunkt GET /room/{id}/moderator mit 11 Spans und zugeordneten Datenbankoperationen. Der rechte Bereich zeigt die korrelierten Loki-Logs, erreichbar über die Funktion *Logs for this span*. Quelle: Eigene Darstellung (Screenshots, Staging-Server).

Die Nachvollziehbarkeit endet an den Grenzen der instrumentierten Komponenten. Der AI Provider und das Frontend sind nicht instrumentiert. Für den instrumentierten Backend-Pfad ist die Request-Rekonstruktion funktional gegeben, für das Gesamtsystem bleibt sie unvollständig. Der Erfüllungsgrad wird daher als *teilweise erfüllt* bewertet.

8.2.2 A2: Mehrstufige Messbarkeit entlang der Golden Signals

Prüffrage: Deckt die Instrumentierung alle drei Messebenen ab?

Das Golden-Signals-Dashboard bildet alle vier Dimensionen in separaten Sektionen ab. Im Belastungsszenario stieg die p95-Latenz unter CPU-Last von rund 20 ms auf über 500 ms, was im Dashboard sofort sichtbar wurde. Nach dem temporären Stopp der Datenbank stieg die 5xx-Rate und die betroffenen Endpunkte erschienen in der Fehlertabelle. Die drei Messebenen,

Applikationsmetriken über den OTel Agent, Infrastrukturmetriken über Exporter und Trace-basierte Span-Metriken über Tempo, sind für die instrumentierten Dienste abgedeckt. Die Lücke betrifft die fehlende Frontend-Telemetrie. Der Erfüllungsgrad wird als *weitgehend erfüllt* eingestuft.

8.2.3 A3: Strukturiertes, korrelierbares Logging

Prüffrage: Können Logeinträge über Korrelations-IDs verknüpft werden?

Die bidirektionale Korrelation zwischen Logs und Traces konnte auf dem Staging-Server nachgewiesen werden. Von einem Trace in Tempo führt die Funktion *Logs for this span* zu einer Loki-Abfrage, die nach Trace-ID und Dienstname filtert. In der Gegenrichtung enthält jeder Log-Eintrag in Loki ein klickbares *TraceID*-Feld, das direkt zum zugehörigen Trace navigiert. Diese Verknüpfung wurde über ein Derived Field in der Loki-Datenquellenkonfiguration realisiert.

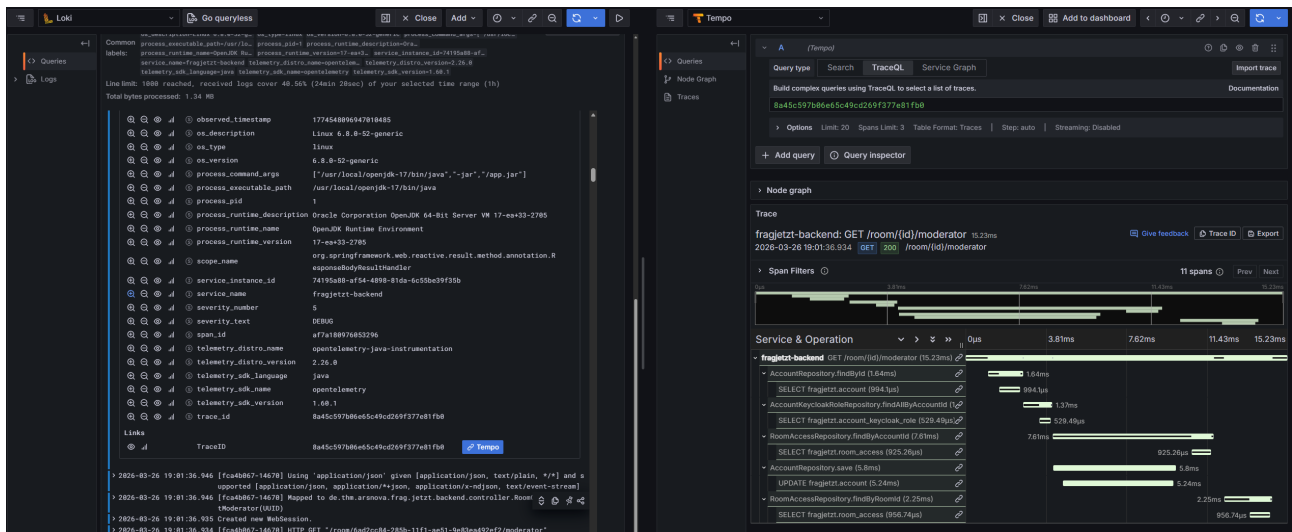


Abbildung 8.2: Log-to-Trace-Korrelation in Loki: Ein Logeintrag mit klickbarem TraceID-Link (links) navigiert zum zugehörigen Trace-Waterfall in Tempo (rechts). Quelle: Eigene Darstellung.

Da die Kernfunktion der Korrelierbarkeit nachgewiesen wurde, wird A3 als *erfüllt* bewertet. Für den Produktivbetrieb wäre eine Reduktion auf strukturierte JSON-Protokollierung erforderlich.

8.2.4 A4: Handlungsfähige Alarmierung

Prüffrage: Reagieren Alarme auf nutzerwirksame Zustände und enthalten sie ausreichend Kontext für eine Erstreaktion?

Der Latenz-Alert wurde durch künstlich erzeugten CPU-Druck auf den Backend-Container im Zustand *Firing* ausgelöst. Abbildung 8.3 zeigt den Alarm mit Symptombeschreibung, Prüfpfad und beispielhaftem Runbook-Link in den Annotations. Die Multi-Signal-Logik bewährte sich. In Leerlaufphasen ohne aktive Requests verhinderte die Mindest-Traffic-Rate von 0,1 req/s, dass der Alert fälschlicherweise auslöst.

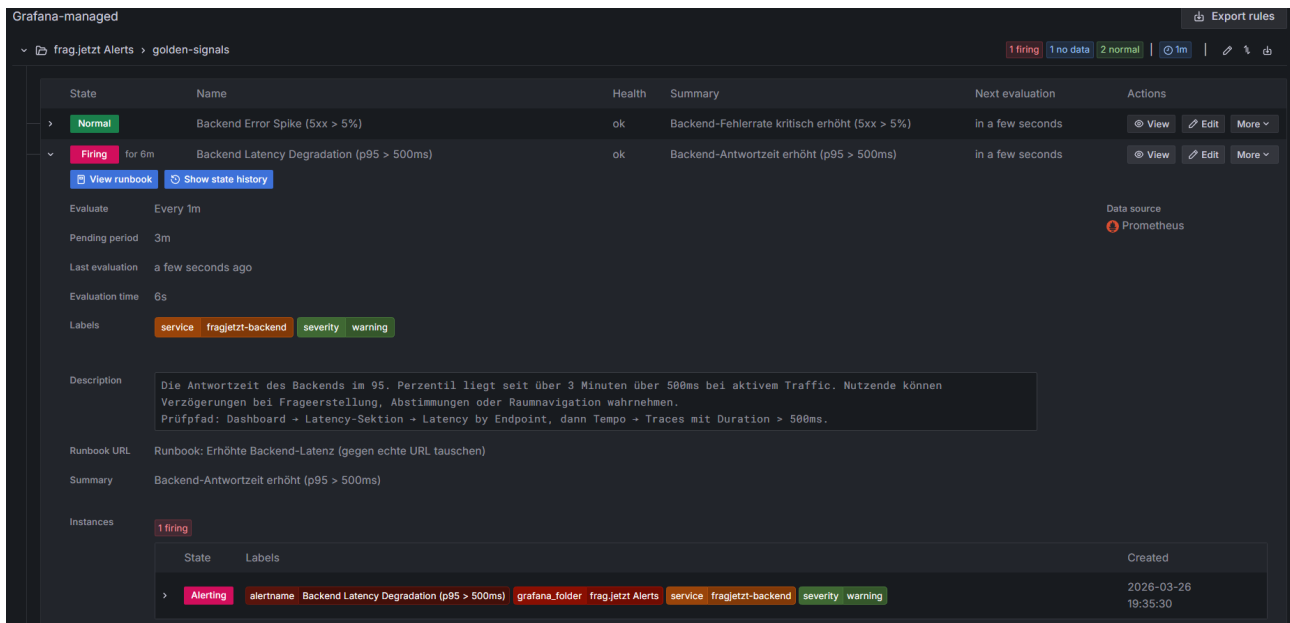


Abbildung 8.3: Latenz-Alert im Zustand *Firing* nach 6 Minuten. Die Annotations enthalten Symptombeschreibung, Prüfpfad und Runbook-Link. Quelle: Eigene Darstellung.

Der Fehler-Alert konnte zuverlässig ausgelöst werden. Die Rate-Berechnung lieferte allerdings bei kurzen Betrachtungszeiträumen keine konsistenten Werte, vermutlich aufgrund einer Diskrepanz zwischen dem dynamisch berechneten Abfragefenster und der Erfassungssemantik der OTel-basierten Metriken. Der Sättigungs-Alert wurde durch gezielte Erschöpfung des PostgreSQL-Verbindungspools validiert. Über parallele `pg_sleep`-Sitzungen wurden mehr als 70 aktive Verbindungen erzeugt, sodass die erste Bedingung der AND-Logik (Verbindungsauslastung über 70%) dauerhaft erfüllt war. Da auf dem Staging-Server keine authentifizierten Requests mit datenbankgestützter Verarbeitung erzeugt werden konnten, wurde der Latenz-Schwellenwert der zweiten Bedingung für den Test an die Staging-Gegebenheiten angepasst. Nach Ablauf der fünfminütigen Pending-Periode wechselte der Alert korrekt in den Zustand *Firing*. Die Multi-Signal-Logik, bei der beide Bedingungen gleichzeitig über die gesamte Debounce-Dauer erfüllt sein müssen, wurde damit funktional bestätigt.

Für den Latenz- und den Sättigungs-Alert ist die Alerting-Konzeption funktional validiert. Für den Fehler-Alert zeigt sich ein Datenqualitätsproblem in der kurzfristigen Metrikdarstellung, das die konzeptionelle Validität der Regel nicht infrage stellt, aber die operative Kurzfristdiagnose einschränkt. Da alle drei Regeln im Belastungstest korrekt ausgelöst wurden und die Multi-Signal-Logik falsch-positive Auslösungen in Leerlaufphasen nachweislich verhinderte, wird der Erfüllungsgrad als *weitgehend erfüllt* eingestuft. Einschränkend bleibt, dass der Sättigungs-Alert mit einem angepassten Schwellenwert getestet wurde und die Fehler-Metriken bei kurzen Zeitfenstern Inkonsistenzen aufwiesen.

8.2.5 A5: Beobachtbarkeit der Infrastrukturschicht

Prüffrage: Werden Container-Ressourcen, Datenbankpool-Auslastung und Queue-Tiefen erfasst?

Die Infrastruktur-Sektion des Dashboards beantwortet diese Frage positiv. Abbildung 8.4 zeigt PostgreSQL-Verbindungszähler, RabbitMQ-Queue-Tiefen, Host-Ressourcen und den Service Graph. Während der Belastungstests stieg die Zahl aktiver PostgreSQL-Verbindungen von einem Normalniveau um 3 auf temporär über 10. Alle fünf Infrastruktur-Exporter lieferten während des gesamten Evaluationszeitraums durchgängig Metriken. Da die Metriken verfügbar und plausibel sind, nicht aber alle Indikatoren unter gezielter Last geprüft wurden, wird A5 als *weitgehend erfüllt* bewertet.



Abbildung 8.4: Infrastruktur-Sektion des Golden-Signals-Dashboards mit PostgreSQL-Verbindungen, RabbitMQ-Queue-Tiefe, Host-Ressourcen und Service Graph. Quelle: Eigene Darstellung.

8.3 Übergreifende Bewertungsdimensionen

Stakeholder-Tauglichkeit. Für die DevOps-Perspektive ist der Prototyp funktional nutzbar. Dashboard, Alerting-Regeln und Runbooks liefern eine zusammenhängende Diagnoseinfrastruktur. Für die Entwicklungsperspektive ermöglicht Grafana Explore die direkte Abfrage von Traces, Logs und Metriken mit bidirektionaler Korrelation, ein dediziertes Entwickler-Dashboard wurde jedoch nicht umgesetzt. Für die Product-Owner-Perspektive fehlt im Prototyp eine eigene Sicht, die technisch aus den vorhandenen Datenquellen ableitbar wäre.

Abdeckung der Risikopunkte. Von den sechs identifizierten Risikopunkten werden R2 (DB-Engpässe), R3 (Intransparenz des Backends) und R5 (Ressourcenverdrängung) durch den Prototyp direkt adressiert. R3 wurde durch die Einführung von Distributed Tracing im instru-

mentierten Backend-Pfad substanziell reduziert. R4 (WebSocket-Verbindungsverlust) wird auf Beobachtungsebene teilweise adressiert, weil RabbitMQ-Queue-Tiefen und das WS-Gateway sichtbar gemacht werden. Der zugrunde liegende Ausfallpfad bleibt jedoch bestehen. R1 (fehlende Timeouts und Resilienz) sowie R6 (Single-Host-Topologie als systemweiter Ausfallpfad) bleiben als architektonische Defizite bestehen, die zwar durch Observability sichtbar, aber nicht durch den Prototyp selbst behoben werden.

Umsetzungsrealismus. Die praktische Implementierung des Observability-Stacks erwies sich als architektonisch komplexer und zeitaufwendiger als initial angenommen. Insbesondere die korrekte Konfiguration der Telemetrie-Pipeline, die feingranulare Abstimmung der verschiedenen Speicher-Backends sowie das Ressourcenmanagement auf dem Single-Host-System erforderten einen erheblichen iterativen Aufwand. Diese unvorhergesehene Komplexität war der primäre Grund für die in Abschnitt 7.1 vorgenommene Abgrenzung des Prototyps. Um eine funktionsfähige und stabile Kerninfrastruktur zu gewährleisten, musste der Implementierungsumfang bewusst reduziert werden. Dies führt dazu, dass nicht alle gestellten Anforderungen im Rahmen dieser Arbeit vollständig erfüllt werden konnten.

8.4 Grenzen und methodische Einschränkungen

Die Evaluation unterliegt fünf Einschränkungen.

Erstens, unvollständige Dienstabdeckung. Backend und WS-Gateway sind instrumentiert, AI Provider und Frontend nicht. Die Bewertung von A1 und A2 gilt nur für einen Teil des Gesamtsystems.

Zweitens, Stabilität des Observability-Stacks unter Last. Tempo wurde während der Belastungstests durch Speicherschöpfung beendet (OOM-Kill), woraufhin auch der OTEL Collector den Betrieb einstellte. Infrastrukturmetriken über die Prometheus-Exporter blieben unberührt. Der Vorfall verdeutlicht, dass der Observability-Stack selbst eine Fehlerquelle darstellt, deren Auswirkungen auf einem Single-Host-System die Beobachtbarkeit des Gesamtsystems beeinträchtigen.

Drittens, eingeschränkte Aussagekraft der Rate-Berechnung. Bei kurzen Betrachtungszeiträumen lieferte die PromQL-Variable `$_rate_interval` inkonsistente Ergebnisse für die HTTP-5xx-Rate. In längeren Zeitfenstern (ab circa 12 Stunden) waren die Werte korrekt. Diese Einschränkung betrifft die Dashboard-Darstellung, nicht die Alerting-Regeln mit festen Zeitfenstern.

Viertens, fehlende Produktivdaten. Alle Tests wurden mit synthetischer Last durchgeführt. Reale Nutzungsmuster, insbesondere die Lastspitzen während Lehrveranstaltungen, konnten nicht abgebildet werden. Die Schwellenwerte müssten im Produktivbetrieb anhand realer Lastprofile kalibriert werden.

Fünftens, prototypische Testmethodik. Die Belastungsszenarien wurden manuell über Konsolenschleifen und Container-Operationen durchgeführt. Für eine belastbarere Validierung wäre ein dedizierter HTTP-Lastgenerator mit kontrollierten Lastprofilen erforderlich.

9 Diskussion

Die Evaluation hat gezeigt, dass eine Observability-Basis für frag.jetzt prototypisch realisierbar ist. Die Aussagekraft dieser Ergebnisse ist jedoch differenziert zu bewerten. Robust belegt sind die Korrelation von Logs und Traces sowie die funktionale Nutzbarkeit symptomorientierter Alert-Regeln für die JVM-basierten Dienste und die Infrastrukturschicht. Vorläufig bleiben dagegen Aussagen zur vollständigen Ende-zu-Ende-Nachvollziehbarkeit und zur Kalibrierung produktionsreifer Schwellenwerte, da Frontend- und AI-Provider-Instrumentierung fehlen und die Validierung ausschließlich unter synthetischer Last erfolgte.

9.1 Einordnung der Ergebnisse im Verhältnis zur Literatur

Die Arbeit bestätigt mehrere Grundannahmen der Observability-Literatur und konkretisiert sie um operative Erkenntnisse aus der prototypischen Umsetzung.

Drei Befunde stützen wesentliche Annahmen der Literatur: Erstens erweist sich die Kombination der drei Telemetriepfeiler als praktisch tragfähig. Erst die Verknüpfung von Span-basierten Applikationsmetriken mit Exporter-basierten Infrastrukturmetriken in einem gemeinsamen Dashboard erzeugt eine diagnostisch nutzbare Gesamtsicht (Albuquerque & Correia, 2025, S. 22). Zweitens hat sich die symptomorientierte Alarmierung bewährt. Der Latenz-Alert verhinderte in der Evaluation falsch-positive Auslösungen in Leerlaufphasen, indem er neben der p95-Überschreitung eine Mindest-Traffic-Rate als Vorbedingung verlangt. Drittens ermöglicht die Auto-Instrumentierung über den OTEL Java Agent einen niedrigschwelligen Einstieg in Distributed Tracing ohne Codeänderungen.

Neben diesen Bestätigungen macht die Arbeit drei Aspekte sichtbar, die in der Literatur eher abstrakt behandelt werden. Der erste betrifft die operative Komplexität. Der OOM-Kill von Tempo, die Netzwerkprobleme in der Docker-Compose-Topologie und das Span-Rauschen durch die Auto-Instrumentierung traten als konkrete Problemkategorien auf, die in den herangezogenen Quellen nicht in dieser Spezifik behandelt werden. Niedermaier et al. (2019, S. 9) identifizieren zwar knappe Ressourcen und reaktive Implementierung als typische Herausforderungen, adressieren aber nicht die Kaskadeneffekte, die auf einem Single-Host-System durch den Observability-Stack selbst entstehen können.

Der zweite Aspekt betrifft die Signalqualität. Statt Sampling, das relevante Daten anteilig verlieren kann, setzt die Arbeit gezieltes Filtering auf Collector-Ebene ein, das diagnostisch wertlose Spans vor der Speicherung verwirft. Dieser Ansatz adressiert ein Problem, das die Literatur unter dem Stichwort Sampling diskutiert (Albuquerque & Correia, 2025, S. 10), ohne eine inhaltlich begründete Selektion auf Collector-Ebene als Lösungsstrategie vorzuschlagen.

Der dritte Aspekt betrifft die Validierbarkeit von Alerting-Regeln. Obwohl alle drei Regeln im Belastungstest korrekt auslösten, zeigte sich bei der Fehler-Rate-Berechnung, dass kurzfristige

Zeitfenster keine konsistent aktuellen Werte lieferten. Auch der Sättigungs-Alert erforderte eine Anpassung des Latenz-Schwellenwerts an die Staging-Bedingungen, weil auf dem Testsystem keine authentifizierten Requests mit datenbankgestützter Verarbeitung provoziert werden konnten. Die Literatur benennt die Notwendigkeit des Testens (Vinnakota & Kolla, 2025, S. 175–176), beleuchtet aber die konkreten Schwierigkeiten, die sich aus dem Zusammenspiel von Metriksemantik, Query-Verhalten und Staging-Verifikation ergeben, nur begrenzt.

Die Arbeit bestätigt damit nicht nur etablierte Annahmen, sondern präzisiert deren praktische Tragweite für kleine Teams und ressourcenbegrenzte Single-Host-Umgebungen. Der Fall `frag.jetzt` zeigt, dass der Hauptaufwand weniger in der theoretischen Ableitung geeigneter Messgrößen als in der operativen Stabilisierung der Beobachtungsinfrastruktur selbst liegt. Allgemeine Empfehlungen zu Tracing, Alerting und standardisierter Instrumentierung bleiben gültig, müssen jedoch um die Perspektive der infrastrukturellen Selbstbeobachtung ergänzt werden.

9.2 Übertragbarkeit auf ähnliche cloud-native Plattformen

Hoch übertragbar sind drei methodische Elemente: die Signal-Matrix-Methodik, die für jeden Dienst die vier Kernmetriken auf Messgrößen, Zwecke und Rollen abbildet, die Fünf-Felder-Runbook-Struktur sowie die symptomorientierte Alert-Logik, die nutzerwirksame Zustände als Auslöser verwendet. Voraussetzung ist eine verteilte Anwendung mit klar trennbaren Diensten und definierten Eskalationspfaden.

Bedingt übertragbar ist der OTel-Agent-basierte Instrumentierungsansatz. Der Java Agent funktioniert besonders reibungsarm in JVM-basierten Umgebungen. Für andere Laufzeitumgebungen kann der Aufwand höher ausfallen (Bissig, 2024, S. 52). Das Grundprinzip der herstellernerneutralen Entkopplung von Instrumentierung und Backend bleibt jedoch unabhängig von der Laufzeitumgebung gültig.

Kontextabhängig sind die konkreten Implementierungsentscheidungen. Der LGTM-Stack hat sich für das Single-Host-Szenario bewährt, stellt aber keine universelle Empfehlung dar. Die Docker-Compose-Topologie und die konkreten Schwellenwerte der Alerting-Regeln sind systemspezifisch und müssen aus den jeweiligen Lastprofilen abgeleitet werden.

9.3 Wissenschaftlicher, technischer und operativer Beitrag

Der wissenschaftliche Beitrag besteht in der methodisch transparenten Ableitungslogik, die von der Systemanalyse über die servicebezogene Operationalisierung bis zur kriteriengestützten Evaluation reicht. Dieses Vorgehen verbindet mehrere in der Literatur häufig getrennt behandelte Konzepte zu einem integrierten Strategieentwurf (Pappula et al., 2021, S. 50). Darüber hinaus liefert die Arbeit fallbezogene operative Hinweise auf Problemkategorien, die in der vorwiegend konzeptionell argumentierenden Literatur wenig Raum einnehmen.

Der technische Beitrag umfasst eine prototypisch funktionsfähige LGTM- und OTel-basierte Infrastruktur mit korrelierbarem Logging, Distributed Tracing und konfigurierten Alert-Regeln. Der operative Beitrag geht darüber hinaus und stellt mit der Signal-Matrix, der symptom-orientierten Alert-Logik und den Fünf-Felder-Runbooks Artefakte bereit, die auch ohne den spezifischen Technologie-Stack adaptiert werden können.

Die positiven Ergebnisse sind dabei nicht ausschließlich als Effekt des gewählten Stacks zu interpretieren. Ein wesentlicher Anteil des Nutzens dürfte auf die vorgelagerte methodische Strukturierung zurückzuführen sein, insbesondere auf die servicebezogene Ableitung relevanter Messgrößen, die Reduktion diagnostisch wertlosen Rauschens und die Verknüpfung von Alerts mit Runbooks. Der Technologie-Stack ist damit Ermöglicher, nicht alleiniger Träger des erzielten Mehrwerts.

Bezogen auf die Forschungsfragen zeigt die Arbeit für FF1 eine methodisch tragfähige, aber noch nicht vollständige serviceübergreifende Operationalisierung der Four Golden Signals. FF2 wurde im Rahmen der definierten Anforderungen und Randbedingungen belastbar beantwortet, bleibt jedoch auf ein Single-Host-Szenario zugeschnitten. FF3 wurde konzeptionell und prototypisch überzeugend adressiert, erreicht seine volle Reife jedoch erst nach produktionsnaher Kalibrierung und Ergänzung weiterer Stakeholder-Sichten.

10 Fazit und Ausblick

Die vorliegende Arbeit ging von einem konkreten betrieblichen Defizit aus: Die Plattform `frag.jetzt` verfügte über eine funktionsfähige Deployment-Infrastruktur, erhob jedoch weder Metriken noch Traces noch korrelierbare Logs. Störungen wurden erst sichtbar, wenn Endnutzende bereits betroffen waren. Um dieses Defizit zu adressieren, wurde eine Observability-Strategie entwickelt, die von der Systemanalyse über die servicebezogene Operationalisierung der Four Golden Signals und die kriteriengestützte Stack-Auswahl bis zur prototypischen Umsetzung und Evaluation reicht. Die folgenden Abschnitte beantworten die drei Forschungsfragen, verdichten die wesentlichen Erkenntnisse und skizzieren priorisierte Anknüpfungspunkte für die Weiterentwicklung.

10.1 Beantwortung der Forschungsfragen

FF1: Wie lassen sich die Four Golden Signals für die spezifischen Services (PWA-Frontend, Spring-Boot-Backend, PostgreSQL, KI-Services) von `frag.jetzt` konkret definieren und instrumentieren?

Die Arbeit beantwortet diese Frage durch eine fünfstufige Ableitungslogik, die für jeden Dienst funktionale Rolle, kritische Interaktionen, geeignete Messgrößen, initiale Schwellenwerte sowie Visualisierungs- und Alarmierungskonsequenzen bestimmt. Das Ergebnis ist eine serviceübergreifende Signal-Matrix, die jedem Dienst pro Golden Signal eine primäre Messgröße, einen Erhebungszweck und eine Stakeholder-Zuordnung zuweist. Für die JVM-basierten Dienste ließ sich diese Ableitung über die Auto-Instrumentierung des OpenTelemetry Java Agent in eine funktionsfähige Telemetrie-Pipeline überführen, die HTTP-Spans, Datenbankoperationen und JVM-Metriken ohne Codeänderungen erfasst. Die Ableitung ist damit methodisch vollständig, praktisch jedoch bisher nur für einen Teil des Gesamtsystems nachgewiesen. Für den AI Provider und das Frontend steht die Instrumentierung noch aus.

FF2: Welche Observability-Stack-Kombination erfüllt die Anforderungen von `frag.jetzt` unter Berücksichtigung der Single-Host-Topologie und der begrenzten Teamressourcen am besten?

Der kriteriengestützte Vergleich dreier Architekturkandidaten ergab den LGTM-Stack aus Prometheus, Loki, Tempo und Grafana als geeignetste Lösung. Drei Argumente waren ausschlaggebend: der geringere Integrationsaufwand gegenüber Elastic Observability, die enge bidirektionale Korrelation zwischen Metriken, Traces und Logs innerhalb einer einzigen Oberfläche sowie die Austauschbarkeit einzelner Backend-Komponenten bei stabilem Instrumentierungskern über OpenTelemetry. Die prototypische Umsetzung auf dem Staging-Server bestätigte

die Praktikabilität dieser Wahl, machte aber auch Ressourcenkonflikte sichtbar, die auf einem Single-Host-System besondere Aufmerksamkeit erfordern.

FF3: Wie kann ein intelligentes Alerting-System konzipiert werden, das False Positives reduziert und actionable Insights für verschiedene Stakeholder-Rollen (DevOps, Entwickler, Product Owner) bereitstellt?

Das entwickelte Alerting-Framework verfolgt zwei Prinzipien: symptomorientierte Auslösung, bei der Alarme an nutzerwirksame Zustände gekoppelt werden, und Multi-Signal-Logik, bei der ein primäres Fehlersignal erst in Kombination mit einer Mindest-Traffic-Rate als handlungsrelevant gewertet wird. Drei exemplarische Regeln wurden konfiguriert und mit strukturierten Runbooks in einer Fünf-Felder-Struktur verknüpft. Alle drei Alerts konnten in der Evaluation durch gezielte Belastungsszenarien ausgelöst werden. In Leerlaufphasen ohne aktive Anfragen verhinderte die Traffic-Bedingung eine falsch-positive Auslösung, unter künstlicher CPU-Last feuerte der Latenz-Alarm korrekt. Der Sättigungs-Alert wurde durch Erschöpfung des PostgreSQL-Verbindungspools mit angepasstem Schwellenwert validiert und bestätigte die AND-Logik über die gesamte Debounce-Dauer. Für den Fehler-Alert zeigte sich ein Datenqualitätsproblem bei kurzfristigen Rate-Berechnungen, das die konzeptionelle Validität nicht infrage stellt. Das Konzept ist damit funktional abgesichert, bedarf aber für den Produktivbetrieb einer Kalibrierung der Schwellenwerte anhand realer Lastprofile.

Zusammenfassend lässt sich festhalten, dass das betriebliche Ausgangsdefizit nicht vollständig beseitigt, aber substanziell verkleinert wurde. frag.jetzt ist von einem weitgehend intransparenten Produktionsbetrieb ohne jegliche Telemetrie zu einer teilweise validierten, methodisch fundierten Observability-Basis überführt worden. Von den fünf Anforderungen aus Kapitel 3 wurden A3 (korrelierbare Logs) als erfüllt und A2 (mehrstufige Messbarkeit), A4 (handlungsfähige Alarmierung) sowie A5 (Infrastrukturbeobachtbarkeit) als weitgehend erfüllt bewertet. A1 (Ende-zu-Ende-Nachvollziehbarkeit) erreichte als einziges den Grad *teilweise erfüllt*, weil die Instrumentierungsabdeckung unvollständig blieb. Die verbleibenden Lücken sind identifiziert und adressierbar, ohne die grundlegende Architektur verändern zu müssen.

10.2 Zentrale Erkenntnisse und übertragbare Artefakte

Über die Beantwortung der Forschungsfragen hinaus lassen sich drei übergreifende Erkenntnisse festhalten, die auch für vergleichbare Projekte von Bedeutung sind.

Die erste betrifft die operative Komplexität einer Observability-Einführung. Die prototypische Umsetzung zeigte, dass die eigentlichen Engpässe weniger in der konzeptionellen Ableitung von Messgrößen liegen als in deren technischer Realisierung. Speichererschöpfung bei Tempo, Netzwerktrennung zwischen Docker-Compose-Projekten und diagnostisch wertloses Span-Rauschen durch Auto-Instrumentierung traten als wiederkehrende Problemkategorien auf. Diese operativen Hürden werden in der herangezogenen Literatur eher abstrakt behandelt (Niedermaier et al.,

2019, S. 7–9), erwiesen sich im vorliegenden Prototyp jedoch als die bestimmenden Faktoren für den Implementierungsaufwand. Insbesondere der OOM-Kill von Tempo verdeutlichte, dass die Beobachtungsinfrastruktur selbst zur Fehlerquelle werden kann, wenn ihre Ressourcendimensionierung nicht sorgfältig auf die verfügbaren Host-Kapazitäten abgestimmt ist (Bissig, 2024, S. 52).

Die zweite Erkenntnis betrifft die praktische Einstiegshürde. Die Auto-Instrumentierung über den OpenTelemetry Java Agent ermöglichte die Erfassung von HTTP-Spans, Datenbankoperationen und JVM-Metriken ohne Codeänderungen an der Anwendung. Für ein Projekt, das Observability erstmals einführt und über begrenzte Teamressourcen verfügt, senkt dieser Ansatz den initialen Aufwand erheblich. Zugleich erfordert die resultierende Signalfut eine bewusste Qualitätssicherung. Statt stochastischem Samplings, das relevante Traces anteilig verlieren kann, erwies sich im vorliegenden Prototyp ein inhaltlich begründetes Filtering auf Collector-Ebene als zweckmäßig: Periodische Hintergrundprozesse ohne diagnostischen Wert wurden gezielt verworfen, sodass die verbleibenden Traces eine höhere Aussagekraft besaßen.

Die dritte Erkenntnis betrifft den eigentlichen Mehrwert der Arbeit über den Einzelfall hinaus. Drei Artefakte sind als methodische Bausteine auch auf vergleichbare cloud-native Plattformen übertragbar: die Signal-Matrix-Methodik, die für jeden Dienst die vier Golden Signals auf konkrete Messgrößen, Erhebungszwecke und Stakeholder-Rollen abbildet, die symptomorientierte Multi-Signal-Alert-Logik, die nutzerwirksame Zustände als Auslöser mit einer Traffic-Vorbedingung verknüpft, und die Fünf-Felder-Runbook-Struktur, die jeden Alarm mit Symptom, Kontext, Diagnoseschritten, Erstmaßnahmen und Eskalationspfad verbindet. Diese Artefakte sind unabhängig vom konkreten Technologie-Stack einsetzbar.

10.3 Ausblick

Die prototypische Umsetzung hat eine funktionsfähige Grundlage geschaffen, deren Weiterentwicklung sich nach Dringlichkeit und Nähe zu den beobachteten Einschränkungen priorisieren lässt.

Den unmittelbar nächsten Schritt bildet die Validierung unter produktionsnahen Bedingungen. Die Belastungsszenarien in Kapitel 8 wurden manuell über Konsolen-Schleifen durchgeführt und bilden reale Nutzungsmuster nur eingeschränkt ab. Ein dedizierter HTTP-Lastgenerator mit kontrollierten Lastprofilen, ergänzt um gezielte Fault-Injection-Szenarien, würde die Aussagekraft der Alerting-Regeln und Schwellenwerte erheblich stärken. Auch das in der Evaluation beobachtete Datenqualitätsproblem bei kurzfristigen Rate-Berechnungen ließe sich unter systematischeren Testbedingungen gezielter eingrenzen.

Der zweite Schritt betrifft die Erweiterung der Instrumentierungsabdeckung. Der AI Provider ist als Python-basierter Dienst mit FastAPI nicht durch den Java Agent abgedeckt und müsste über das OpenTelemetry Python SDK instrumentiert werden. Ebenso fehlt eine Browser-Telemetry für das Angular-Frontend. Erst mit diesen Erweiterungen ließe sich die Ende-zu-Ende-

Nachvollziehbarkeit (A1) vollständig realisieren und die in Kapitel 6 konzipierte Signal-Matrix auch für die bisher nicht instrumentierten Dienste praktisch einlösen.

Drittens steht die Ableitung formaler Service Level Objectives aus. Die Arbeit hat SLOs bewusst ausgeklammert, weil für frag.jetzt keine historischen Produktivdaten vorliegen. Sobald die Instrumentierung im Produktivbetrieb aktive Lastprofile erfasst, können aus den erhobenen Metriken SLIs abgeleitet und mit Zielwerten hinterlegt werden. Dieser Schritt würde das regelbasierte Alerting um eine geschäftsbezogene Steuerungsebene erweitern (Niedermaier et al., 2019, S. 11–12).

Viertens hat die prototypische Umsetzung gezeigt, dass die Ressourcendimensionierung des Observability-Stacks auf einem Single-Host-System sorgfältiger Abstimmung bedarf. Eine systematische Ermittlung von Mindest-, Durchschnitts- und Maximalverbräuchen der einzelnen Observability-Container unter verschiedenen Lastszenarien würde die Grundlage für belastbare Speicherlimits schaffen und das Risiko von OOM-Kills reduzieren.

Fünftens fehlt im Prototyp die in Kapitel 6 konzipierte rollenspezifische Dashboard-Sicht für den Product Owner. Die Evaluation hat gezeigt, dass die DevOps-Perspektive funktional nutzbar und die Entwicklungsperspektive über Grafana Explore adressierbar ist. Für die Product-Owner-Perspektive, die verdichtete Indikatoren wie aktive Raumsitzungen und globale Fehlerraten zeigen könnte, wäre ein eigenes Dashboard aus den vorhandenen Datenquellen ableitbar und würde die Stakeholder-Tauglichkeit der Strategie vervollständigen.

Langfristig könnte das regelbasierte Alerting um anomaliebasierte Verfahren ergänzt werden (Mistry, 2025, S. 40). Statische Schwellenwerte bilden saisonale Lastschwankungen nur unzureichend ab (Vinnakota & Kolla, 2025, S. 174). Ansätze, die aus historischen Metriken dynamische Baselines ableiten, könnten die Erkennungsrate verbessern und die Anzahl falsch-positiver Alarme weiter senken (Alsubaie & Alharbi, 2025, S. 75). Eine solche Erweiterung setzt allerdings einen ausreichenden Bestand an historischen Telemetriedaten voraus (Abieba et al., 2025, S. 273) und wäre als spätere Ausbaustufe zu betrachten. Vorrangig ist für frag.jetzt daher nicht eine weitere funktionale Ausweitung, sondern zunächst die Konsolidierung, Vervollständigung und empirische Absicherung der entwickelten Observability-Basis.

Literaturverzeichnis

- Abieba, O. A., Ubamadu, B. C., Hassan, Y. G., Owobu, W. O., Daraojimba, A. I., & Gbenle, P. (2025). Advanced Observability and Monitoring Practices for Enhancing Production Environment Reliability. <https://worldscientificnews.com/wp-content/uploads/2025/06/WSN-204-2025-270-296.pdf>
- Albuquerque, C., & Correia, F. F. (2025). Tracing and Metrics Design Patterns for Monitoring Cloud-native Applications. *arXiv preprint arXiv:2510.02991*. <https://doi.org/10.48550/arXiv.2510.02991>
- Alsubaie, N. S., & Alharbi, O. M. (2025). Exploring the Impact of Artificial Intelligence on the Evolution of Observability Tools. *International Journal of Technology Innovation and Management (IJTIM)*, 5(2), 74–81. <https://doi.org/10.54489/ijtim.v5i2.566>
- ARSnova. (n. d.). *frag.jetzt – Open-Source Q&A-Plattform*. GitLab. <https://gitlab.arsnova.eu/arsnova/frag.jetzt>
- Banerjee, R. (2025). A Comparative Analysis of System Monitoring Architecture: Evaluating Prometheus, Elk Stack, and Custom dashboards for performance and Scalability. *International Journal of Science and Research (IJSR)*, 14(9), 916–924. <https://doi.org/10.21275/SR25917095133>
- Bissig, N. (2024). *Unified Application Observability in Heterogeneous Distributed Systems* [Diss., Hochschule für angewandte Wissenschaften München]. https://opus4.kobv.de/opus4-hm/frontdoor/deliver/index/docId/590/file/Bissig_2024_MA.pdf
- Borges, M. C., Bauer, J., Werner, S., Gebauer, M., & Tai, S. (2024). Informed and assessable observability design decisions in cloud-native microservice applications. *2024 IEEE 21st International Conference on Software Architecture (ICSA)*, 69–78. <https://doi.org/10.1109/ICSA59870.2024.00015>
- Dasari, H. (2025). SITE RELIABILITY ENGINEERING PRACTICES FOR ERROR BUDGET MANAGEMENT IN LARGE-SCALE SYSTEMS. *International Journal of Applied Mathematics*, 38(5s), 991–1001. <https://doi.org/10.12732/ijam.v38i5s.366>
- Elastic. (n. d.). *Use OpenTelemetry with Elastic APM*. Elastic Docs. <https://www.elastic.co/docs/solutions/observability/apm/opentelemetry>
- Elastic. (2024). *FAQ on Software Licensing*. Elastic. <https://www.elastic.co/pricing/faq/licensing>
- Ewaschuk, R. (2016). Monitoring Distributed Systems. In B. Beyer, C. Jones, J. Petoff & N. R. Murphy (Hrsg.), *Site Reliability Engineering: How Google Runs Production Systems* (S. 55–66). O'Reilly Media. <https://sre.google/sre-book/monitoring-distributed-systems/>
- Faseeha, U., Jamil Syed, H., Samad, F., Zehra, S., & Ahmed, H. (2025). Observability in Microservices: An In-Depth Exploration of Frameworks, Challenges, and Deployment Paradigms. *IEEE Access*, 13, 72011–72039. <https://doi.org/10.1109/ACCESS.2025.3562125>

- Garimilla, M. (2024). Designing Tomorrow's Observability: A Software Architect's Guide to Building Effective Monitoring Solutions'. *International Journal for Research in Applied Science and Engineering Technology*, 12, 155–63. <https://doi.org/10.22214/ijraset.2024.64151>
- Giamattei, L., Guerriero, A., Pietrantuono, R., Russo, S., Malavolta, I., Islam, T., Dinga, M., Koziolk, A., Singh, S., Armbruster, M., et al. (2024). Monitoring tools for DevOps and microservices: A systematic grey literature review. *Journal of Systems and Software*, 208, 111906. <https://doi.org/10.1016/j.jss.2023.111906>
- Grafana Labs. (n.d. a). *Set up for tracing*. Grafana Tempo documentation. <https://grafana.com/docs/tempo/latest/set-up-for-tracing/>
- Grafana Labs. (n.d. b). *Understand labels*. Grafana Loki documentation. <https://grafana.com/docs/loki/latest/get-started/labels/>
- Grafana Labs. (2025). *Configure Grafana*. Grafana Labs. <https://grafana.com/docs/grafana/latest/setup-grafana/configure-grafana/>
- Gudimetla, S. (2025). Transitioning to Open-Source Observability: A Cost-Benefit Analysis and Migration Framework. *Journal of Computer Science and Technology Studies*, 7(12), 495–512. <https://doi.org/10.32996/jcsts.2025.7.12.55>
- Hallur, J. (2024). From Monitoring to Observability: Enhancing System Reliability and Team Productivity. *International Journal of Science and Research (IJSR)*, 13(10), 602–606. <https://doi.org/10.21275/SR241004083612>
- Jalalvand, F., Baruwat Chhetri, M., Nepal, S., & Paris, C. (2024). Alert prioritisation in security operations centres: A systematic survey on criteria and methods. *ACM Computing Surveys*, 57(2), 1–36. <https://doi.org/10.1145/3695462>
- Kosińska, J., Baliś, B., Konieczny, M., Malawski, M., & Zieliński, S. (2023). Toward the observability of cloud-native applications: The overview of the state-of-the-art. *IEEE Access*, 11, 73036–73052. <https://doi.org/10.1109/ACCESS.2023.3281860>
- Li, B., Peng, X., Xiang, Q., Wang, H., Xie, T., Sun, J., & Liu, X. (2022). Enjoy your observability: an industrial survey of microservice tracing and analysis. *Empirical Software Engineering*, 27(1), 25. <https://doi.org/10.1007/s10664-021-10063-9>
- Madupati, B. (2023). Observability in Microservices Architectures: Leveraging Logging, Metrics, and Distributed Tracing in Large-Scale Systems. *European Journal of Advances in Engineering and Technology*, 10(11), 24–31. <https://doi.org/10.5281/zenodo.13951033>
- Mahida, A. (2023). Enhancing observability in distributed systems-a comprehensive review. *Journal Of Mathematical & Computer Applications. Src/Jmca-166.*, 135, 2–4. [https://doi.org/10.47363/JMCA/2023\(2\)135](https://doi.org/10.47363/JMCA/2023(2)135)
- Mistry, h. (2025). The future of reliability engineering: Integrating next-gen observability into cloudnative infrastructures. *World Journal of Advanced Engineering Technology and Sciences*. <https://doi.org/10.30574/wjaets.2025.16.3.1327>

- Niedermaier, S., Koetter, F., Freymann, A., & Wagner, S. (2019). On Observability and Monitoring of Distributed Systems – An Industry Interview Study. *arXiv preprint arXiv:1907.12240*. <https://doi.org/10.48550/arXiv.1907.12240>
- Pappula, K. K., Anasuri, S., & Rusum, G. P. (2021). Building Observability into Full-Stack Systems: Metrics That Matter. *International Journal of Emerging Research in Engineering and Technology*, 2(4), 48–58. <https://doi.org/10.63282/3050-922X.IJERET-V2I4P106>
- Prometheus Authors. (n. d.). *Monitoring Docker container metrics using cAdvisor*. The Linux Foundation. <https://prometheus.io/docs/guides/cadvisor/>
- Sakinala, K. (2025). Monitoring and Observability for Cloud-Native Applications. *Journal of Computer Science and Technology Studies*, 7(8), 101–115. <https://al-kindipublishers.org/index.php/jcsts/article/view/10459>
- Soldani, J., & Brogi, A. (2022). Anomaly Detection and Failure Root Cause Analysis in (Micro) Service-Based Cloud Applications: A Survey. *ACM Comput. Surv.*, 55(3). <https://doi.org/10.1145/3501297>
- Sundström, J. (2025). Finding Faults Faster: Improving Error Tracing in Reactive Monitored Systems with Distributed Tracing Tools. <https://www.diva-portal.org/smash/record.jsf?dswid=1070&pid=diva2%3A1964604>
- Tzanettis, I., Androna, C.-M., Zafeiropoulos, A., Fotopoulou, E., & Papavassiliou, S. (2022). Data fusion of observability signals for assisting orchestration of distributed applications. *Sensors*, 22(5), 2061. <https://doi.org/10.3390/s22052061>
- Usman, M., Ferlin, S., Brunstrom, A., & Taheri, J. (2022). A survey on observability of distributed edge & container-based microservices. *IEEE Access*, 10, 86904–86919. <https://doi.org/10.1109/ACCESS.2022.3193102>
- Vinnakota, K., & Kolla, M. (2025). Creating Effective Alerts for Monitoring Distributed Systems. *International Journal of Computer Trends and Technology*, 73, 172–178. <https://doi.org/10.14445/22312803/IJCTT-V73I5P122>

A C4-Container-Diagramm in Vollansicht

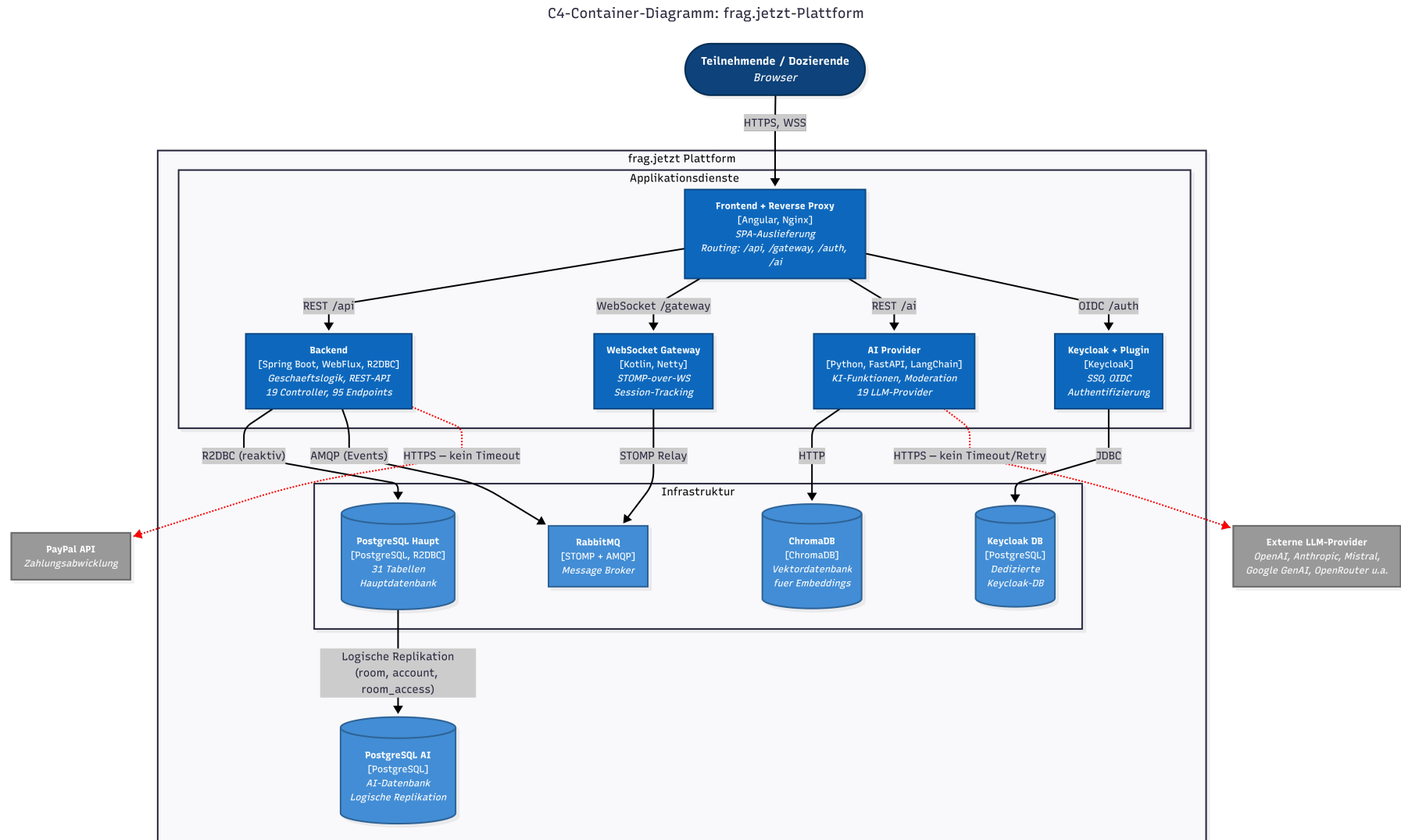


Abbildung A.1: C4-Container-Diagramm der frag.jetzt-Architektur in Vollseitenansicht. Quelle: Eigene Darstellung.

B OpenTelemetry Collector Konfiguration

Alle nachfolgenden Konfigurationsdateien, Alert-Regeln und Runbooks sind auch im öffentlichen Repository verfügbar.¹ Die folgende Datei zeigt die vollständige Konfiguration des OpenTelemetry Collector (Version 0.120.0), wie sie auf dem Staging-Server von frag.jetzt eingesetzt wird. Die Konfiguration definiert die Empfangs-, Verarbeitungs- und Exportpipelines für alle drei Telemetriearten.

Der `filter/traces`-Processor (Zeilen 16–28) ist die zentrale projektspezifische Anpassung. Er verwirft acht namentlich benannte Span-Typen, die von periodischen Hintergrundprozessen erzeugt werden und keinen diagnostischen Wert für die Fehlersuche besitzen (vgl. Abschnitt 7.5.3). Dieser Ansatz ersetzt ein probabilistisches Sampling durch eine inhaltlich begründete Selektion.

```
1 receivers:
2   otlp:
3     protocols:
4       grpc:
5         endpoint: 0.0.0.0:4317
6       http:
7         endpoint: 0.0.0.0:4318
8
9 processors:
10  batch:
11    send_batch_size: 1024
12    timeout: 5s
13  resource:
14    attributes:
15      - key: deployment.environment
16        value: staging
17        action: upsert
18  memory_limiter:
19    check_interval: 5s
20    limit_mib: 200
21    spike_limit_mib: 50
22  filter/traces:
23    error_mode: ignore
24    traces:
25      span:
26        - 'name == "ScheduledMessageSender.send"'
27        - 'name == "LivepollEventSource.sendAndUpdate"'
28        - 'name == "ScheduledMessageSender.sendToTopic"'
29        - 'name == "UserRegistryTask.run"'
30        - 'name == "MailScheduler.onTick"'
```

1 <https://github.com/LeonRau03/frag.jetzt-observability>

```

31     - 'name == "CommentNotificationService.sendNotifications"'
32     - 'name == "clientInboundChannel process"'
33     - 'name == "clientOutboundChannel process"'
34
35 exporters:
36   otlphttp/prometheus:
37     endpoint: http://prometheus:9090/api/v1/otlp
38     tls:
39       insecure: true
40   otlphttp/loki:
41     endpoint: http://loki:3100/otlp
42     tls:
43       insecure: true
44   otlp/tempo:
45     endpoint: tempo:4317
46     tls:
47       insecure: true
48   debug:
49     verbosity: basic
50     sampling_initial: 5
51     sampling_thereafter: 200
52
53 extensions:
54   health_check:
55     endpoint: 0.0.0.0:13133
56
57 service:
58   extensions: [health_check]
59   pipelines:
60     metrics:
61       receivers: [otlp]
62       processors: [memory_limiter, resource, batch]
63       exporters: [otlphttp/prometheus, debug]
64     logs:
65       receivers: [otlp]
66       processors: [memory_limiter, resource, batch]
67       exporters: [otlphttp/loki, debug]
68     traces:
69       receivers: [otlp]
70       processors: [memory_limiter, filter/traces, resource, batch]
71       exporters: [otlp/tempo, debug]
72   telemetry:
73     logs:
74       level: info

```

Listing B.1: OpenTelemetry Collector Konfiguration (config.yaml).

C Docker Compose Override für die Instrumentierung

Die Instrumentierung der JVM-basierten Dienste erfolgt ohne Änderungen am Applikationscode oder an den Docker-Images. Der OpenTelemetry Java Agent wird als Read-Only-Volume in die Container gemountet und über Umgebungsvariablen aktiviert. Die folgende Datei zeigt das Docker-Compose-Override-File, das die Produktionskonfiguration um die Observability-relevanten Einstellungen ergänzt (vgl. Abschnitt 7.2).

```
1 services:
2   frontend:
3     volumes:
4       - './logs/nginx:/var/log/nginx'
5       - './configs/favicon-jetzt.png:/var/www/favicon.png'
6       - './configs/nginx.conf:/etc/nginx/conf.d/default.conf'
7
8   backend:
9     volumes:
10      - './backend:/data/'
11      - '/opt/containers/staging_frag_jetzt/otel/
12        opentelemetry-javaagent.jar:/opt/otel-agent.jar:ro'
13      - '/opt/containers/staging_frag_jetzt/otel/
14        logback-spring.xml:/app/config/logback-spring.xml:ro'
15    environment:
16      - JAVA_TOOL_OPTIONS=-javaagent:/opt/otel-agent.jar
17      - OTEL_SERVICE_NAME=fragjetzt-backend
18      - OTEL_EXPORTER_OTLP_ENDPOINT=http://otel-collector:4317
19      - OTEL_EXPORTER_OTLP_PROTOCOL=grpc
20      - OTEL_TRACES_EXPORTER=otlp
21      - OTEL_METRICS_EXPORTER=otlp
22      - OTEL_LOGS_EXPORTER=otlp
23      - OTEL_RESOURCE_ATTRIBUTES=deployment.environment=staging
24      - OTEL_INSTRUMENTATION_LOGBACK_APPENDER_ENABLED=true
25      - OTEL_INSTRUMENTATION_LOG4J_APPENDER_ENABLED=true
26      - LOGGING_CONFIG=/app/config/logback-spring.xml
27      - LOGGING_LEVEL_ORG_SPRINGFRAMEWORK_WEB=DEBUG
28      - LOGGING_LEVEL_REACTOR_NETTY_HTTP_SERVER=DEBUG
29
30   ws-gateway:
31     volumes:
32      - '/opt/containers/staging_frag_jetzt/otel/
33        opentelemetry-javaagent.jar:/opt/otel-agent.jar:ro'
34    environment:
35      - JAVA_TOOL_OPTIONS=-javaagent:/opt/otel-agent.jar
36      - OTEL_SERVICE_NAME=fragjetzt-ws-gateway
37      - OTEL_EXPORTER_OTLP_ENDPOINT=http://otel-collector:4317
38      - OTEL_EXPORTER_OTLP_PROTOCOL=grpc
```

```
39     - OTEL_TRACES_EXPORTER=otlp
40     - OTEL_METRICS_EXPORTER=otlp
41     - OTEL_LOGS_EXPORTER=otlp
42     - OTEL_RESOURCE_ATTRIBUTES=deployment.environment=staging
43
44 networks:
45   fragjetzt:
46     name: fragjetzt-internal-network
47     driver: bridge
48     internal: false
```

Listing C.1: Docker Compose Override (`docker-compose.override.yml`).

D Alerting-Regeln

Die folgenden drei Alerting-Regeln wurden in Grafana Alerting konfiguriert und bilden die prototypische Umsetzung des in Abschnitt 6.5 konzipierten Alerting-Frameworks. Alle Regeln verwenden Multi-Signal-Logik, bei der ein primäres Fehlersignal erst in Kombination mit einer Mindest-Traffic-Rate als handlungsrelevant gewertet wird (vgl. Abschnitt 7.4).

D.1 Alert 1: User-facing Latency Degradation

Severity	Warning
Primäre Rolle	DevOps
Zugehöriges Runbook	Erhöhte Backend-Latenz (Anhang E.1)

Logik. Der Alert feuert, wenn *beide* Bedingungen gleichzeitig für mindestens drei Minuten erfüllt sind:

1. Die Backend-Antwortzeit im 95. Perzentil überschreitet 500 ms.
2. Die Anfragerate liegt über 0,1 req/s (kein Leerlauf).

PromQL – Condition A (Latenz).

```
histogram_quantile(0.95,
  sum(rate(
    http_server_request_duration_seconds_bucket{
      job="fragjetzt-backend"
    }[5m]
  )) by (1e)
) > 0.5
```

PromQL – Condition B (Mindest-Traffic).

```
sum(rate(
  traces_spanmetrics_calls_total{
    service="fragjetzt-backend",
    span_kind="SPAN_KIND_SERVER"
  }[5m]
)) > 0.1
```

Grafana-Konfiguration. Evaluation Group: golden-signals, Evaluation Interval: 1 min, Pending Period: 3 min, Condition: A AND B.

Annotations. *Summary:* Backend-Antwortzeit erhöht (p95 > 500 ms). *Description:* Die Antwortzeit des Backends im 95. Perzentil liegt seit über drei Minuten über 500 ms bei aktivem Traffic.

Nutzende können Verzögerungen bei Frageerstellung, Abstimmungen oder Raumnavigation wahrnehmen.

Labels. severity: warning, service: fragjetzt-backend, signal: latency, stakeholder: devops.

D.2 Alert 2: Backend Error Spike

Severity	Critical
Primäre Rolle	Developer
Zugehöriges Runbook	Erhöhte Fehlerquote / HTTP 5xx (Anhang E.2)

Logik. Der Alert feuert, wenn *beide* Bedingungen gleichzeitig für mindestens drei Minuten erfüllt sind:

1. Die HTTP-5xx-Fehlerrate des Backends überschreitet 5 %.
2. Die Anfragerate liegt über 0,1 req/s.

PromQL – Condition A (Fehlerrate).

```
(
  sum(rate(
    http_server_request_duration_seconds_count{
      job="fragjetzt-backend",
      http_response_status_code=~"5.."
    }[5m]
  ))
  /
  clamp_min(sum(rate(
    http_server_request_duration_seconds_count{
      job="fragjetzt-backend"
    }[5m]
  )), 1e-9)
) > 0.05
```

PromQL – Condition B (Mindest-Traffic).

```
sum(rate(
  http_server_request_duration_seconds_count{
    job="fragjetzt-backend"
  }[5m]
)) > 0.1
```

Grafana-Konfiguration. Evaluation Group: golden-signals, Evaluation Interval: 1 min, Pending Period: 3 min, Condition: A AND B.

Annotations. *Summary:* Backend-Fehlerrate kritisch erhöht (5xx > 5%). *Description:* Mehr als 5 % der Backend-Anfragen liefern HTTP-5xx-Fehler bei aktivem Traffic. Betroffene Endpunkte können über die Errors-Sektion des Dashboards und über Tempo (Trace-Suche, Status Error) identifiziert werden.

Labels. severity: critical, service: fragjetzt-backend, signal: errors, stakeholder: developer.

D.3 Alert 3: Infrastructure Saturation with Service Impact

Severity	Warning
Primäre Rolle	DevOps
Zugehöriges Runbook	Infrastruktur-Sättigung (DB / Queue / Host) (Anhang E.3)

Logik. Der Alert feuert, wenn *eine* der folgenden Variantenbedingungen für mindestens fünf Minuten erfüllt ist:

Variante A (Datenbank-Sättigung): PostgreSQL aktive Verbindungen > 70 **und** Backend p95-Latenz > 200 ms.

Variante B (Queue-Rückstau): RabbitMQ Queue Messages > 1000.

PromQL – Condition A1 (DB Connections).

```
sum(pg_stat_activity_count{
  job="postgres", state="active"
}) > 70
```

PromQL – Condition A2 (Latenz-Bestätigung).

```
histogram_quantile(0.95,
  sum(rate(
    http_server_request_duration_seconds_bucket{
      job="fragjetzt-backend"
    }[5m]
  )) by (le)
) > 0.2
```

PromQL – Condition B1 (Queue Depth).

```
sum(rabbitmq_queue_messages{job="rabbitmq"}) > 1000
```

Grafana-Konfiguration. Evaluation Group: infrastructure, Evaluation Interval: 1 min, Pending Period: 5 min, Condition: (A1 AND A2) OR B1.

Annotations. *Summary:* Infrastruktur-Sättigung mit Servicewirkung. *Description:* Infrastrukturressourcen (Datenbankverbindungen oder Message Queue) nähern sich ihren Kapazitätsgrenzen und gleichzeitig zeigen Servicemetriken eine Verschlechterung. Ohne Gegenmaßnahmen kann die Störung auf weitere Dienste kaskadieren.

Labels. severity: warning, service: infrastructure, signal: saturation, stakeholder: devops.

E Runbooks

Die folgenden Runbooks folgen der in Abschnitt 6.5 beschriebenen Fünf-Felder-Struktur und ergänzen die Alerting-Regeln aus Anhang D um strukturierte Handlungsanweisungen. Jedes Runbook gliedert sich in Symptombeschreibung, relevante Beobachtungssignale, Sofortmaßnahmen, Diagnosepfade und Eskalationsregeln.

E.1 Runbook: Erhöhte Backend-Latenz

Incident-Typ User-facing Latency Degradation

Trigger Alert „Backend p95-Latenz > 500 ms bei aktivem Traffic“ feuert seit ≥ 3 Minuten.

Nutzerwirkung. Verzögerungen bei interaktiven Aktionen wie Frageerstellung, Abstimmungen und Raum-Navigation. Bei Lehrveranstaltungen mit synchroner Nutzung sind viele Teilnehmende gleichzeitig betroffen.

Signal	Dashboard-Panel	Erwartetes Verhalten
Backend p95/p99 Latenz	Latency → Backend HTTP Latency	Anstieg über 500 ms
Request Rate	Traffic → Request Rate by Service	Aktiver Traffic (> 0,1 req/s)
Latenz pro Endpunkt	Latency → Latency by Endpoint	Einzelne Endpunkte auffällig
DB Connection Count	Infrastructure → PostgreSQL Active Conn.	Möglicherweise erhöht
JVM Heap / GC	Saturation → JVM Heap / GC Pause	Möglicherweise erhöht

Relevante Signale.

Sofortmaßnahmen (0–5 Minuten). *DevOps (primäre Rolle):*

1. Dashboard öffnen: Golden Signals → Latency-Sektion prüfen.
2. Scope eingrenzen: Betrifft die Latenz alle Endpunkte oder nur bestimmte? Panel „Latency by Endpoint (p95)“ prüfen.
3. Infrastruktur prüfen: Host-Ressourcen (CPU > 80 %? Memory > 90 %?), PostgreSQL (Verbindungen nahe 100?), RabbitMQ (Queue-Rückstau?).

4. Container-Status prüfen:

```
sudo docker ps --format "table {{.Names}}\t{{.Status}}" \
| grep fragjetzt
sudo docker stats --no-stream | head -15
```

Developer (sekundäre Rolle):

1. Trace-Analyse: Grafana → Explore → Tempo → Search. Filter: Service = **fragjetzt-backend**, Duration > 500 ms. Span-Waterfall öffnen und prüfen, wo der Request die meiste Zeit verbringt.
2. DB-Spans prüfen: R2DBC-Spans (SELECT/UPDATE) mit Laufzeit > 50 ms identifizieren.
3. Logs korrelieren: Vom langsamen Trace über die Funktion *Logs for this span* zu den zugehörigen Loki-Logs navigieren.

Diagnosepfad (5–15 Minuten). *Mögliche Ursache 1: Datenbankengpass.* Indikator: DB-Spans im Trace > 50 ms, PostgreSQL Connections > 70. Prüfung:

```
sudo docker exec fragjetzt-postgres psql -U fragjetzt -c \
"SELECT state, count(*) FROM pg_stat_activity
GROUP BY state;"
sudo docker exec fragjetzt-postgres psql -U fragjetzt -c \
"SELECT pid, now()-query_start AS duration, query
FROM pg_stat_activity WHERE state='active'
ORDER BY duration DESC LIMIT 5;"
```

Maßnahme: Langlaufende Queries identifizieren und gegebenenfalls terminieren.

Mögliche Ursache 2: JVM-Ressourcenerschöpfung. Indikator: GC Pause Duration steigt, Heap nahe Limit. Maßnahme: Backend-Container neu starten.

```
cd /opt/containers/staging_frag_jetzt/\
frag.jetzt-docker-orchestration
sudo docker compose restart backend
```

Mögliche Ursache 3: Externe Abhängigkeit (Keycloak, KI-Dienst). Indikator: Trace zeigt langen ausgehenden HTTP-Span. Prüfung: Span-Details im Trace, Ziel-URL und Dauer. Maßnahme: Betroffenen Upstream-Dienst prüfen oder Request-Pfad isolieren.

Eskalation. Wenn die Latenz nach 15 Minuten nicht sinkt: Hauptentwickler kontaktieren. Bei Totalausfall: Proxy-Level-Wartungsseite aktivieren.

Kriterium für Entwarnung. Backend p95-Latenz unter 200 ms für mindestens 5 Minuten, keine auffälligen Endpunkte im Latenz-Panel, PostgreSQL Connections im Normbereich (< 30 aktive).

E.2 Runbook: Erhöhte Fehlerquote / HTTP 5xx

Incident-Typ Backend Error Spike

Trigger Alert „HTTP-5xx-Rate > 5 % bei aktivem Traffic“ feuert seit ≥ 3 Minuten.

Nutzerwirkung. Nutzende erhalten Fehlermeldungen bei Interaktionen mit der Plattform. Betroffene Funktionen können Frageerstellung, Abstimmungen, Login oder KI-gestützte Features sein. Bei synchroner Nutzung führen sichtbare Fehler zu Vertrauensverlust und Unterbrechung des Veranstaltungsablaufs.

Signal	Dashboard-Panel	Erwartetes Verhalten
HTTP 5xx Rate	Errors → HTTP 5xx Rate (Backend)	Über 5 %
Error Rate by Service	Errors → Error Rate by Service	Backend-Linie steigt
Top Error Spans	Errors → Top Error Spans (5 min)	Betroffene Operationen sichtbar
Request Rate	Traffic → Request Rate by Service	Aktiver Traffic bestätigt
Backend Logs	Loki → {service_name="fragjetzt-backend"}	Exception-Stacktraces

Relevante Signale.

Sofortmaßnahmen (0–5 Minuten). *Developer (primäre Rolle):*

1. Fehlerquelle identifizieren: Golden Signals Dashboard → Errors-Sektion. Panel „Top Error Spans (5 min)“ prüfen und betroffene Funktionsbereiche eingrenzen (Q&A, Voting, Auth, KI).
2. Fehlerhafte Traces öffnen: Grafana → Explore → Tempo → Search (Service = `fragjetzt-backend`, Status = Error). Span-Waterfall analysieren: HTTP Status Code und Exception Message prüfen.
3. Logs korrelieren: Vom fehlerhaften Trace zu Loki navigieren. Exception-Stacktraces lesen und Fehlermuster identifizieren.

DevOps (sekundäre Rolle):

1. Container-Gesundheit prüfen:

```
sudo docker ps --format "table {{.Names}}\t{{.Status}}" \
| grep fragjetzt
sudo docker logs fragjetzt-backend --since 5m 2>&1 \
| grep -i "error\|exception" | tail -20
```

2. Letzte Änderungen prüfen: Gab es ein kürzliches Deployment oder eine Konfigurationsänderung?
3. Abhängigkeiten prüfen: Sind PostgreSQL, RabbitMQ und Keycloak erreichbar?

Diagnosepfad (5–15 Minuten). *Mögliche Ursache 1: Datenbankverbindungsfehler.* Indikator: Error Spans enthalten `ConnectionException`, `R2dbcException` oder `TimeoutException`. Maßnahme: PostgreSQL-Container prüfen und gegebenenfalls neu starten.

Mögliche Ursache 2: Fehler in einem spezifischen Endpunkt. Indikator: Top Error Spans zeigt einen einzelnen Endpunkt. Maßnahme: Bug-Report erstellen, gegebenenfalls betroffene Funktion temporär deaktivieren.

Mögliche Ursache 3: Externe Abhängigkeit nicht erreichbar. Indikator: Error Spans auf ausgehenden HTTP-Calls (Keycloak, PayPal, LLM-Provider). Maßnahme: Externen Dienst prüfen, bei Nichterreichbarkeit Graceful Degradation erwägen.

Mögliche Ursache 4: Ressourcenerschöpfung. Indikator: JVM Heap nahe Limit, OOM-Fehler in Logs. Maßnahme: Backend-Container neu starten.

Escalation. Wenn die Fehlerrate nach Backend-Restart nicht sinkt: Hauptentwickler kontaktieren. Wenn mehrere Endpunkte gleichzeitig betroffen sind: systemweites Problem vermuten und alle Abhängigkeiten prüfen.

Kriterium für Entwarnung. HTTP 5xx Rate unter 1 % für mindestens 5 Minuten, keine neuen Error Spans in der Top-Error-Tabelle, Backend-Logs ohne wiederkehrende Exceptions.

E.3 Runbook: Infrastruktur-Sättigung (DB / Queue / Host)

Incident-Typ	Database / Queue Saturation with Service Impact
Trigger	Alert „Infrastruktur-Sättigung mit Servicewirkung“ feuert, wenn eine der folgenden Bedingungen seit ≥ 5 Minuten erfüllt ist: (a) PostgreSQL aktive Verbindungen > 70 und Backend p95-Latenz > 200 ms, oder (b) RabbitMQ Queue Messages > 1000 dauerhaft.

Nutzerwirkung. Verzögerungen oder Ausfälle bei datenbankgestützten Funktionen (Frageerstellung, Abstimmungen, Raum-Verwaltung) oder bei Echtzeit-Updates über WebSocket (Live-Voting, Teilnehmerzählung). Die Störung kann von einer einzelnen Funktion auf weitere kaskadieren, wenn die Ressourcenerschöpfung zunimmt.

Signal	Dashboard-Panel	Erwartetes Verhalten
PostgreSQL Connections	Infrastructure → PostgreSQL Active Conn.	> 70 aktive
RabbitMQ Queue Depth	Infrastructure → RabbitMQ Messages	> 1000 oder steigend
Backend p95 Latenz	Latency → Backend HTTP Latency	Korreliert erhöht
Host Memory / CPU	Infrastructure → Host Memory & CPU	Möglicherweise erhöht
JVM Thread Count	Saturation → JVM Thread Count	Möglicherweise erhöht

Relevante Signale.

Sofortmaßnahmen (0–5 Minuten). *DevOps (primäre Rolle):*

1. Sättigungsquelle identifizieren: Dashboard → Infrastructure-Sektion. PostgreSQL Panel, RabbitMQ Panel und Host Panel prüfen.
2. Container-Ressourcen prüfen:

```
sudo docker stats --no-stream \
  --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}" \
  | head -15
```

3. PostgreSQL-Verbindungen im Detail prüfen:

```
sudo docker exec fragjetzt-postgres psql -U fragjetzt -c \
  "SELECT state, count(*),
    avg(now()-query_start) as avg_duration
  FROM pg_stat_activity
  WHERE datname = 'fragjetzt'
  GROUP BY state ORDER BY count DESC;"
```

4. RabbitMQ-Queue-Status prüfen:

```
sudo docker exec fragjetzt-rabbitmq \
  rabbitmqctl list_queues name messages consumers
```

Developer (sekundäre Rolle):

1. Langlaufende Queries identifizieren:

```
sudo docker exec fragjetzt-postgres psql -U fragjetzt -c \
  "SELECT pid, now()-query_start AS duration,
    left(query, 80) as query
  FROM pg_stat_activity
  WHERE state='active' AND datname='fragjetzt'
  ORDER BY duration DESC LIMIT 10;"
```

2. Trace-Korrelation: Grafana → Tempo → Search. Traces mit langen DB-Spans (> 100 ms) suchen und prüfen, ob ein bestimmter Endpunkt unverhältnismäßig viele Verbindungen verbraucht.
3. Vote-Trigger prüfen (bekannter Engpass, vgl. Risikopunkt R2): Sind viele parallele Abstimmungen aktiv? Zeigen Traces Row-Level Contention auf der `comment`-Tabelle?

Diagnosepfad (5–15 Minuten). *Mögliche Ursache 1: Connection-Pool-Erschöpfung.* Indikator: Aktive Verbindungen > 70, idle Verbindungen niedrig. Maßnahme: Idle-in-Transaction-Verbindungen beenden:

```
sudo docker exec fragjetzt-postgres psql -U fragjetzt -c \
  "SELECT pg_terminate_backend(pid)
  FROM pg_stat_activity
  WHERE state = 'idle in transaction'
  AND query_start < now() - interval '5 minutes';"
```

Mögliche Ursache 2: RabbitMQ-Consumer-Rückstand. Indikator: Queue-Tiefe steigt, Consumer-Zahl niedrig oder null. Maßnahme: WS-Gateway-Container prüfen und gegebenenfalls neu starten (`docker compose restart ws-gateway`).

Mögliche Ursache 3: Host-Ressourcenverdrängung (R5). Indikator: Host CPU > 80 % oder Memory > 90 %, mehrere Container betroffen. Maßnahme: Nicht-kritische Container temporär stoppen (z. B. AI Provider).

Mögliche Ursache 4: Lastspitze durch Lehrveranstaltung. Indikator: Request-Rate und WebSocket-Verbindungen gleichzeitig hoch. Maßnahme: Situation beobachten. Die Sättigung sinkt nach Veranstaltungsende. Bei wiederholtem Auftreten Kapazitätsplanung einleiten.

Eskalation. Wenn PostgreSQL-Verbindungen nach Bereinigung erneut ansteigen: Hauptentwickler einbeziehen (möglicherweise Connection-Leak im Code). Wenn Host-Ressourcen dauerhaft erschöpft: Infrastruktur-Skalierung besprechen.

Kriterium für Entwarnung. PostgreSQL aktive Verbindungen unter 30, RabbitMQ Queue-Tiefe unter 100, Backend p95-Latenz unter 200 ms, Host CPU unter 60 % und Memory unter 75 %.

F Golden-Signals-Dashboard

Das Dashboard ("frag.jetzt - Golden Signals") umfasst 17 Panels in fünf Sektionen. Tabelle F.1 gibt eine Übersicht über alle Panels, ihre primäre Datenquelle und die zugeordnete Golden-Signal-Dimension.

Tabelle F.1: Panelübersicht des Golden-Signals-Dashboards. Quelle: Eigene Darstellung.

Sektion	Panel	Datenquelle
Latency	Backend HTTP Latency (p50/p95/p99)	Prometheus (OTel)
	Latency by Endpoint (p95)	Prometheus (Span-Metriken)
Traffic	Request Rate by Service	Prometheus (Span-Metriken)
	Top Endpoints by Request Rate (Backend)	Prometheus (Span-Metriken)
Errors	Error Rate by Service (Span-based)	Prometheus (Span-Metriken)
	HTTP 5xx Rate (Backend)	Prometheus (OTel)
	Top Error Spans (5 min)	Prometheus (Span-Metriken)
Saturation	JVM Heap Used	Prometheus (OTel)
	JVM Thread Count	Prometheus (OTel)
	JVM CPU Utilization	Prometheus (OTel)
Infrastr.	PostgreSQL Active Connections	Prometheus (Exporter)
	Nginx Connections	Prometheus (Exporter)
	Nginx Accepted vs Handled	Prometheus (Exporter)
	GC Pause Duration	Prometheus (OTel)
	RabbitMQ Messages	Prometheus (Plugin)
	Host Memory & CPU	Prometheus (Node Exp.)
	Service Map (Tempo)	Tempo

Nachfolgend zeigt Listing F.1 einen repräsentativen Auszug aus der Dashboard-JSON-Definition. Das Panel „Backend HTTP Latency“ illustriert die PromQL-Queries für die drei Perzentil-Zeitreihen.

```

1 {
2   "title": "Backend HTTP Latency (p50 / p95 / p99)",
3   "type": "timeseries",
4   "datasource": { "type": "prometheus", "uid": "prometheus" },
5   "fieldConfig": {
6     "defaults": { "unit": "s" }
7   },
8   "targets": [
9     {
10      "expr": "histogram_quantile(0.50,
11        sum(rate(http_server_request_duration_seconds_bucket
12          {job=\"fragjetzt-backend\"}
13            [$_rate_interval])) by (le))",
14      "legendFormat": "p50"
15    },
16    {
17      "expr": "histogram_quantile(0.95,
18        sum(rate(http_server_request_duration_seconds_bucket
19          {job=\"fragjetzt-backend\"}
20            [$_rate_interval])) by (le))",
21      "legendFormat": "p95"
22    },
23    {
24      "expr": "histogram_quantile(0.99,
25        sum(rate(http_server_request_duration_seconds_bucket
26          {job=\"fragjetzt-backend\"}
27            [$_rate_interval])) by (le))",
28      "legendFormat": "p99"
29    }
30  ]
31 }

```

Listing F.1: Auszug aus der Dashboard-JSON: Panel „Backend HTTP Latency“ (gekürzt).

Die vollständige JSON-Definition (1761 Zeilen) ist im öffentlichen Observability-Repository des Projekts verfügbar.¹

¹ <https://github.com/LeonRau03/frag.jetzt-observability>

G Verzeichnis der KI-Nutzung

G.1 Erklärung zur Nutzung von KI-basierten Werkzeugen

Bei der Erstellung dieser Arbeit wurden KI-basierte Werkzeuge unterstützend eingesetzt. Der Einsatz umfasste vier primäre Anwendungsbereiche: (1) Literaturrecherche und Zusammenfassung von Forschungsarbeiten, (2) Strukturierungs- und Formulierungsverbesserungen im Fließtext, (3) Unterstützung bei der technischen Implementierung (Instrumentierung, Dashboard-Konfiguration, Deployment) und (4) Code-Exploration zur Architekturhebung in Kapitel 3.

Alle von KI-Systemen generierten Vorschläge wurden eigenständig geprüft, fachlich bewertet und bei Bedarf überarbeitet oder verworfen. Sämtliche inhaltlichen Entscheidungen, insbesondere die Auswahl und Bewertung von Quellen, die Interpretation der Ergebnisse sowie die endgültige Textfassung, liegen in der Verantwortung des Verfassers.

Tabelle G.1 dokumentiert die eingesetzten Werkzeuge, den jeweiligen Verwendungszweck und die betroffenen Kapitel.

G.2 Declaration on the Use of AI-Based Tools

During the preparation of this thesis, AI-based tools were utilized for supportive purposes. Their application encompassed four primary areas: (1) literature research and the summarization of research papers, (2) structural and phrasing improvements within the text, (3) support during the technical implementation (instrumentation, dashboard configuration, deployment), and (4) code exploration for the architectural analysis in Chapter 3.

All suggestions generated by AI systems were independently reviewed, technically evaluated, and either revised or discarded as deemed necessary. All content-related decisions, in particular the selection and evaluation of sources, the interpretation of the results, and the final wording of the text, remain the sole responsibility of the author.

Table G.1 documents the specific tools employed, their respective purposes, and the affected chapters.

Tabelle G.1: Verzeichnis der in dieser Arbeit genutzten KI-Werkzeuge. Quelle: Eigene Darstellung.				
KI-Tool	Zweck	Eingabe / Kontext	Generierte Ausgabe	Kapitel
Consensus	Validierung von Forschungsinformationen	Behauptungen und Fakten aus der Literaturrecherche	Bestätigungen oder Korrekturen basierend auf aggregierten Forschungsergebnissen	Kap. 2–5
Claude Opus 4.5/4.6	Konfigurationshilfe bei der Implementierung	OTel Collector Config, Docker Compose Files, Fehlermeldungen, Prometheus-Queries	Konfigurationsvorschläge, Debugging-Hinweise, PromQL-Korrekturen	Kap. 7
Claude Opus 4.6	Formatierung der Runbooks	Vorlage der Runbooks	Strukturierte Runbook-Entwürfe in LaTeX und Markdown	Anhang E
Claude Sonnet 4.6	Strukturierung und Formulierungsbesserungen	Gliederungsvorgaben, Feedback-Iterationen	Tabellenstrukturen, LaTeX-Formatierungen und Formulierungen für Fließtext	Kap. 1–10
Elicit	Suche nach relevanten Forschungsarbeiten	Forschungsfragen und Schlüsselbegriffe aus der Arbeit	Listen von relevanten Papers mit Kurzbeschreibungen	Kap. 2–5
Gemini 3 Pro	Zusammenfassung von Forschungsarbeiten	Abstracts oder einzelne Kapitel aktueller Forschungsarbeiten zu Observability	Kompakte Zusammenfassungen, thematische Einordnung und Relevanzbewertung	Kap. 2

Fortsetzung auf nächster Seite

Tabelle G.1 — Fortsetzung				
KI-Tool	Zweck	Eingabe / Kontext	Generierte Ausgabe	Kapitel
GitHub Copilot	Quellcode-Exploration für Architekturerhebung	Fragen zur Repository-Struktur, Controller-Anzahl, Kommunikationspfaden von frag.jetzt	Architekturübersichten, Abhängigkeitslisten, Konfigurationsextrakte	Kap. 3
OpenAI GPT-5.2	Formulierungsverbesserungen	Entwurf einzelner Absätze	Alternative Formulierungen und Umstrukturierungsvorschläge	Kap. 1–10
OpenAI GPT-5.3	Unterstützung bei Dashboard-Konzeption	Beschreibung des gewünschten Dashboard-Layouts und der Metriken	Vorschläge zu Panel-Struktur und PromQL-Queries	Kap. 6, 7

G.3 Copilot Prompts für die unterstützende Systemanalyse von frag.jetzt

- 1 Analysiere die Projektstruktur von frag.jetzt. Welche Module, Services und Hauptkomponenten gibt es? Erstelle eine tabellarische Uebersicht mit Modulname, Technologie/Framework und Hauptverantwortung.
- 2 Identifiziere alle externen HTTP-Aufrufe (REST-Clients, WebClients, Feign-Clients) im Spring-Boot-Backend. Welche externen Dienste werden aufgerufen, mit welchen Endpunkten, und gibt es Timeout- oder Retry-Konfigurationen?
- 3 Liste alle REST-Controller im Backend auf. Gruppiere sie nach fachlicher Funktion (z. B. Room-Management, Question-Management, Auth, AI-Service) und notiere die HTTP-Methoden und Pfade.
- 4 Gibt es bereits Observability-bezogene Konfigurationen? Suche nach: Micrometer, Actuator, OpenTelemetry, Prometheus, Logging-Frameworks (Logback, Log4j), Health-Check-Endpoints, Tracing-Bibliotheken.
- 5 Analysiere die Datenbankschicht. Welche Entities/Tabellen gibt es, welche Repositories werden verwendet, und gibt es komplexe Queries oder Stored Procedures, die als Performancerisiko gelten koennten?
- 6 Beschreibe die WebSocket-Kommunikation. Welche STOMP-Topics oder Channels gibt es, welche Nachrichten werden darueber ausgetauscht, und wie wird die Verbindung verwaltet?
- 7 Analysiere die Deployment-Konfiguration (docker-compose.yml, Kubernetes-Manifeste, CI/CD-Pipeline). Welche Container/Services werden deployt, wie sind sie vernetzt, und welche Ressourcenlimits sind konfiguriert?

Listing G.1: Copilot-Prompts zur Architekturanalyse

Erklärung der Selbstständigkeit

Hiermit versichere ich, die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt zu haben.

Frankfurt am Main, den 31. März 2026

Leon Rausch